אוניברסיטת אריאל בשומרון
המחלקה להנדסת חשמל ואלקטרוניקה

מערכת מנעול חכם מבוססת זיהוי פנים על גבי כרטיס PYNQ-Z2

פרויקט גמר לתואר B.Sc.

מוגש ע"י:

אסובאלאו בלאיי
ת.ז.:

מנחה: פרופ' רצאבי יואל

כ"ה אלול תשפ"ו 07 ספטמבר 2026

תקציר

בפרויקט זה תוכננה ומומשה מערכת מנעול חכם המבוססת על כרטיס הפיתוח PYNQ-Z2, זיהוי פנים, לוח מקשים לגיבוי, חיישני סביבה, חיישן מצב דלת ושליחת התראות בזמן אמת באמצעות Telegram. המערכת נועדה לאפשר בקרת כניסה חכמה ומאובטחת, שבה פתיחת המנעול בוצעה באופן אוטומטי לאחר זיהוי משתמש מורשה, או לחלופין באמצעות הקשת סיסמה תקינה בלוח מקשים.

המערכת שילבה בין חומרה שתוכננה ב־ FPGA לבין תוכנה שנכתבה בשפת Python. בצד החומרה תוכנן בקר מנוע צעד בשפת VHDL אשר שימש לשליטה במנוע צעד באמצעות דרייבר ULN2003. ובנוסף גם בלוח מקשים תוכנן בצד החומרתי. מנוע הצעד שימש לייצוג פעולת הפתיחה והנעילה של המנעול. בנוסף, חוברו למערכת חיישן תנועה מסוג PIR וחיישן אור דיגיטלי מסוג LM393 חיישן דלת מגנטי מסוג MC-38 לוח מקשים, מצלמת USB, ממסר, זמזם ונורות חיווי. רכיבי הקלט והפלט חוברו לכרטיס הפיתוח באמצעות ממשקי GPIO והשליטה בהם בוצעה מתוך סביבת Jupyter Notebookעל גבי מערכת PYNQ.

בצד התוכנה פותחה מערכת בקרה מרכזית אשר ניהלה את פעולת המנעול, קראה את מצב החיישנים, הפעילה את המנוע, קלטה נתונים מלוח המקשים, ביצעה צילום תמונה, הפעילה תהליכי גילוי וזיהוי פנים, ניהלה ניסיונות גישה שגויים ושלחה התראות למשתמש. גילוי הפנים בוצע באמצעות אלגוריתם Haar Cascade וזיהוי הפנים בוצע באמצעות אלגוריתם LBPH על בסיס מודל שאומן מראש עבור המשתמש המורשה.

אופן פעולת המערכת התבסס על זיהוי תנועה באזור הדלת. כאשר זוהתה תנועה, נבדק מצב התאורה בסביבה. במקרה שבו זוהתה חשכה, הופעל ממסר לצורך הדלקת תאורה לפני ביצוע הצילום. לאחר מכן בוצע צילום באמצעות מצלמת USB, והתמונה נותחה לצורך גילוי וזיהוי פנים. כאשר זוהה משתמש מורשה, נשלחה פקודת פתיחה לבקר מנוע הצעד, והמנעול נפתח. לאחר פתיחת המנעול בוצעה בדיקה רציפה של חיישן הדלת המגנטי MC-38 ורק לאחר שזוהתה סגירה פיזית של הדלת נשלחה פקודת נעילה למנוע הצעד. בדרך זו נמנעה נעילה אוטומטית כאשר הדלת עדיין הייתה פתוחה.

לוח המקשים שולב כדרך גיבוי לפתיחת המנעול. הזנת הסיסמה התקינה אפשרה פתיחה של המנעול גם ללא זיהוי פנים, ולאחר מכן בוצעה נעילה בהתאם למצב חיישן הדלת. בנוסף, שולב זמזם לצורך מתן משוב קולי בעת לחיצה על מקשי לוח המקשים ובמצבי מערכת שונים.

לצורך שיפור רמת האבטחה, מומש מנגנון לניהול ניסיונות גישה שגויים. במקרה של הזנת קוד שגוי בלוח המקשים, נספר ניסיון כושל. לאחר שלושה ניסיונות שגויים, בוצע צילום של אזור הכניסה, נשלחה תמונת החשוד ל־ Telegram, והמערכת נכנסה למצב נעילה זמנית. בנוסף, כישלונות בזיהוי פנים נספרו בנפרד, ורק לאחר שלושה כישלונות רצופים נשלחה התראה מתאימה. באופן זה נמנעה שליחת התראות מיותרות עבור כל כשל זיהוי בודד, תוך שמירה על יכולת תגובה לאירועים חריגים.

המערכת נבדקה באופן מעשי, ונמצא כי תתי־המערכות המרכזיות פעלו בצורה תקינה. נבדקו טעינת ה־ Overlay השליטה בממשקי GPIO הפעלת מנוע הצעד, קריאת לוח המקשים, הפעלת זמזם עבור כל לחיצה, זיהוי תנועה, בדיקת מצב תאורה, הפעלת ממסר, צילום תמונה, גילוי וזיהוי פנים, קריאת מצב חיישן הדלת MC-38, נעילה לאחר סגירת הדלת, שליחת הודעות ותמונות ל־ Telegram כתיבת לוגים לקובץ CSV והפעלה אוטומטית באמצעות system.

תוצאות הפרויקט הראו כי ניתן לממש מערכת מנעול חכם המבוססת על שילוב בין FPGA תוכנה משובצת, עיבוד תמונה, חיישנים ותקשורת רשת. המערכת סיפקה בקרת כניסה חכמה, אפשרות פתיחה באמצעות זיהוי פנים, אפשרות גיבוי באמצעות קוד, זיהוי מצב פיזי של הדלת, ניהול ניסיונות פריצה ושליחת התראות בזמן אמת. בכך הודגמה מערכת אבטחה משובצת בעלת פוטנציאל להרחבה עתידית ולשילוב במערכות בקרת כניסה מתקדמות.

הכרת תודה

ברצוני להביע תודה לכל הגורמים אשר סייעו, תמכו וליוו את תהליך ביצוע פרויקט הגמר.

ראשית, מובעת תודה מיוחדת למנחה הפרויקט, פרופ׳ יואל רצאבי, על אספקת הציוד האיכותי, הזמינות, הסבלנות והליווי לאורך שלבי העבודה. תרומתו המקצועית סייעה בגיבוש רעיון הפרויקט, בהבנת דרישות המערכת, בהתמודדות עם אתגרי התכנון ובמימוש מערכת משולבת חומרה ותוכנה.

בנוסף, מובעת תודה לסגל המחלקה להנדסת חשמל ואלקטרוניקה באוניברסיטת אריאל, אשר העניק לאורך שנות הלימודים ידע תאורטי ומעשי בתחומי האלקטרוניקה, המערכות הספרתיות, התכנות, התקשורת והמערכות המשובצות. ידע זה שימש בסיס חשוב לתכנון ולמימוש הפרויקט.

תודה נוספת מובעת לצוות המעבדות באוניברסיטת אריאל, על מתן סביבת עבודה לימודית, ציוד מתאים ותמיכה טכנית לאורך תקופת הלימודים. הניסיון המעשי שנרכש במסגרת המעבדות תרם רבות להבנת החיבור בין תכנון תאורטי לבין מימוש מעשי.

כמו כן, מובעת תודה למשפחה ולחברים על התמיכה, העידוד והסבלנות לאורך תקופת העבודה על הפרויקט. תמיכתם סייעה רבות בהתמודדות עם האתגרים, בעמידה בלוחות הזמנים ובהשלמת הפרויקט בצורה מיטבית.

לבסוף, מובעת תודה לכל מי שסייע באופן ישיר או עקיף בתהליך הלמידה, הפיתוח והבדיקות של המערכת. תרומתם הייתה משמעותית להשלמת פרויקט זה.

תוכן עניינים

1. מבוא **6**
2. מטרת הפרויקט **6**
3. דרישות המערכת **7**
4. רקע תיאורטי **8**
   - 4.1. כרטיס הפיתוח PYNQ-Z2 ..... **9**
     - *4.1.1. רכיב Zynq-7000* ***10***
     - *4.1.2. ממשק AXI* ***11***
     - *4.1.3. ממשק AXI GPIO* ***12***
     - *4.1.4. שימוש ב Overlay בסביבת PYNQ* ***13***
     - *4.1.5. סביבת PYNQ ו Jupyter Notebook* *13*
   - 4.2. מנוע צעד 28BYJ-48 ..... **14**
     - *4.2.1. דרייבר ULN2003* *15*
   - 4.3. חיישן תנועה PIR 16
   - 4.4. חיישן אור דיגיטלי LM393 17
   - 4.5. חיישן דלת מגנטי MC-38 18
   - 4.6. מצלמת USB מסוג LOGITECH BRIO 300 19
   - 4.7. לוח מקשים 4**X**4 20
   - 4.8. ממסר להפעלת תאורה 21
   - 4.9. זמזם 22
   - 4.10. נורות חיווי LED 22
   - 4.11. עיבוד תמונה 23
     - *4.11.1. ייצוג תמונה דיגיטלית* *23*
     - *4.11.2. ראייה ממוחשבת* *24*
     - *4.11.3. גילוי פנים וזיהוי פנים* *24*
     - *4.11.4. ספריית OpenCV* *25*
     - *4.11.5. אלגוריתם Haar Cascade* *25*
     - *4.11.6. אלגוריתם LBPH* *26*
   - 4.12. בוט טלגרם 27
   - 4.13. מערכת SYSTEMD 28
5. מימוש המערכת 29
   - 5.1. רשימת ציוד 30
   - 5.2. הפעולה הכללית של המערכת 31
   - 5.3. מצבי פעולה של המערכת 32
   - 5.4. סכמת בלוקים 32
6. ארכיטקטורת החומרה 33
   - 6.1. תרשים חיבורים של המערכת 33
   - 6.2. טבלת חיבורים של המערכת 34
7. מימוש החומרה 34
   - 7.1. בקר לוח המקשים 35
   - 7.2. תכנון בקר מנוע הצעד 40
     - *7.2.1. קומפוננט של מחלק התדר* *40*
     - *7.2.2. קומפונטה של סיבוב המנוע עם כיוון השעון* *41*
     - *7.2.3. קומפונטה של המנוע נגד כיוון השעון* *43*
     - *7.2.4. קוד הראשי של מנוע הצעד* *46*
   - 7.3. השלבים ליצירת המערכת ב VIVADO 47
8. מימוש התוכנה 52
   - 8.1. ייבוא ספריות והגדרת סביבת העבודה 53
   - 8.2. הגדרת פרמטריים מרכזיים 54
   - 8.3. הגדרת מערכת ההתראות על ידי טלגרם 54

8.4. הגדרת קבועי וזמני המערכת 55
8.5. מיפוי הביטים של רכיבי הפלט והקלט 56
8.6. הגדרת מכונת המצבים של המערכת באמצעות ENUM 57
8.7. הגדרת פונקציות עזר כלליות 57
8.8. הגדרת המחלקה הראשית SMARTLOCKSYSTEM ופונקציית האתחול __INIT__ 58
8.9. ניהול נתיבי הקבצים 59
8.10. ניהול קובץ הלוג 59
8.11. מנגנון ההודעות של TELEGRAM 60
8.12. השליחה בפועל ל־ TELEGRAM BOT API 62
8.13. טעינת ה־ OVERLAY ואתחול ממשקי ה־AXI GPIO 64
8.14. זיהוי רוחב ממשקי ה־ GPIO, יצירת מסכות הביטים ואתחול יציאות המערכת 66
8.15. קריאה וכתיבה לממשקי ה־ GPIO 68
8.16. שליטה בנורות החיווי, בממסר ובזמזם 69
8.17. בקרת מנוע הצעד 71
8.18. התחלת מחזור הדלת והמתנה לשינוי במצב החיישן 73
8.19. השהיית הנעילה וטיפול במצבי TIMEOUT של הדלת 74
8.20. ניהול מחזור הפתיחה והנעילה 76
8.21. טעינת מודל גילוי הפנים ומודל זיהוי הפנים 77
8.22. צילום תמונה מהמצלמה 78
8.23. גילוי וזיהוי פנים 79
8.24. קריאת לוח המקשים וטיפול בסיסמה 81
8.25. איפוס מנגנון האבטחה ובדיקת מצב LOCKOUT 83
8.26. טיפול בניסיון גישה כושל והפעלת LOCKOUT 84
8.27. טיפול בכשלי זיהוי פנים 87
8.28. טיפול בגישה מוצלחת 88
8.29. ניתוב תוצאת הגישה 89
8.30. טיפול באירוע תנועה ושילוב חיישן האור, המצלמה וזיהוי הפנים 89
8.31. הלולאה הראשית של המערכת 92
8.32. כיבוי בטוח של המערכת 94
8.33 פונקציית הפעלת המערכת 95
8.34. סיכום מימוש התוכנה 95

9. ניסויים ובדיקות 97

9.1. בדיקות וסימולציות בסביבת VIVADO 97
9.1.1. בדיקת מחלק התדר 97
9.1.2. בדיקת סיבוב המנוע עם כיוון השעון 98
9.1.3. בדיקת סיבוב המנוע נגד כיוון השעון 98
9.1.4. בדיקת מנוע הצעד באופן מלא 99
9.2. בדיקת לוח המקשים 100
9.3. בדיקת כלל המערכת בסביבת JUPYTER 101

10. תוצאות ומסקנות 102

10.1. תוצאות 102
10.2. מסקנות 106

11. סיכום 107

12. רשימות 107

12.1. רשימת טבלאות 107
12.2. רשימת איורים 107

13. ביבליוגרפיה 108

1. מבוא

תחום האבטחה הפיזית נחשב לאחד התחומים המרכזיים בשמירה על רכוש, פרטיות ובטיחות. לאורך השנים נעשה שימוש במנגנוני נעילה מכניים, כגון מפתחות, מנעולים ובריחים, אשר סיפקו פתרון בסיסי לבקרת כניסה. עם זאת, בעקבות התפתחות הטכנולוגיה והעלייה בצורך בבקרה חכמה, החלו להשתלב במערכות האבטחה רכיבים אלקטרוניים, מערכות משובצות מחשב, חיישנים, מצלמות ואמצעי תקשורת.

מערכות נעילה חכמות נועדו לשפר את רמת האבטחה ואת נוחות השימוש ביחס למנעולים מכניים רגילים. מערכות אלו מאפשרות לבצע פתיחה ונעילה באופן מבוקר, לזהות משתמשים מורשים, לתעד אירועים חריגים, ולהתריע בזמן אמת על ניסיונות גישה לא מורשים. בכך מתאפשרת בקרה יעילה יותר על הכניסה למרחב מוגן, תוך הפחתת התלות במפתח פיזי אשר עלול ללכת לאיבוד, להיגנב או להיות מועתק.

אחת הדרכים לשיפור בקרת הכניסה היא שימוש באמצעי זיהוי ביומטריים. זיהוי פנים הוא אחד האמצעים הנפוצים בתחום זה, מאחר שהוא מאפשר לזהות אדם על פי מאפייני פניו ללא צורך במגע פיזי עם המערכת. שילוב טכנולוגיה זו במערכת נעילה מאפשר פתיחה אוטומטית עבור משתמש מורשה, תוך שמירה על תהליך כניסה טבעי ונוח.

במקביל, מערכות אבטחה מודרניות אינן מסתפקות רק בפעולת פתיחה ונעילה, אלא כוללות גם יכולת תגובה לאירועים חריגים. לכן קיימת חשיבות לשילוב מנגנוני גיבוי, חיישנים לזיהוי מצב הסביבה, תיעוד ניסיונות גישה ושליחת התראות למשתמש. שילוב רכיבים אלו מאפשר לבנות מערכת אמינה יותר, אשר מסוגלת לפעול בתרחישים שונים ולהגיב למצבי כשל או לניסיונות גישה לא מורשים.

בפרויקט זה תוכננה ומומשה מערכת מנעול חכם, אשר שילבה בין חומרה דיגיטלית, תוכנה משובצת, עיבוד תמונה, חיישנים ותקשורת. המערכת נועדה להדגים פתרון בקרת כניסה מתקדם, שבו פעולת הנעילה לא התבססה על מפתח מכני בלבד, אלא על שילוב של זיהוי משתמש, בדיקת תנאי סביבה, בקרה על מצב הדלת וניהול ניסיונות גישה. פירוט מבנה המערכת, רכיבי החומרה, אופן פעולת התוכנה, תהליכי הבדיקה והתוצאות יוצג בסעיפים הבאים.

2. מטרת הפרויקט

מטרת הפרויקט הייתה לתכנן ולממש מערכת מנעול חכם המבוססת על כרטיס הפיתוח PYNQ-Z2 אשר אפשרה בקרת כניסה מאובטחת באמצעות שילוב בין זיהוי פנים, לוח מקשים, חיישנים, בקרת מנוע ושליחת התראות בזמן אמת.

המערכת נועדה לאפשר פתיחה אוטומטית של מנעול לאחר זיהוי משתמש מורשה, תוך שימוש במצלמת USB ובאלגוריתמים לגילוי וזיהוי פנים. בנוסף, נדרש היה לשלב מנגנון גיבוי לפתיחת המנעול באמצעות הקשת סיסמה בלוח מקשים, כך שהמערכת תוכל לפעול גם במקרים שבהם זיהוי הפנים לא בוצע או לא היה זמין.

מטרה נוספת של הפרויקט הייתה לשלב בין תכנון חומרה ב־ FPGA לבין תוכנה משובצת בשפת Python במסגרת זו תוכנן בקר מנוע צעד בשפת VHDL אשר שימש לשליטה במנגנון הפתיחה והנעילה. בנוסף, פותחה תוכנה אשר ניהלה את כלל פעולת המערכת, קראה את מצב החיישנים, הפעילה את רכיבי הפלט, ביצעה עיבוד תמונה, ניהלה ניסיונות גישה ושמרה תיעוד של פעולות המערכת.

כמו כן, נדרש היה לשפר את רמת האבטחה של המערכת באמצעות מנגנון לניהול ניסיונות גישה שגויים. במקרה של הקשת קוד שגוי או כישלונות חוזרים בזיהוי פנים, נספרו ניסיונות הגישה, ובמקרים חריגים נשלחה התראה למשתמש באמצעות Telegram בדרך זו ניתנה למערכת יכולת תגובה לאירועים שאינם חלק מתהליך כניסה תקין.

בנוסף, שולב חיישן דלת מגנטי מסוג MC-38 לצורך זיהוי מצב הדלת לאחר פתיחת המנעול. מטרת שילוב זה הייתה לוודא כי פעולת הנעילה תבוצע רק לאחר שהדלת נסגרה בפועל, ובכך למנוע נעילה בזמן שהדלת עדיין פתוחה.

בסיכומו של דבר, מטרת הפרויקט הייתה להדגים מערכת בקרת כניסה חכמה ומשובצת, המשלבת בין חומרה, תוכנה, עיבוד תמונה, חיישנים ותקשורת, ומספקת פתרון מתקדם יותר ממנגנון נעילה מכני רגיל.

3. דרישות המערכת

במסגרת הפרויקט הוגדרו דרישות מערכת ותנאי תכנון, אשר שימשו בסיס לתכנון, למימוש ולבדיקת מערכת המנעול החכם. הדרישות הוגדרו כך שהמערכת תוכל לפעול כמערכת בקרת כניסה חכמה, המשלבת בין חומרה דיגיטלית, תוכנה משובצת, חיישנים, עיבוד תמונה, בקרה על מנגנון נעילה ושליחת התראות בזמן אמת.

להלן דרישות המערכת:

- ביצוע בקרת כניסה חכמה באמצעות זיהוי פנים, תוך פתיחת המנעול רק עבור משתמש מורשה.
- זיהוי תנועה באזור הדלת באמצעות חיישן PIR והפעלת תהליך הצילום והזיהוי רק בעת זיהוי תנועה.
- ביצוע צילום של אזור הכניסה באמצעות מצלמת USB, ועיבוד התמונה לצורך גילוי וזיהוי פנים.
- מימוש אלגוריתם גילוי פנים וזיהוי פנים באמצעות OpenCV תוך שימוש במודל מאומן לזיהוי המשתמש המורשה.
- מימוש מנגנון פתיחה ונעילה באמצעות מנוע צעד הנשלט על ידי חומרה שתוכננה ב־ FPGA
- שילוב לוח מקשים כדרך גיבוי לפתיחת המנעול באמצעות סיסמה תקינה.
- זיהוי מצב הדלת באמצעות חיישן מגנטי MC-38 וביצוע נעילה רק לאחר זיהוי סגירה פיזית של הדלת.
- זיהוי תנאי תאורה בסביבת הדלת, והפעלת תאורה באמצעות ממסר במקרה של חשכה.
- ניהול ניסיונות גישה שגויים, לרבות ספירת ניסיונות כושלים והפעלת מצב נעילה זמנית לאחר מספר ניסיונות חריגים.
- שליחת התראות בזמן אמת למשתמש באמצעות Telegram כולל שליחת תמונה במקרה של ניסיון גישה חריג.
- תיעוד פעולות המערכת ואירועי הגישה בקובץ לוג לצורך בדיקה, מעקב וניתוח תוצאות.
- הפעלת המערכת כמערכת משובצת עצמאית המבוססת על כרטיס PYNQ-Z2 תוך שילוב בין תוכנה ב־ Python וחומרה ב־ VHDL

4. רקע תיאורטי

מערכת מנעול חכם משלבת מספר תחומים הנדסיים, ובהם מערכות משובצות מחשב, תכנון חומרה דיגיטלית, עיבוד תמונה, חיישנים, בקרת מנועים ותקשורת. שילוב תחומים אלו מאפשר בניית מערכת המסוגלת לקלוט מידע מהסביבה, לעבד אותו, לקבל החלטות בהתאם לתנאים שהוגדרו מראש, ולהפעיל רכיבי חומרה בהתאם לתוצאת ההחלטה.

במערכות משובצות מחשב נעשה שימוש ברכיבי עיבוד ייעודיים לצורך ביצוע משימה מוגדרת. בניגוד למחשב כללי, מערכת משובצת מתוכננת בדרך כלל עבור יישום מסוים, ולכן היא כוללת שילוב בין תוכנה, חומרה ורכיבי קלט ופלט. בפרויקט זה נעשה שימוש בגישה זו לצורך מימוש מערכת בקרת כניסה, שבה נדרשה אינטראקציה בין חיישנים, מצלמה, מנוע, לוח מקשים ומערכת התראות.

אחד המרכיבים המרכזיים במערכת היה כרטיס פיתוח המשלב בין מעבד לבין FPGA המעבד שימש להרצת התוכנה, לניהול הלוגיקה הראשית של המערכת, לעיבוד התמונה ולשליחת התראות. רכיב ה־FPGA שימש למימוש חומרה דיגיטלית ייעודית, ובפרט למימוש בקר מנוע הצעד. חלוקה זו אפשרה להפריד בין פעולות שהתאימו למימוש תוכנתי לבין פעולות שהתאימו למימוש חומרתי.

עיבוד התמונה שימש לזיהוי המשתמש באזור הדלת. בתהליך זה נקלטה תמונה ממצלמת USB, ולאחר מכן בוצעו עליה פעולות גילוי וזיהוי פנים. תחילה נדרש היה לזהות האם קיימים פנים בתמונה, ולאחר מכן נדרש היה לבדוק האם הפנים שזוהו שייכים למשתמש מורשה. לצורך כך נעשה שימוש באלגוריתמים מוכרים בתחום הראייה הממוחשבת, אשר אפשרו לבצע גילוי פנים וזיהוי פנים על בסיס תמונות שנקלטו בזמן פעולת המערכת.

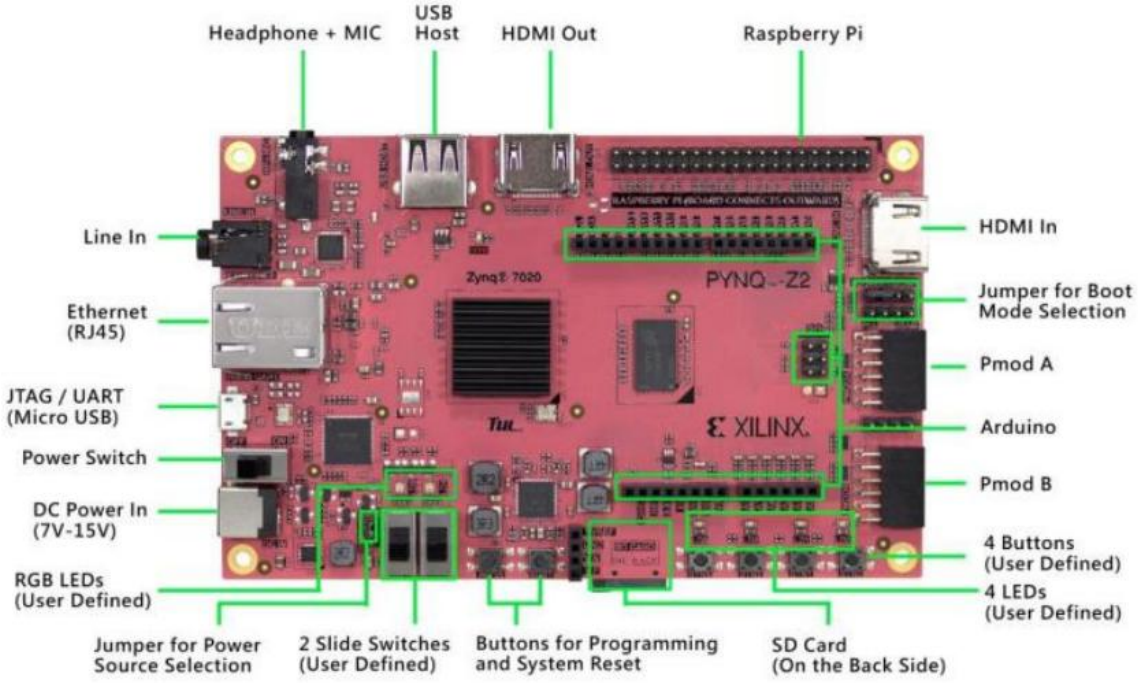
בנוסף לזיהוי פנים, שולבו במערכת חיישנים אשר שימשו לקבלת מידע על סביבת הדלת. חיישן תנועה שימש להפעלת תהליך הצילום רק כאשר זוהתה תנועה באזור הכניסה. חיישן תאורה שימש לזיהוי תנאי חושך ולהפעלת תאורה במקרה הצורך. חיישן דלת מגנטי שימש לזיהוי מצב הדלת לאחר פתיחת המנעול, כך שניתן היה לבצע נעילה רק לאחר זיהוי סגירה פיזית של הדלת.

בקרת המנעול בוצעה באמצעות מנוע צעד. מנוע מסוג זה מאפשר תנועה בזוויות קבועות ומבוקרות, ולכן הוא מתאים למערכות שבהן נדרשת שליטה מדויקת בכיוון ובמשך הסיבוב. בפרויקט זה שימש מנוע הצעד לייצוג פעולת הפתיחה והנעילה, כאשר כיוון סיבוב אחד ייצג פתיחה וכיוון הסיבוב ההפוך ייצג נעילה.

שכבת האבטחה של המערכת התבססה על שילוב בין זיהוי משתמש מורשה, סיסמת גיבוי, ספירת ניסיונות גישה שגויים ושליחת התראות. מנגנון זה נועד לאפשר כניסה תקינה למשתמש מורשה, ובמקביל לזהות מצבים חריגים כגון הקשת סיסמה שגויה או כישלונות חוזרים בזיהוי פנים. במקרה של אירועים חריגים, נשלחה התראה למשתמש ונשמר תיעוד של האירוע.

4.1. כרטיס הפיתוח PYNQ-Z2

כרטיס הפיתוח PYNQ-Z2 הוא כרטיס פיתוח המבוסס על רכיב Zynq-7000 של חברת Xilinx רכיב זה משלב באותו שבב בין מערכת עיבוד מבוססת ARM לבין רכיב FPGA שילוב זה מאפשר לממש מערכת משובצת שבה חלק מהפעולות מבוצעות בתוכנה על ידי המעבד, וחלק מהפעולות מבוצעות בחומרה דיגיטלית ייעודית בתוך ה־ FPGA.



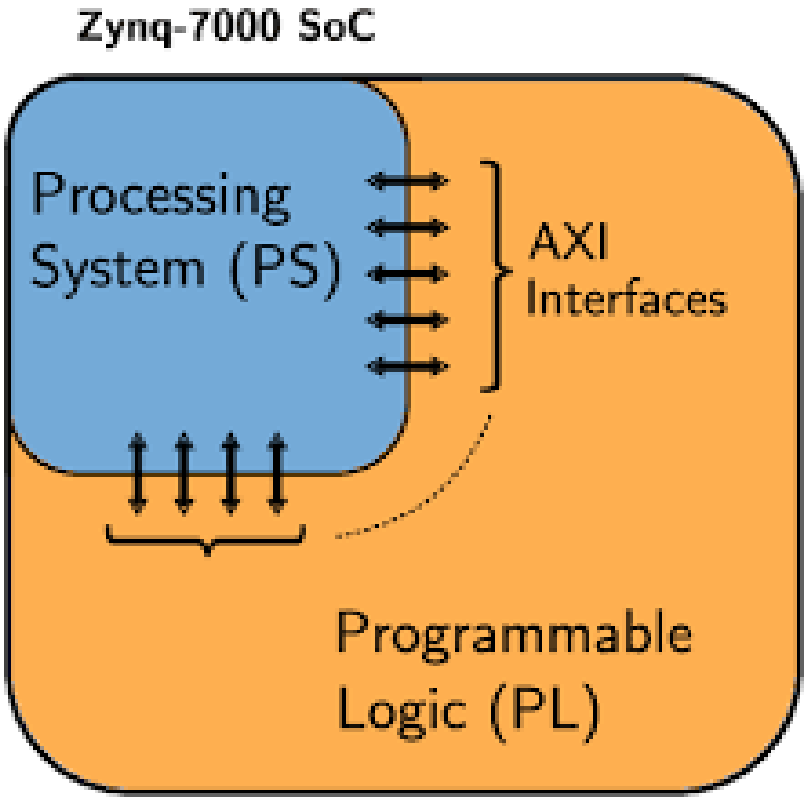
איור 1 פירוט החיבורים על כרטיס פיתוח PYNQ-Z2

הכרטיס מאפשר פיתוח מערכות חומרה ותוכנה על גבי פלטפורמה אחת. מצד אחד, ניתן להריץ עליו קוד תוכנה בשפות עיליות כגון Python ומצד שני ניתן לתכנן בו חומרה דיגיטלית באמצעות שפות תיאור חומרה כגון VHDL שילוב זה מאפשר לממש מערכת שבה פעולות מסוימות מבוצעות בתוכנה, ואילו פעולות אחרות מבוצעות בחומרה ייעודית.

בכרטיס PYNQ-Z2 קיימים ממשקי חיבור שונים, המאפשרים לחבר אליו רכיבי קלט ופלט חיצוניים. בין הרכיבים שניתן לחבר לכרטיס נכללים חיישנים, נורות חיווי, זמזמים, ממסרים, מנועים, מצלמות ורכיבים דיגיטליים נוספים. יכולת זו מאפשרת להשתמש בכרטיס כמרכז בקרה של מערכת משובצת, הקולטת מידע מהסביבה ומפעילה רכיבי חומרה בהתאם להחלטות שמתקבלות בתוכנה או בחומרה.

ייחודו של כרטיס PYNQ-Z2 הוא בכך שהוא תומך בסביבת PYNQ סביבה זו מאפשרת לשלוט בחומרה שתוכננה על גבי ה־ FPGA באמצעות שפת Python ובאמצעות מחברות Jupyter Notebook כלומר, לאחר שתוכננה חומרה מתאימה ונוצרו קובצי החומרה הדרושים, ניתן לטעון אותם אל הכרטיס ולהפעיל את החומרה מתוך קוד Python בצורה נוחה יחסית.

הכרטיס התאים למימוש מערכת המנעול החכם מאחר שהוא איפשר לשלב בין מספר תחומים הנדסיים באותה מערכת. התוכנה שימשה לניהול הלוגיקה הראשית, לעיבוד תמונה, לזיהוי פנים, לקריאת חיישנים ולשליחת התראות. במקביל, החומרה שתוכננה ב־ FPGA שימשה לשליטה במנוע הצעד אשר ייצג את פעולת הפתיחה והנעילה של המנעול. שילוב זה הפך את הכרטיס לפלטפורמה מרכזית ומתאימה עבור מערכת בקרת כניסה חכמה.



איור 2 דיאגרמה של ZYNQ 7000

כרטיס הפיתוח PYNQ-Z2 מבוסס על רכיב Zynq-7000 של חברת Xilinx רכיב זה שייך למשפחת רכיבים המשלבת באותו שבב בין מערכת עיבוד מבוססת מעבד ARM לבין אזור לוגי ניתן לתכנות מסוג FPGA שילוב זה מאפשר לפתח מערכת שבה תוכנה וחומרה פועלות יחד בתוך רכיב אחד.

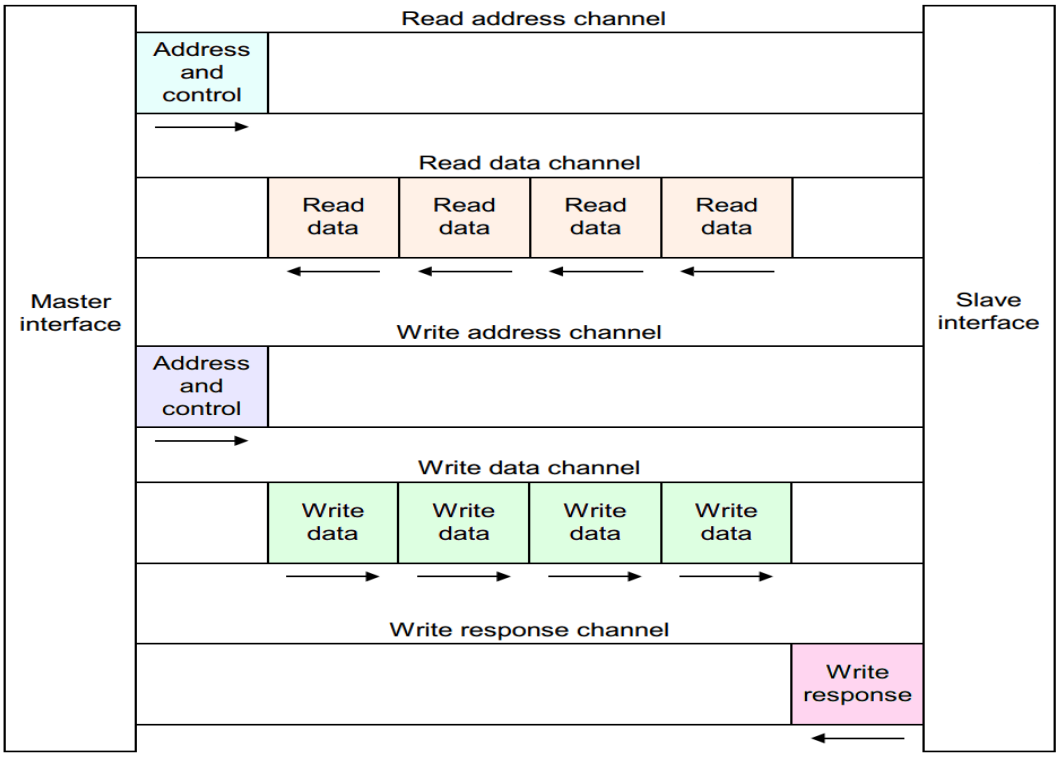
במערכות רגילות המבוססות על מיקרו־בקר או מעבד בלבד, רוב פעולות המערכת מבוצעות באמצעות תוכנה. לעומת זאת, במערכות המבוססות על FPGA ניתן לתכנן חומרה דיגיטלית ייעודית לביצוע פעולות מסוימות. רכיב Zynq-7000 משלב בין שתי הגישות, ולכן ניתן לנצל גם את הגמישות של תוכנה וגם את היכולת לממש חומרה ייעודית בהתאם לצורך.

המבנה הפנימי של רכיב Zynq-7000 מחולק לשני חלקים עיקריים. החלק הראשון הוא Processing System, או בקיצור PS והוא כולל את יחידת העיבוד, הזיכרון וממשקי התקשורת. החלק השני הוא Programmable Logic או בקיצור PL והוא כולל את אזור ה־ FPGA שבו ניתן לממש חומרה דיגיטלית. בין שני החלקים קיימים ממשקי תקשורת פנימיים, המאפשרים העברת מידע ופקודות בין התוכנה לבין החומרה.

יתרון משמעותי של רכיב Zynq-7000 הוא האפשרות לחלק את משימות המערכת בצורה יעילה. פעולות הדורשות גמישות, קבלת החלטות, עבודה עם קבצים, תקשורת או עיבוד ברמת תוכנה יכולות להתבצע ב־ PS לעומת זאת, פעולות הדורשות תזמון מדויק, מימוש לוגיקה דיגיטלית או שליטה ישירה בחומרה יכולות להתבצע ב־ PL

בפרויקט זה נוצל מבנה זה לצורך מימוש מערכת מנעול חכם. חלק ה־ PS שימש להרצת תוכנת הבקרה, לעיבוד תמונה, לזיהוי פנים, לקריאת חיישנים ולשליחת התראות. חלק ה־ PL שימש למימוש בקר מנוע הצעד, אשר תוכנן בשפת VHDL ונועד להפעיל את מנגנון הפתיחה והנעילה של המנעול.

4.1.2. ממשק AXI



איור 3 פרוטוקול AXI

ממשק AXI הוא פרוטוקול תקשורת המשמש להעברת נתונים ופקודות בין רכיבים שונים במערכות מבוססות Xilinx ברכיב Zynq-7000 נעשה שימוש בממשק זה לצורך קישור בין חלק ה־ Processing System לבין חלק ה־ Programmable Logic כלומר, ממשק AXI מאפשר לתוכנה הפועלת על המעבד לתקשר עם החומרה שתוכננה באזור ה־ FPGA

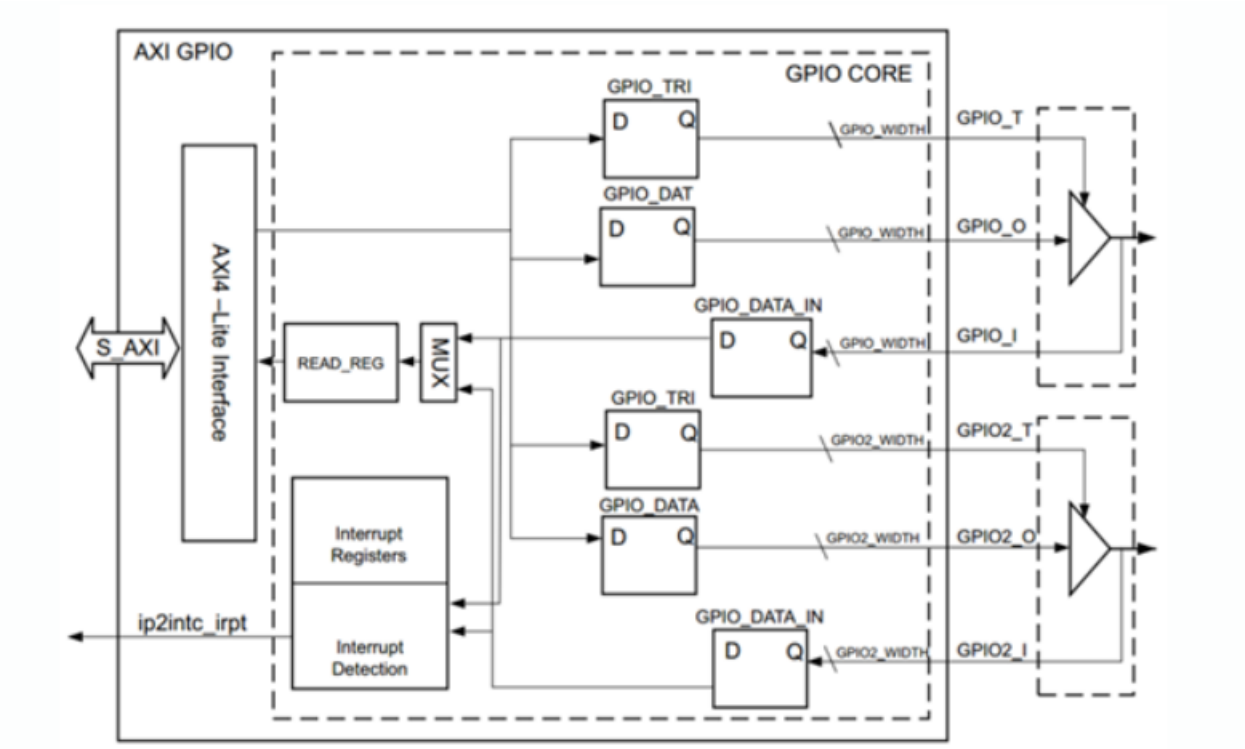
במערכת המבוססת על Zynq ה־ PS וה־ PL אינם פועלים כמערכות נפרדות לחלוטין. על מנת שהמעבד יוכל לשלוט ברכיבי חומרה שתוכננו ב־ FPGA נדרש מנגנון תקשורת פנימי. ממשק AXI מספק מנגנון זה, ובאמצעותו ניתן לשלוח ערכים, לקרוא נתונים, להפעיל רכיבים חומרתיים ולקבל מידע מרכיבי קלט המחוברים למערכת.

עקרון הפעולה של AXI מבוסס על העברת מידע בין יחידת Master לבין יחידת Slave יחידת ה־ Master היא בדרך כלל הרכיב שיוזם את פעולת הקריאה או הכתיבה, ואילו יחידת ה־ Slave היא הרכיב שמגיב לפעולה. לדוגמה, כאשר המעבד נדרש לשלוח פקודת פתיחה לבקר מנוע שתוכנן ב־ FPGAהמעבד משמש כ־ Master ורכיב החומרה שאליו נשלחת הפקודה משמש כ־ Slave

באמצעות ממשק AXI ניתן לבצע פעולות כתיבה וקריאה. בפעולת כתיבה, המעבד שולח ערך מסוים אל רכיב חומרה. ערך זה יכול לשמש להפעלת מנוע, הדלקת נורה, הפעלת זמזם או שינוי מצב של רכיב פלט. בפעולת קריאה, המעבד מקבל ערך מרכיב חומרה, לדוגמה מצב של חיישן תנועה, חיישן אור, לוח מקשים או חיישן מצב דלת.

היתרון המרכזי של ממשק AXI הוא בכך שהוא מאפשר חיבור מסודר ויעיל בין תוכנה לבין חומרה. מצד אחד, התוכנה יכולה להתייחס לרכיבי החומרה כאל כתובות או רגיסטרים שניתן לקרוא מהם ולכתוב אליהם. מצד שני, החומרה שתוכננה ב־ FPGA יכולה לקבל פקודות מהמעבד ולספק לו נתונים בהתאם למצב המערכת.

בפרויקט זה שימש ממשק AXI כבסיס לתקשורת בין תוכנת הבקרה לבין רכיבי החומרה. באמצעות ממשק זה נשלחו פקודות להפעלת מנוע הצעד, ונקראו או נכתבו ערכים הקשורים לחיישנים ולרכיבי הפלט. כך התאפשרה שליטה מלאה של התוכנה במערכת החומרתית, תוך שמירה על הפרדה ברורה בין לוגיקת הבקרה לבין המימוש החומרתי.



איור 4 AXI GPIO

AXI GPIO הוא רכיב IP המשמש לחיבור בין ממשק AXI לבין אותות קלט ופלט דיגיטליים. רכיב זה מאפשר למעבד הנמצא בצד ה־ PS לקרוא אותות דיגיטליים המגיעים מה־ PL או לכתוב ערכים דיגיטליים אל יציאות הנמצאות בצד ה־PL בדרך זו ניתן לחבר בין תוכנה הרצה על המעבד לבין רכיבי חומרה חיצוניים המחוברים לכרטיס.

רכיב GPIO או General Purpose Input Output משמש כקלט ופלט כללי. כאשר אות מוגדר כקלט, ניתן לקרוא את ערכו מתוך התוכנה ולקבל מידע על מצב רכיב חיצוני, כגון חיישן. כאשר אות מוגדר כפלט, ניתן לכתוב אליו ערך מתוך התוכנה ובכך להפעיל רכיב חיצוני, כגון נורת חיווי, זמזם, ממסר או פקודת שליטה לבקר מנוע.

במערכת המבוססת על Zynq רכיב AXI GPIO מחובר מצד אחד לממשק AXI ומצד שני לאותות דיגיטליים ב־ PL המעבד מתקשר עם רכיב ה־ AXI GPIO דרך כתובות פנימיות, והתוכנה יכולה לקרוא או לכתוב ערכים אל אותן כתובות. כך מתקבלת שליטה פשוטה יחסית ברכיבי חומרה מתוך קוד Python

בפרויקט זה נעשה שימוש ב־ AXI GPIO לצורך חיבור בין תוכנת הבקרה לבין רכיבי הקלט והפלט של מערכת המנעול החכם. בצד הקלט נקראו אותות מחיישן התנועה, מחיישן האור ומחיישן מצב הדלת. בצד הפלט הופעלו נורות חיווי, זמזם, ממסר ופקודות שליטה עבור מנגנון הנעילה.

בנוסף, רכיב AXI GPIO שימש להעברת פקודות מהתוכנה אל בקר מנוע הצעד שתוכנן ב־ VHDL כאשר התקבלה החלטה לפתוח או לנעול את המנעול, נכתב ערך מתאים אל יציאת ה־ GPIO והערך שימש להפעלת הבקר החומרתי בכיוון המתאים. בכך התאפשרה שליטה תוכנתית במנוע, תוך שימוש בבקר חומרתי שתוכנן באזור ה־ PL

יתרונו של AXI GPIO הוא בפשטותו ובגמישותו. הוא מאפשר לחבר רכיבי קלט ופלט למערכת ללא צורך במימוש פרוטוקול מורכב, ולכן מתאים במיוחד למערכות שבהן נדרשת קריאת חיישנים והפעלת רכיבים דיגיטליים פשוטים. בפרויקט זה, רכיב זה שימש כאחד הממשקים המרכזיים בין התוכנה לבין החומרה.

4.1.4. שימוש ב Overlay בסביבת PYNQ

בסביבת PYNQ נעשה שימוש במושג Overlay לצורך טעינת חומרה שתוכננה מראש אל אזור ה־ FPGA שבכרטיס. ה־ Overlay מייצג את תצורת החומרה שתוכננה בכלי הפיתוח Vivado והוא מאפשר לתוכנה לגשת אל רכיבי החומרה ולשלוט בהם מתוך Python

כאשר תוכננה מערכת חומרה ב־ Vivado נוצר קובץ bit אשר הכיל את תיאור החומרה שיועד להיטען אל ה־ PL קובץ זה הגדיר את מבנה המעגלים הדיגיטליים, את רכיבי ה־ IP את החיבורים הפנימיים ואת האותות שנדרשו להפעלת המערכת. בנוסף לקובץ זה נוצר קובץ hwh אשר הכיל מידע על מבנה החומרה, שמות הרכיבים, כתובות פנימיות וממשקי הגישה אל רכיבי ה־ IP

לאחר העברת קובצי ה־ bit וה־ hwh אל כרטיס PYNQ-Z2 ניתן היה לטעון את ה־ Overlay מתוך סביבת Python טעינה זו גרמה להגדרת אזור ה־ FPGA בהתאם לחומרה שתוכננה, ולאחר מכן התאפשרה גישה לרכיבי החומרה מתוך התוכנה. בדרך זו ניתן היה לשלוט ברכיבי AXI GPIO לקרוא כניסות, לכתוב יציאות ולהפעיל את הבקר שתוכנן ב־ VHDL

בפרויקט זה שימש ה־ Overlay כגשר בין תכנון החומרה לבין תוכנת הבקרה. לאחר טעינתו, התוכנה יכלה לגשת לרכיבי ה־ GPIO שהוגדרו בתכנון, לקרוא את מצב החיישנים, להפעיל את רכיבי החיווי, לשלוט בממסר ולשלוח פקודות לבקר מנוע הצעד. ללא טעינת ה־ Overlay החומרה שתוכננה ב־ Vivado לא הייתה זמינה לשליטה מתוך Python

היתרון המרכזי של שימוש ב־ Overlay הוא האפשרות להפריד בין שלב תכנון החומרה לבין שלב הפעלת המערכת בתוכנה. תחילה תוכננה החומרה בכלי Vivado ולאחר מכן נטענה אל הכרטיס והופעלה דרך סביבת PYNQ כך התאפשר שילוב בין חומרה ייעודית שתוכננה ב־ VHDL לבין תוכנה גמישה שנכתבה בשפת Python

4.1.5. סביבת PYNQ ו־ Jupyter Notebook

סביבת PYNQ היא סביבת פיתוח המיועדת להקל על העבודה עם כרטיסים המבוססים על רכיבי Zynq מטרת הסביבה היא לאפשר שליטה בחומרה שתוכננה ב־ FPGA באמצעות שפת Python ללא צורך בכתיבת דרייברים מורכבים או עבודה ישירה מול שכבות נמוכות של מערכת ההפעלה. בדרך זו ניתן לחבר בין תכנון חומרה לבין פיתוח תוכנה בצורה נוחה ומהירה יחסית.

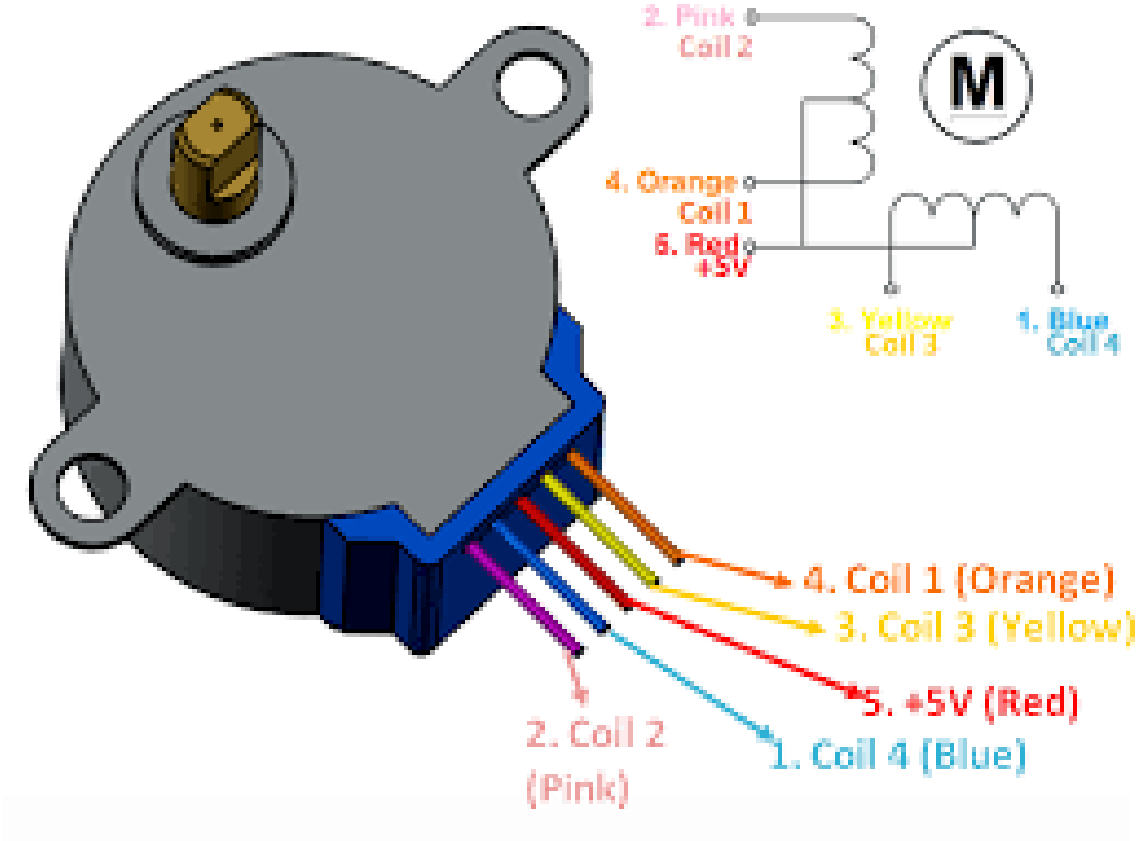
בכרטיס PYNQ-Z2 הופעלה מערכת הפעלה מבוססת Linux אשר אפשרה להריץ קוד Python להשתמש בספריות תוכנה שונות, לנהל קבצים ולעבוד עם התקנים חיצוניים. מעל מערכת ההפעלה פעלה סביבת PYNQ אשר סיפקה ספריות ייעודיות לגישה אל רכיבי החומרה שתוכננו ב־ FPGA באמצעות ספריות אלו ניתן היה לטעון Overlay לגשת לרכיבי AXI GPIO לקרוא כניסות ולכתוב יציאות מתוך קוד Python

Jupyter Notebook שימש כסביבת העבודה המרכזית להפעלת הקוד ולבדיקת המערכת. סביבת עבודה זו אפשרה לכתוב קוד בתאים, להריץ כל חלק בנפרד, לצפות בתוצאות באופן מיידי ולבצע בדיקות הדרגתיות של רכיבי המערכת. יכולת זו הייתה חשובה במיוחד במהלך פיתוח מערכת משובצת, שבה נדרש היה לבדוק בנפרד חיישנים, מנוע, מצלמה, לוח מקשים ותקשורת חיצונית.

באמצעות Jupyter Notebook ניתן היה לבצע תהליך פיתוח הדרגתי. תחילה נבדקה טעינת ה־ Overlay לאחר מכן נבדקה הגישה ל־ GPIO ובהמשך נבדקו רכיבי החומרה והחיישנים. לאחר שכל תת־מערכת נבדקה בנפרד, בוצע שילוב הדרגתי של כלל הרכיבים במערכת אחת. גישה זו אפשרה לאתר תקלות בצורה מסודרת ולוודא את תקינות כל חלק לפני שילובו במערכת הסופית.

4.2. מנוע צעד 28BYJ-48

מנוע צעד הוא מנוע חשמלי המאפשר תנועה זוויתית מדורגת ומבוקרת. בניגוד למנוע DC רגיל, אשר מסתובב באופן רציף כאשר מסופק לו מתח, מנוע צעד נע בצעדים קבועים. כל צעד מייצג שינוי זוויתי קטן ומוגדר במיקום ציר המנוע. תכונה זו מאפשרת שליטה מדויקת יחסית בכיוון הסיבוב, במספר הצעדים ובמיקום הסופי של הציר.



איור 5 במנוע צעד מסוג 28BYJ-48

במערכות בקרה נעשה שימוש במנועי צעד כאשר נדרשת תנועה מדויקת ללא צורך הכרחי בחיישן משוב. מנועים מסוג זה נפוצים במערכות כגון מדפסות, סורקים, מנגנוני פתיחה וסגירה, מערכות מיקום, רובוטיקה ומערכות אוטומציה. השליטה במנוע מתבצעת באמצעות הפעלה מסודרת של סלילי המנוע לפי רצף מוגדר.

בפרויקט זה נעשה שימוש במנוע צעד מסוג 28BYJ-48 מנוע זה הוא מנוע צעד קטן ונפוץ, הפועל בדרך כלל במתח של 5 V ומתאים למערכות לימודיות ולמערכות בקרה קטנות. המנוע כולל מספר סלילים פנימיים, והפעלתם ברצף מתאים גורמת לסיבוב הדרגתי של ציר המנוע.

עקרון הפעולה של המנוע מבוסס על יצירת שדה מגנטי משתנה בסלילי המנוע. כאשר זרם חשמלי מוזרם דרך סליל מסוים, נוצר שדה מגנטי אשר מושך את הרוטור לכיוון מתאים. כאשר הסלילים מופעלים בזה אחר זה בסדר קבוע, הרוטור מתקדם בצעדים, ונוצר סיבוב. שינוי סדר הפעלת הסלילים משנה את כיוון הסיבוב של המנוע.

במערכת המנעול החכם שימש מנוע הצעד לייצוג פעולת הפתיחה והנעילה של המנעול. סיבוב בכיוון אחד ייצג פתיחה של המנעול, וסיבוב בכיוון ההפוך ייצג נעילה. השליטה בכיוון הסיבוב ובמשך הפעולה בוצעה באמצעות בקר מנוע שתוכנן ב־VHDL אשר הפיק את רצף האותות המתאים להפעלת המנוע.

לצורך הפעלת מנוע הצעד נעשה שימוש ברצף לוגי מוגדר עבור ארבע כניסות הדרייבר IN1–IN4. בכל צעד הופעלה פאזה אחת בלבד, וסדר הפעלת הפאזות קבע את כיוון הסיבוב של המנוע. כאשר הרצף בוצע בסדר אחד, התקבל סיבוב עם כיוון השעון. כאשר אותו רצף בוצע בסדר הפוך, התקבל סיבוב נגד כיוון השעון. בטבלאות הבאות מוצגים שני רצפי ההפעלה ששימשו במערכת: רצף עבור פתיחת המנעול ורצף עבור נעילתו.

טבלה 5 רצף לוגי להפעל מנוע צעד עם כיוון השעון

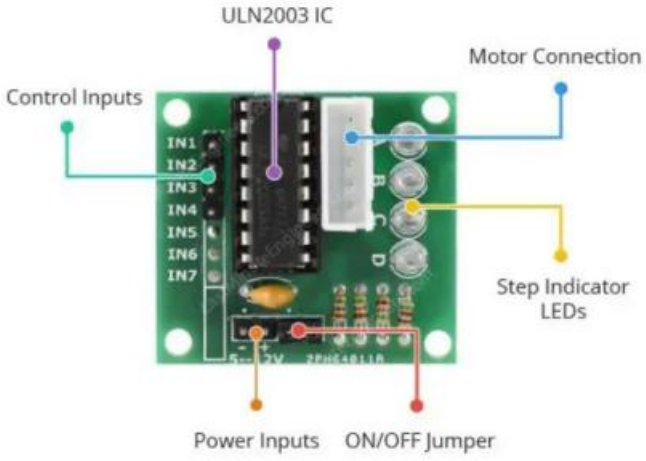
מספר צעד	IN1	IN2	IN3	IN4	מוצא לוגי
1	1	0	0	0	1000
2	0	1	0	0	0100
3	0	0	1	0	0010
4	0	0	0	1	0001

טבלה 1 רצף לוגי להפעל מנוע צעד נגד כיוון השעון

מספר צעד	IN1	IN2	IN3	IN4	מוצא לוגי
1	0	0	0	1	0001
2	0	0	1	0	0010
3	0	1	0	0	0100
4	1	0	0	0	1000

4.2.1. דרייבר ULN2003

מנוע הצעד 28BYJ-48 דרש שימוש בדרייבר מתאים, מאחר שלא ניתן היה להפעיל את סלילי המנוע ישירות מיציאות הכרטיס. לכן שולב במערכת דרייבר ULN2003 אשר שימש כמתווך בין אותות הבקרה לבין זרם ההפעלה הדרוש למנוע.



איור 6 דרייבר ULN2003

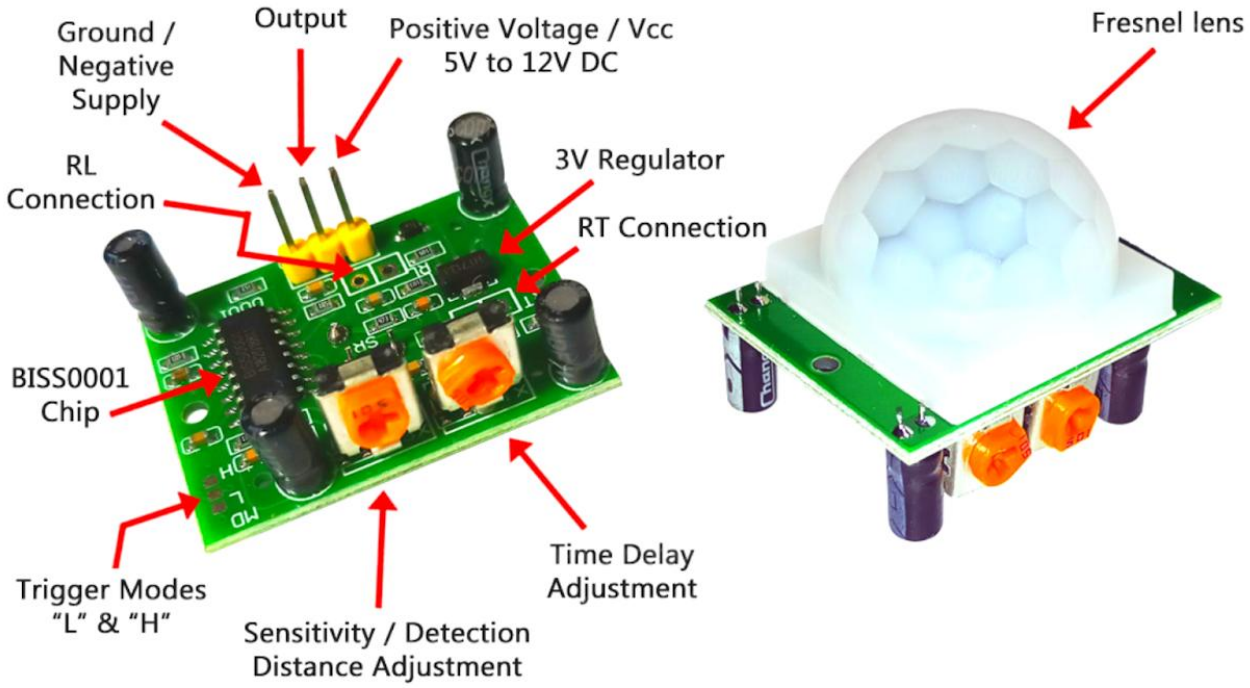
דרייבר ULN2003 הוא רכיב המשמש להפעלת עומסים הדורשים זרם גבוה יותר מזה שניתן לספק ישירות מיציאות בקרה דיגיטליות. רכיב זה נפוץ במיוחד בהפעלה של מנועי צעד קטנים, ממסרים, נורות ועומסים אינדוקטיביים נוספים.

במערכות דיגיטליות, יציאות בקרה של כרטיסי פיתוח או רכיבי FPGA מסוגלות לספק אות לוגי, אך בדרך כלל אינן מיועדות להזרים את הזרם הדרוש להפעלת סלילי מנוע. לכן נדרש רכיב תיווך בין אותות הבקרה לבין העומס. תפקידו של ה־ ULN2003 הוא לקבל אותות לוגיים בכניסותיו, ולהפעיל בהתאם יציאות המסוגלות להעביר זרם גבוה יותר אל העומס.

רכיב ULN2003 מבוסס על מערך טרנזיסטורי Darlington מבנה זה מאפשר הגברת זרם, כך שאות בקרה קטן יחסית יכול לשלוט בזרם גדול יותר. בנוסף, הרכיב כולל דיודות הגנה פנימיות, אשר מיועדות להגן על המעגל מפני מתחי השראה הנוצרים בעת הפעלה וכיבוי של עומסים אינדוקטיביים, כגון סלילי מנוע.

4.3. חיישן תנועה PIR

חיישן PIR הוא חיישן תנועה המבוסס על זיהוי שינויי קרינה בתחום האינפרא־אדום. ראשי התיבות PIR מציינים Passive Infrared Sensor כלומר חיישן אינפרא־אדום פסיבי. החיישן נקרא פסיבי משום שהוא אינו משדר קרינה אל הסביבה, אלא קולט שינויים בקרינה האינפרא־אדומה הנפלטת מגופים הנמצאים באזור הגילוי.



איור 7 חיישן תנועה PIR

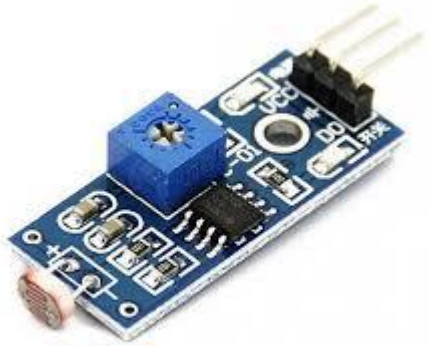
כל גוף בעל טמפרטורה מעל האפס המוחלט פולט קרינה תרמית בתחום האינפרא־אדום. גוף האדם פולט קרינה בתחום זה, ולכן ניתן לזהות תנועה של אדם כאשר קיימת תזוזה יחסית בין הגוף לבין אזור הקליטה של החיישן. החיישן אינו מזהה את האדם עצמו, אלא מזהה שינוי ברמת הקרינה הנקלטת באזור מסוים.

חיישן PIR כולל רכיב רגיש לקרינה אינפרא־אדומה, ובדרך כלל משולבת בו עדשת Fresnel תפקידה של העדשה הוא לרכז את הקרינה המגיעה מאזורים שונים במרחב אל הרכיב הרגיש. כאשר אדם נע בתוך אזור הגילוי, משתנה פיזור הקרינה הנקלטת על ידי החיישן, ובעקבות זאת מתקבל שינוי במוצא החיישן.

מוצא החיישן הוא בדרך כלל אות דיגיטלי. כאשר לא מזוהה תנועה, המוצא נמצא במצב לוגי נמוך. כאשר מזוהה תנועה, המוצא עובר למצב לוגי גבוה למשך זמן מסוים. זמן זה יכול להיות מושפע מהגדרות החיישן, כגון רגישות וזמן השהיה.

4.4. חיישן אור דיגיטלי LM393

חיישן האור הדיגיטלי LM393 הוא רכיב המשמש לזיהוי עוצמת תאורה בסביבה. החיישן מבוסס על רכיב רגיש לאור, כגון LDR ועל משווה מתח מסוג LM393 שילוב זה מאפשר להמיר שינוי בעוצמת האור לאות דיגיטלי, אשר ניתן לקריאה על ידי מערכת בקרה.

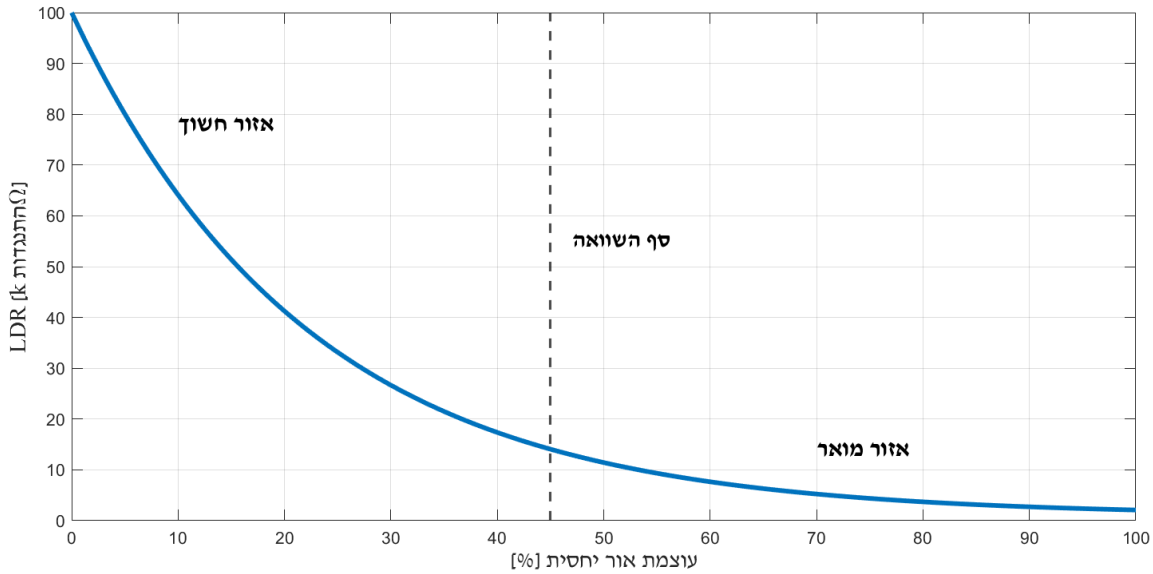


איור 8 חיישן אור דיגיטלי LM393

רכיב LDR או Light Dependent Resistor הוא נגד שהתנגדותו משתנה בהתאם לעוצמת האור הפוגע בו. כאשר עוצמת האור גבוהה, התנגדות ה־ LDR קטנה. כאשר עוצמת האור נמוכה, התנגדות ה־ LDR גדלה. שינוי זה בהתנגדות יוצר שינוי במתח הנמדד במעגל, וניתן להשתמש בו לצורך קביעת מצב התאורה בסביבה.

ה־ LM393 משמש כמשווה מתח. תפקידו להשוות בין מתח המתקבל ממעגל חיישן האור לבין מתח ייחוס שנקבע מראש. כאשר המתח הנמדד עובר את סף הייחוס, מוצא החיישן משתנה בין מצב לוגי נמוך לבין מצב לוגי גבוה. באופן זה מתקבל אות דיגיטלי פשוט, אשר מייצג האם סביבת החיישן מוארת או חשוכה.

בחיישנים מסוג זה קיימת בדרך כלל אפשרות לכיוון רגישות באמצעות פוטנציומטר. כיוון זה מאפשר לקבוע את נקודת הסף שבה החיישן יחליף את מצב המוצא. כלומר, ניתן להגדיר באיזו רמת תאורה החיישן יזהה את הסביבה כחשוכה או כמוארת.



איור 9 גרף התנגדות בהתאם לעוצמת האור

4.5. חיישן דלת מגנטי MC-38

חיישן MC-38 הוא חיישן מגנטי המשמש לזיהוי מצב פתיחה או סגירה של דלת, חלון או מכסה. החיישן מורכב משני חלקים עיקריים: חלק אחד כולל מפסק מגנטי מסוג Reed Switch והחלק השני כולל מגנט קבוע. כאשר שני החלקים קרובים זה לזה, מתקבל מצב חשמלי אחד, וכאשר הם מתרחקים זה מזה, מתקבל מצב חשמלי אחר.



איור 10 חיישן דלת מגנטי MC-38

עקרון הפעולה של החיישן מבוסס על השפעת שדה מגנטי על מגעי ה־ Reed Switch. כאשר המגנט נמצא בקרבת המפסק, השדה המגנטי גורם למגעים הפנימיים לשנות את מצבם. כאשר הדלת נפתחת והמגנט מתרחק מהמפסק, השדה המגנטי נחלש, והמגעים חוזרים למצבם המקורי. שינוי זה מאפשר למערכת לזהות האם הדלת פתוחה או סגורה.

חיישן MC-38 מתאים במיוחד למערכות אבטחה ובקרת כניסה, מכיוון שהוא פשוט להפעלה, אמין, ואינו דורש מנגנון מכני מורכב. החיישן מספק אינדיקציה דיגיטלית למצב הדלת, ולכן ניתן לחברו למערכת בקרה ולקרוא את מצבו כאות לוגי.

במערכת המנעול החכם שימש חיישן MC-38 לזיהוי מצב הדלת לאחר פתיחת המנעול. לאחר שהמנעול נפתח, בוצעה בדיקה של מצב החיישן כדי לוודא האם הדלת נסגרה בחזרה. רק לאחר שזוהתה סגירה פיזית של הדלת, נשלחה פקודת נעילה למנוע הצעד.

שילוב חיישן זה שיפר את אמינות פעולת המערכת, מאחר שהוא מנע מצב שבו המנעול היה ננעל כאשר הדלת עדיין הייתה פתוחה. בדרך זו פעולת הנעילה בוצעה בהתאם למצב הפיזי של הדלת, ולא רק לפי השהיית זמן קבועה.

במערכת זו נעשה שימוש בחיבור שבו מצב הדלת תורגם לאות לוגי שנקרא על ידי כרטיס הפיתוח. כאשר הדלת הייתה סגורה התקבל מצב לוגי אחד, וכאשר הדלת הייתה פתוחה התקבל מצב לוגי אחר. מידע זה שימש את תוכנת הבקרה לצורך קבלת החלטה האם להפעיל את מנגנון הנעילה.

4.6. מצלמת USB מסוג Logitech Brio 300

מצלמת USB היא רכיב קלט חזותי המשמש לצילום תמונות או וידאו והעברתם למערכת עיבוד. המצלמה מתחברת למחשב או לכרטיס פיתוח באמצעות ממשק USB ומאפשרת קבלת תמונה דיגיטלית שניתן לעבד באמצעות תוכנה.



איור 11 מצלמת Logitech Brio 300 USB

בפרויקט זה נעשה שימוש במצלמת USB מסוג Logitech Brio 300 מצלמה זו שימשה לצילום אזור הכניסה לאחר זיהוי תנועה על ידי חיישן ה- PIR לאחר שהתקבלה אינדיקציה לתנועה באזור הדלת, הופעלה המצלמה וצולמה תמונה של האדם שנמצא מול המערכת. תמונה זו שימשה כקלט עבור תהליך גילוי הפנים ותהליך זיהוי הפנים.

במערכות ראייה ממוחשבת, מצלמה משמשת כשלב הקלט הראשוני של המערכת. התמונה הנקלטת מהמצלמה מיוצגת כמטריצה של פיקסלים, כאשר כל פיקסל מכיל מידע על עוצמת האור או על רכיבי הצבע בתמונה. לאחר קליטת התמונה ניתן לבצע עליה פעולות עיבוד שונות, כגון המרה לגווני אפור, סינון, גילוי תבניות, זיהוי אובייקטים וזיהוי פנים.

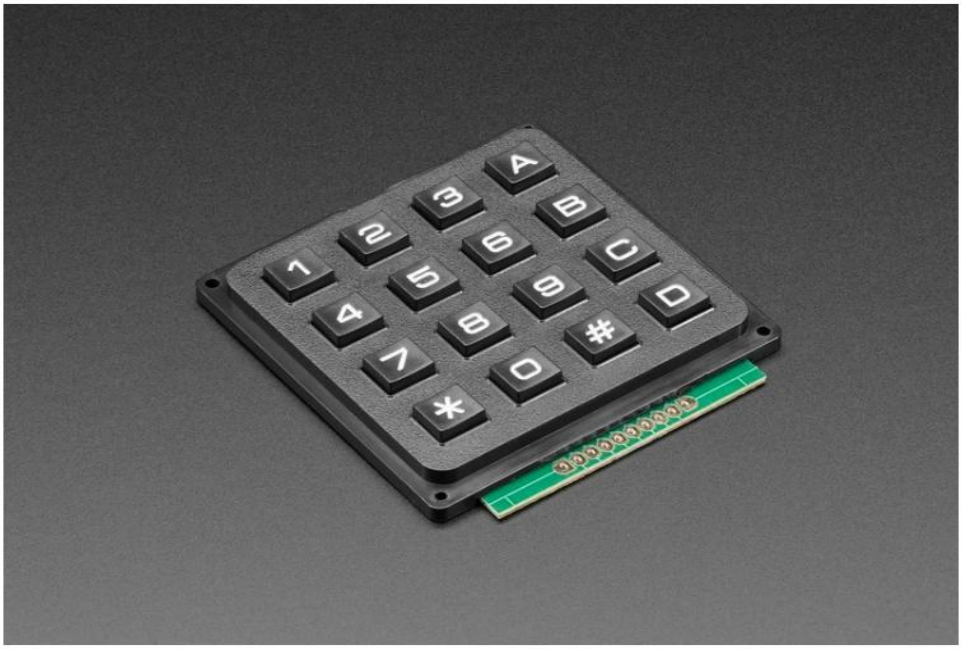
השימוש במצלמת Logitech Brio 300 התאים לפרויקט מאחר שהיא סיפקה דרך פשוטה ונוחה לקבלת תמונה דיגיטלית בתוך סביבת Python באמצעות ספריית OpenCV ניתן היה לפתוח את המצלמה, לקלוט תמונה, לשמור אותה כקובץ ולהעביר אותה להמשך עיבוד. בדרך זו שולבה המצלמה כחלק מתהליך בקרת הכניסה של מערכת המנעול החכם.

איכות התמונה שהתקבלה מהמצלמה השפיעה על יכולת גילוי וזיהוי הפנים. לכן, לפני צילום התמונה נבדקו תנאי התאורה באזור הדלת.

4.7. לוח מקשים 4X4

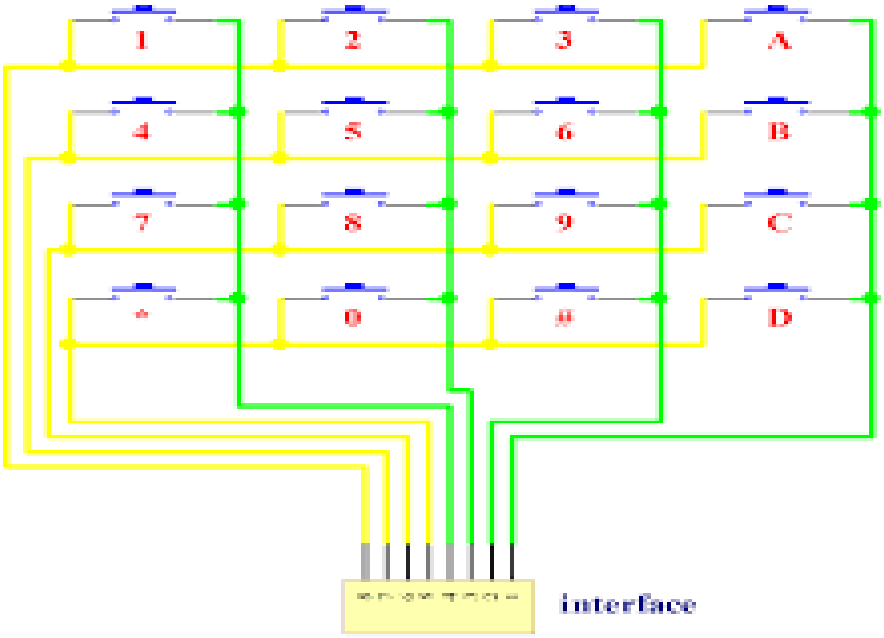
לוח מקשים מטריציוני 4x4 הוא רכיב קלט דיגיטלי המאפשר זיהוי לחיצות על מקשים באמצעות מבנה של שורות ועמודות. בלוח מקשים מסוג זה קיימים 16 מקשים, המסודרים בארבע שורות ובארבע עמודות. כל לחיצה על מקש יוצרת חיבור חשמלי בין שורה מסוימת לבין עמודה מסוימת, ובאמצעות זיהוי השורה והעמודה ניתן לקבוע איזה מקש נלחץ.

השימוש במבנה מטריציוני מאפשר לחסוך במספר חיבורי הקלט והפלט הדרושים למערכת. במקום לחבר כל מקש בנפרד, נעשה שימוש בשמונה קווים בלבד: ארבעה קווי שורות וארבעה קווי עמודות. בדרך זו ניתן לזהות 16 מקשים באמצעות מספר קטן יחסית של חיבורים.



איור 12 לוח מקשים 4 על 4

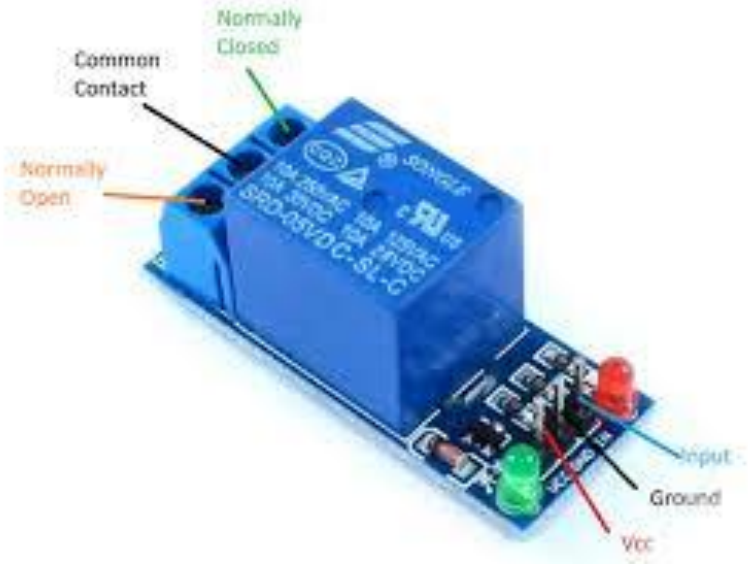
עקרון הפעולה של לוח מקשים מטריציוני מבוסס על סריקה מחזורית של השורות או העמודות. במהלך הסריקה מופעל בכל פעם קו אחד, ובמקביל נקרא מצב הקווים בכיוון השני. כאשר מזוהה שינוי במצב הקלט, ניתן לקבוע איזו שורה ואיזו עמודה חוברו זו לזו, ועל פי צירוף זה נקבע המקש שנלחץ.



איור 13 דיאגרמה של לוח מקשים 4 על 4

4.8. ממסר להפעלת תאורה

ממסר הוא רכיב מיתוג חשמלי המאפשר לשלוט בעומס חשמלי באמצעות אות בקרה. במערכות משובצות מחשב נעשה שימוש בממסר כאשר נדרש להפעיל רכיב חיצוני שאינו מתאים להפעלה ישירה מתוך יציאת בקרה דיגיטלית. יציאות הבקרה של מיקרו־בקרים ומחשבים משובצים מסוגלות בדרך כלל לספק זרם ומתח נמוכים בלבד, ולכן אינן יכולות להפעיל ישירות התקנים הצורכים הספק גבוה יותר. הממסר משמש כמתווך בין מערכת הבקרה לבין ההתקן החיצוני, ומאפשר שליטה בטוחה ויעילה בעומסים שונים .



איור 14 Single Channel RELAY Module

הממסר מאפשר הפרדה בין מעגל הבקרה לבין המעגל המופעל, ובכך מספק שכבת הגנה נוספת לרכיבי המערכת. הפרדה זו חשובה במיוחד כאשר המעגל הנשלט פועל במתחים או בזרמים גבוהים יותר מאלו שבהם פועלת מערכת הבקרה. בזכות תכונה זו ניתן להגן על המיקרו־בקר, על המחשב המשובץ ועל רכיבים אלקטרוניים רגישים מפני נזקים שעלולים להיגרם כתוצאה מעומסי יתר, קצרים חשמליים או הפרעות במעגל החיצוני.

עקרון הפעולה של ממסר מבוסס על מיתוג חשמלי. כאשר מתקבל אות בקרה מתאים, משתנה מצב המגעים הפנימיים של הממסר, וכך נסגר או נפתח מעגל חשמלי חיצוני. בממסרים אלקטרומכניים, הנפוצים במערכות רבות, אות הבקרה מפעיל סליל אלקטרומגנטי היוצר שדה מגנטי. שדה זה מושך זרוע מכנית פנימית וגורם לשינוי מצב המגעים החשמליים. כאשר אות הבקרה מפסיק, המגעים חוזרים למצבם המקורי באמצעות קפיץ פנימי. תהליך זה מאפשר שליטה אמינה ופשוטה במעגלים חשמליים שונים.

לממסר קיימים בדרך כלל שלושה חיבורים עיקריים בצד המיתוג: מגע משותף COM מגע פתוח בדרך כלל – NO Normally Open ומגע סגור בדרך כלל NC – Normally Closed כאשר הממסר אינו מופעל, המגע המשותף מחובר למגע הסגור בדרך כלל. בעת הפעלת הממסר משתנה החיבור והמגע המשותף מתחבר למגע הפתוח בדרך כלל. מבנה זה מאפשר גמישות רבה בתכנון מערכות בקרה ובהתאמת אופן הפעולה לצורכי המערכת.

בפרויקט זה נעשה שימוש בממסר לצורך הפעלת תאורה חיצונית בהתאם לתנאי הסביבה. כאשר חיישן האור מזהה רמת תאורה נמוכה, מערכת הבקרה מפעילה את הממסר, והממסר מחבר את מקור המתח אל יחידת התאורה. כתוצאה מכך נדלקת התאורה ומספקת תנאי הארה מתאימים לצילום. כאשר רמת התאורה בסביבה מספקת, הממסר נשאר כבוי והתאורה אינה מופעלת, ובכך נחסכת צריכת אנרגיה מיותרת.

השימוש בממסר בפרויקט מעניק מספר יתרונות משמעותיים. ראשית, הוא מאפשר להפעיל התקן חיצוני בעל צריכת זרם גבוהה יותר מזו שמסוגלת לספק יציאת הבקרה. שנית, הוא מספק הפרדה בין מערכת הבקרה לבין מערכת התאורה, דבר

המשפר את אמינות המערכת ומקטין את הסיכון לפגיעה ברכיבים האלקטרוניים. בנוסף, הממסר מאפשר הרחבה עתידית של המערכת וחיבור התקנים נוספים ללא צורך בשינויים מהותיים במעגל הבקרה.

4.9. זמזם

זמזם Buzzer הוא רכיב פלט קולי המשמש להפקת צליל לצורך חיווי, התראה או משוב למשתמש. במערכות משובצות מחשב נעשה שימוש בזמזם כאשר נדרש לספק אינדיקציה מיידית על פעולה שבוצעה, על שינוי מצב במערכת או על אירוע חריג.

קיימים שני סוגים עיקריים של זמזמים: זמזם אקטיבי וזמזם פסיבי. זמזם אקטיבי כולל מעגל פנימי ליצירת הצליל, ולכן ניתן להפעילו באמצעות מתן מתח מתאים בלבד. לעומת זאת, זמזם פסיבי דורש אות בתדר משתנה לצורך יצירת הצליל. במערכות פשוטות, זמזם אקטיבי נחשב נוח יותר להפעלה, מכיוון שניתן לשלוט בו באמצעות אות דיגיטלי רגיל.

עקרון הפעולה של הזמזם מבוסס על המרת אות חשמלי לצליל. כאשר מתקבל אות הפעלה מתאים, הרכיב מייצר תנודות מכניות בתדר הנשמע לאוזן האדם, וכתוצאה מכך מופק צליל. הצליל יכול לשמש לחיווי קצר, להתראה ממושכת או לסימון מצב פעולה מסוים במערכת.



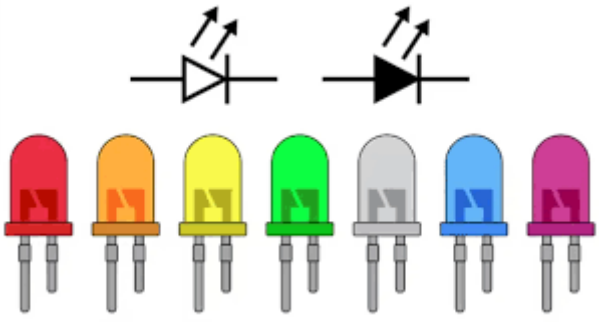
איור 15 זמזם

4.10. נורות חיווי LED

נורת LED היא רכיב אלקטרוני הפולט אור כאשר זורם דרכו זרם בכיוון ההולכה. ראשי התיבות LED מציינים Light Emitting Diode כלומר דיודה פולטת אור. בדומה לדיודה רגילה, רכיב זה מאפשר זרימת זרם בעיקר בכיוון אחד, אך בשונה מדיודה רגילה, בזמן ההולכה נפלט אור נראה.

נורות LED נפוצות מאוד במערכות אלקטרוניות ומשובצות מחשב, מכיוון שהן מאפשרות חיווי פשוט, מהיר וברור של מצבי מערכת. באמצעות נורת LED ניתן להציג למשתמש מידע בסיסי, כגון האם המערכת פועלת, האם התקבל קלט, האם התרחשה שגיאה, או האם רכיב מסוים נמצא במצב פעיל.

בדרך כלל, נורת LED מחוברת בטור עם נגד הגבלת זרם. תפקיד הנגד הוא למנוע זרם גבוה מדי דרך הנורה, אשר עלול לגרום לפגיעה ברכיב. כאשר מופעל מתח מתאים על הנורה בכיוון ההולכה, זורם זרם דרך ה־LED, והנורה נדלקת. כאשר לא מסופק מתח מתאים, או כאשר היציאה נמצאת במצב לוגי נמוך, הנורה כבויה.



איור 16 נורות LED

4.11. עיבוד תמונה

עיבוד תמונה הוא תחום העוסק בקליטה, ייצוג, שינוי וניתוח של תמונות דיגיטליות באמצעות מחשב. תמונה דיגיטלית מתקבלת בדרך כלל מחיישן צילום, מצלמה או מקור חזותי אחר, ולאחר מכן מיוצגת במחשב כאוסף ערכים מספריים. ערכים אלו מייצגים את עוצמת האור או את רכיבי הצבע בכל נקודה בתמונה.

במערכות הנדסיות נעשה שימוש בעיבוד תמונה לצורך שיפור איכות תמונה, גילוי עצמים, זיהוי תבניות, מדידת מאפיינים חזותיים וקבלת החלטות על סמך מידע חזותי. תחום זה משמש במגוון רחב של יישומים, כגון מערכות אבטחה, מערכות רפואיות, רובוטיקה, כלי רכב חכמים, בקרת איכות בתעשייה ומערכות זיהוי ביומטריות.

תמונה דיגיטלית מורכבת מפיקסלים. כל פיקסל מייצג נקודה קטנה בתמונה, ולכל פיקסל משויך ערך מספרי המתאר את עוצמת האור או את צבעו. ככל שמספר הפיקסלים בתמונה גדול יותר, כך רזולוציית התמונה גבוהה יותר וניתן לייצג פרטים קטנים יותר. עם זאת, רזולוציה גבוהה יותר מגדילה גם את כמות המידע שיש לעבד, ולכן היא דורשת יותר משאבי חישוב וזיכרון.

בתהליך עיבוד תמונה ניתן לבצע פעולות שונות על התמונה. חלק מהפעולות נועדו לשפר את איכות התמונה, כגון שינוי בהירות, שינוי ניגודיות, סינון רעשים וחידוד קצוות. פעולות אחרות נועדו להפיק מידע מתוך התמונה, כגון זיהוי אזורים, גילוי תבניות, איתור אובייקטים וזיהוי פנים.

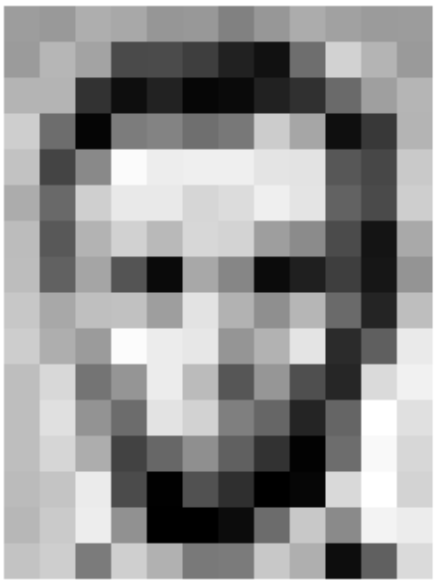
פעולות עיבוד תמונה יכולות להתבצע על תמונה בודדת או על רצף תמונות המתקבל מווידאו. כאשר מדובר בווידאו, כל פריים מעובד כתמונה בפני עצמה, ולעיתים מתבצע גם ניתוח של שינויי תנועה בין פריימים שונים. עיבוד מסוג זה מאפשר לזהות תנועה, לעקוב אחר עצמים ולהפיק מידע דינמי מתוך הסביבה המצולמת.

עיבוד תמונה מהווה בסיס לתחום רחב יותר הנקרא ראייה ממוחשבת. בעוד שעיבוד תמונה מתמקד בשינוי, שיפור וניתוח של התמונה עצמה, ראייה ממוחשבת עוסקת בהפקת משמעות מתוך המידע החזותי. לכן, תחום זה מהווה בסיס חשוב למערכות זיהוי עצמים, זיהוי פנים, בקרת איכות, ניווט רובוטי ומערכות אבטחה מתקדמות.

4.11.1. ייצוג תמונה דיגיטלית

תמונה דיגיטלית מיוצגת במחשב כאוסף סופי של נקודות הנקראות פיקסלים. כל פיקסל מייצג נקודה קטנה במרחב התמונה, ולכל פיקסל משויך ערך מספרי המתאר את עוצמת האור או את הצבע באותה נקודה. אוסף הפיקסלים יוצר מטריצה דו־ממדית, שבה כל תא במטריצה מתאים למיקום מסוים בתמונה.

רזולוציית התמונה נקבעת לפי מספר הפיקסלים בכיוון האופקי ובכיוון האנכי. לדוגמה, תמונה ברזולוציה של 1280×720 כוללת 1280 פיקסלים בכל שורה ו־720 שורות. ככל שמספר הפיקסלים גדול יותר, כך ניתן לייצג פרטים קטנים יותר בתמונה. עם זאת, הגדלת הרזולוציה מגדילה גם את כמות המידע שיש לשמור ולעבד.



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

איור 17 ייצוג תמונת פנים דיגיטלית כמטריצת ערכי פיקסלים

בתמונה בגווני אפור, כל פיקסל מיוצג בדרך כלל על ידי ערך יחיד. ערך זה מתאר את עוצמת הבהירות של הפיקסל. במקרים רבים נעשה שימוש בטווח ערכים בין 0 ל־255, כאשר הערך 0 מייצג שחור, הערך 255 מייצג לבן, והערכים שביניהם מייצגים רמות שונות של אפור.

בתמונה צבעונית, כל פיקסל מיוצג בדרך כלל באמצעות מספר ערוצי צבע. אחד המודלים הנפוצים הוא מודל RGB, שבו כל פיקסל מורכב משלושה רכיבים: אדום, ירוק וכחול. כל אחד מהרכיבים מיוצג בערך מספרי, ושילוב שלושת הערכים יוצר את הצבע הסופי של הפיקסל. גם במקרה זה, כל ערוץ צבע מיוצג לרוב בטווח ערכים שבין 0 ל־255.

ייצוג תמונה כמטריצה מאפשר לבצע עליה פעולות מתמטיות וחישוביות. ניתן לשנות ערכי פיקסלים, לבצע סינון, לזהות קצוות, למדוד בהירות, לאתר אזורים בעלי מאפיינים דומים ולהפעיל אלגוריתמים לזיהוי תבניות. לכן, ייצוג מטריציוני של תמונה מהווה בסיס מרכזי לעיבוד תמונה ולראייה ממוחשבת.

בנוסף, קיימת חשיבות למרחב הצבע שבו מיוצגת התמונה. מרחב הצבע קובע כיצד הערכים המספריים מתורגמים לצבעים או לעוצמות אור. בחירת מרחב צבע מתאים יכולה להשפיע על מורכבות העיבוד, על מהירות החישוב ועל דיוק האלגוריתמים המופעלים על התמונה.

4.11.2. ראייה ממוחשבת

ראייה ממוחשבת היא תחום במדעי המחשב ובהנדסה העוסק בהפקת מידע ומשמעות מתוך תמונות או וידאו באמצעות מחשב. מטרת התחום היא לאפשר למערכת ממוחשבת לנתח מידע חזותי, לזהות תבניות, לאתר עצמים, לסווג מידע ולקבל החלטות על סמך תוכן התמונה.

בעוד שעיבוד תמונה מתמקד בעיקר בשיפור, שינוי וניתוח של התמונה עצמה, ראייה ממוחשבת עוסקת בהבנת המידע הקיים בתמונה. כלומר, עיבוד תמונה יכול לשמש כשלב מקדים שבו התמונה עוברת המרה, סינון או שיפור, ואילו ראייה ממוחשבת משתמשת בתוצאה כדי להסיק מסקנות לגבי העצמים או האירועים שמופיעים בתמונה.

במערכות ראייה ממוחשבת נעשה שימוש באלגוריתמים המזהים מאפיינים חזותיים בתמונה. מאפיינים אלו יכולים לכלול קצוות, אזורים בעלי ניגודיות גבוהה, צורות גאומטריות, מרקמים, צבעים, תבניות חוזרות או יחסים מרחביים בין חלקים שונים בתמונה. באמצעות ניתוח מאפיינים אלו ניתן לאתר עצמים, לעקוב אחר תנועה, לזהות תבניות ולסווג אובייקטים.

תחום הראייה הממוחשבת משמש במגוון רחב של יישומים, כגון מערכות אבטחה, בקרת איכות בתעשייה, רובוטיקה, כלי רכב אוטונומיים, מערכות רפואיות, מערכות זיהוי ביומטריות ומערכות מעקב. במערכות אלו המצלמה משמשת כרכיב קלט, והאלגוריתמים מאפשרים להפיק מידע שימושי מתוך התמונה שנקלטה.

איכות פעולת הראייה הממוחשבת מושפעת ממספר גורמים, ובהם רזולוציית התמונה, תנאי התאורה, איכות המצלמה, זווית הצילום, רעשי תמונה, חדות התמונה ואיכות האלגוריתם שבו נעשה שימוש. ככל שתנאי הקלט טובים יותר וככל שהאלגוריתם מתאים יותר למשימה, כך ניתן לקבל תוצאה אמינה ומדויקת יותר.

4.11.3. גילוי פנים וזיהוי פנים

גילוי פנים וזיהוי פנים הם שני תהליכים שונים בתחום הראייה הממוחשבת. אף על פי שלעיתים נעשה שימוש במונחים אלו יחד, קיימת ביניהם הבחנה חשובה. גילוי פנים עוסק באיתור קיומן של פנים בתמונה, ואילו זיהוי פנים עוסק בקביעת זהותו של האדם שפניו הופיעו בתמונה.

גילוי פנים הוא השלב שבו נבדקת תמונה במטרה למצוא אזורים המכילים פנים אנושיות. בשלב זה לא נקבעת זהותו של האדם, אלא רק מזוהה האם קיימת בתמונה תבנית המתאימה למבנה פנים. אלגוריתמים לגילוי פנים מחפשים בדרך כלל מאפיינים אופייניים כגון אזור העיניים, האף, הפה, קווי המתאר של הפנים והיחסים המרחביים בין חלקי הפנים.

זיהוי פנים הוא שלב מתקדם יותר, המתבצע לאחר שאותר אזור פנים בתמונה. מטרת שלב זה היא לבדוק למי שייכות הפנים שזוהו. לצורך כך נדרש בדרך כלל מאגר תמונות או מודל שאומן מראש על תמונות של אנשים ידועים. האלגוריתם מחלץ מאפיינים מתוך תמונת הפנים ומשווה אותם למאפיינים שנשמרו במודל האימון.

ההבדל המרכזי בין שני התהליכים הוא סוג השאלה שעליה כל אחד מהם עונה. גילוי פנים עונה על השאלה האם קיימים פנים בתמונה והיכן הם נמצאים. זיהוי פנים עונה על השאלה למי שייכות הפנים שאותרו. לכן, במערכות רבות מבוצע תחילה גילוי פנים, ורק אם נמצאו פנים בתמונה מתבצע לאחר מכן תהליך זיהוי פנים.

דיוק תהליך גילוי וזיהוי הפנים מושפע ממספר גורמים, כגון תנאי התאורה, זווית הפנים ביחס למצלמה, איכות התמונה, חדות התמונה, הבעות פנים, הסתרות חלקיות, גודל מאגר האימון ואיכות האלגוריתם. כאשר תנאי הצילום טובים יותר וכאשר מאגר האימון מייצג בצורה טובה את המשתמשים, ניתן לקבל תוצאות זיהוי מדויקות יותר.

גילוי וזיהוי פנים משמשים במערכות רבות, כגון בקרת כניסה, טלפונים חכמים, מערכות נוכחות, מערכות אבטחה, מערכות מעקב וזיהוי משתמשים. השימוש בטכנולוגיה זו מאפשר זיהוי ביומטרי ללא צורך במגע פיזי, ולכן הוא מתאים ליישומים שבהם נדרשת נוחות שימוש לצד בקרת גישה.

4.11.4. **ספריית** OpenCV

OpenCV היא ספריית קוד פתוח המשמשת לעיבוד תמונה, עיבוד וידאו וראייה ממוחשבת. הספרייה כוללת אוסף רחב של פונקציות וכלים המאפשרים לבצע פעולות שונות על תמונות, כגון קריאת תמונה, שמירת תמונה, המרה בין מרחבי צבע, שינוי גודל, סינון, זיהוי קצוות, גילוי אובייקטים, עבודה עם מצלמות והפעלת אלגוריתמים לזיהוי תבניות.

הספרייה נחשבת לאחת הספריות הנפוצות בתחום הראייה הממוחשבת, מאחר שהיא מאפשרת לבצע פעולות עיבוד תמונה בצורה יעילה ונוחה יחסית .OpenCV תומכת במספר שפות תכנות, ובהן Python ו־ ,C++ ולכן היא מתאימה הן ללמידה ולמחקר והן לפיתוח מערכות מעשיות.

אחד היתרונות המרכזיים של OpenCV הוא היכולת לעבוד עם מקורות וידאו ותמונות בזמן אמת. באמצעות הספרייה ניתן לפתוח התקן מצלמה, לקלוט פריימים, להציג תמונות, לשמור קבצים ולבצע עיבוד על כל תמונה שנקלטה. יכולת זו חשובה במערכות שבהן נדרש ניתוח חזותי של הסביבה בזמן פעולה.

OpenCV מאפשרת לבצע פעולות בסיסיות רבות בתמונה, כגון המרה מתמונה צבעונית לתמונה בגווני אפור, שינוי בהירות וניגודיות, טשטוש, סינון רעשים, חיתוך אזורי עניין ושינוי רזולוציה. פעולות אלו משמשות לעיתים כשלב מקדים לפני הפעלת אלגוריתמים מתקדמים יותר.

בנוסף לפעולות בסיסיות ,OpenCV כוללת כלים מתקדמים לגילוי עצמים ולזיהוי תבניות. ניתן להשתמש במסווגים מאומנים מראש לצורך גילוי פנים, עיניים או עצמים אחרים בתמונה. כמו כן, הספרייה כוללת כלים לעבודה עם מודלים לזיהוי פנים, חילוץ מאפיינים, השוואת תמונות ומעקב אחר עצמים בווידאו.

השימוש ב־ OpenCV מאפשר לפתח מערכות שבהן מתקבל מידע חזותי ממצלמה, מתבצע עליו עיבוד, ולאחר מכן מתקבלת החלטה בהתאם לתוכן התמונה. לכן, הספרייה מהווה כלי מרכזי במערכות ראייה ממוחשבת, מערכות אבטחה, רובוטיקה, זיהוי פנים, בקרת איכות ומערכות חכמות נוספות.

4.11.5. **אלגוריתם** Haar Cascade

Haar Cascade הוא אלגוריתם נפוץ בתחום הראייה הממוחשבת, המשמש לגילוי עצמים בתמונה, ובמיוחד לגילוי פנים. האלגוריתם מבוסס על זיהוי תבניות חזותיות פשוטות, אשר נקראות מאפייני Haar מאפיינים אלו מתארים הבדלים בעוצמת האור בין אזורים סמוכים בתמונה, כגון אזור בהיר ליד אזור כהה או שינוי חד בין שני אזורים.

עקרון הפעולה של האלגוריתם מבוסס על כך שלפנים אנושיות קיימות תבניות חזותיות אופייניות. לדוגמה, אזור העיניים נוטה להיות כהה יותר מאזור הלחיים, אזור האף יוצר שינויי בהירות מסוימים, וקיימים יחסים מרחביים אופייניים בין חלקי הפנים. באמצעות בדיקת תבניות מסוג זה ניתן להעריך האם אזור מסוים בתמונה מכיל פנים.

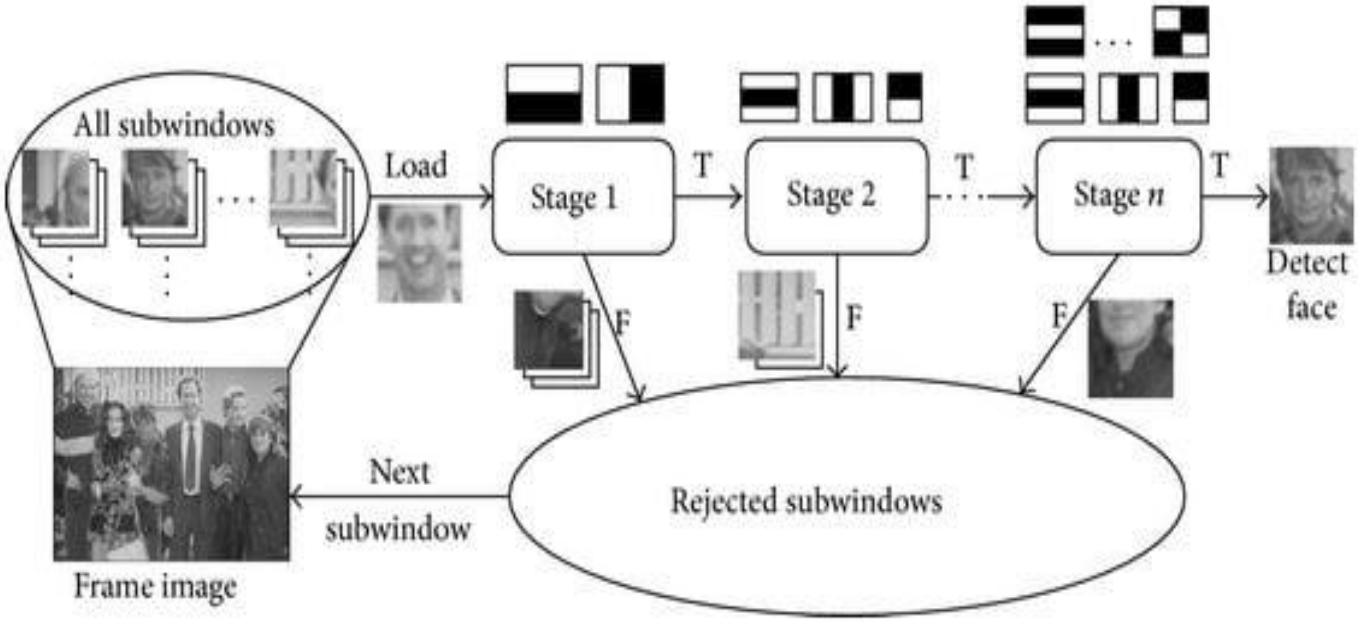
בשלב הראשון, התמונה נסרקת באמצעות חלונות בגודל משתנה. בכל חלון נבדקים מאפייני Haar שונים, אשר משווים בין סכומי עוצמות הפיקסלים באזורים שונים של החלון. אם התבניות שנמצאו מתאימות למאפיינים שנלמדו מראש, החלון עשוי להיחשב כאזור המכיל את העצם המבוקש.

המונח Cascade מתייחס למבנה מדורג של מסווגים. במקום לבדוק את כל המאפיינים בכל אזור בתמונה, האלגוריתם משתמש במספר שלבים. בשלבים הראשונים נבדקים מאפיינים פשוטים ומהירים לחישוב. אם אזור מסוים אינו מתאים כבר בשלבים הראשונים, הוא נפסל מיד ואינו ממשיך לשלבים הבאים. רק אזורים שעברו את השלבים הראשונים נבדקים באמצעות מאפיינים נוספים ומורכבים יותר.

שיטה זו מאפשרת לבצע גילוי עצמים בצורה יעילה יחסית, משום שרוב האזורים בתמונה אינם מכילים את העצם המבוקש ונפסלים במהירות. כתוצאה מכך נחסך זמן עיבוד, והאלגוריתם מסוגל לפעול גם במערכות שבהן משאבי החישוב מוגבלים.

לצורך שימוש ב־ Haar Cascade נדרש מסווג שאומן מראש. מסווג זה נבנה על בסיס אוסף תמונות חיוביות, המכילות את העצם שאותו רוצים לגלות, ואוסף תמונות שליליות, שאינן מכילות את העצם. לאחר האימון מתקבל קובץ מסווג, אשר ניתן להשתמש בו לצורך גילוי האובייקט בתמונות חדשות.

יתרונו של Haar Cascade הוא בפשטות היחסית שלו ובמהירות הפעולה שלו. הוא מתאים במיוחד למשימות גילוי בסיסיות, כגון גילוי פנים בתמונה. עם זאת, ביצועיו עשויים להיות מושפעים מתנאי תאורה, זווית צילום, הסתרות חלקיות ואיכות התמונה. לכן, במקרים רבים נדרשת תמונה ברורה יחסית ותנאי צילום סבירים כדי לקבל תוצאות גילוי אמינות.



איור 18 תהליך גילוי פנים באלגוריתם haar cascade

4.11.6. אלגוריתם LBPH

LBPH הוא אלגוריתם לזיהוי פנים המבוסס על ניתוח תבניות מקומיות בתמונה. ראשי התיבות LBPH מציינים Local Binary Patterns Histograms האלגוריתם משלב בין שיטת Local Binary Patterns המשמשת לחילוץ מאפיינים מקומיים מתוך התמונה, לבין שימוש בהיסטוגרמות לצורך ייצוג והשוואה של מאפיינים אלו.

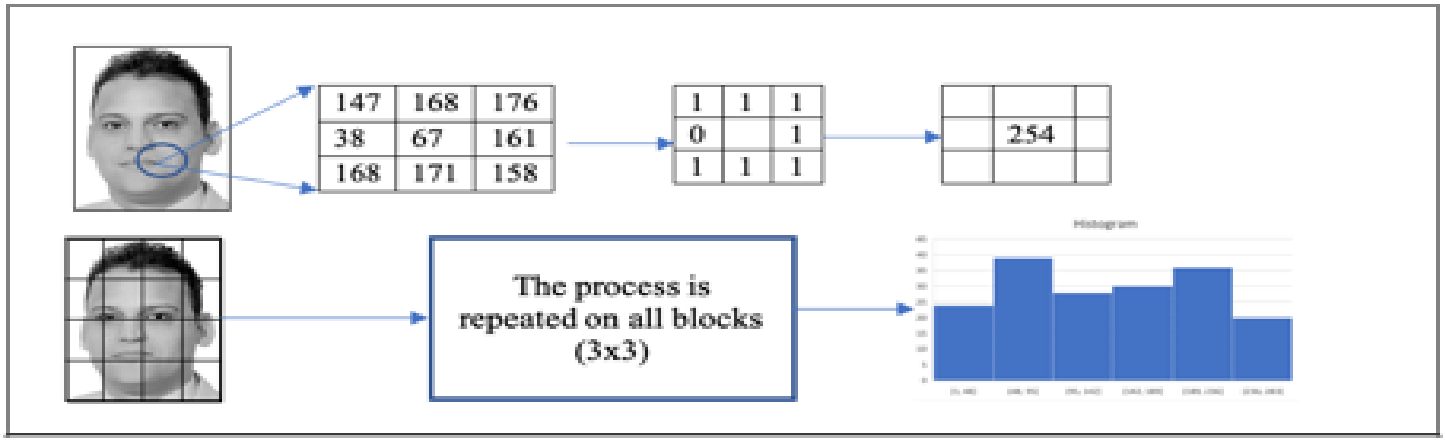
עקרון הפעולה של Local Binary Patterns מבוסס על השוואה בין כל פיקסל לבין הפיקסלים הסמוכים אליו. עבור כל אזור קטן בתמונה נבחר פיקסל מרכזי, וכל אחד מהפיקסלים השכנים מושווה אליו. אם ערך הפיקסל השכן גדול או שווה לערך הפיקסל המרכזי, מתקבל ערך בינארי 1. אם ערכו קטן יותר, מתקבל ערך בינארי 0. רצף הערכים הבינאריים שנוצר מייצג את התבנית המקומית סביב אותו פיקסל.

באמצעות תהליך זה מתקבלת תמונה חדשה, שבה כל פיקסל מייצג תבנית מקומית של מרקם או שינויי בהירות באזור קטן בתמונה. תבניות אלו יכולות לתאר מאפיינים חזותיים כגון קצוות, אזורים כהים ובהירים, שינויי מרקם ומבנים מקומיים נוספים. מאחר שפנים אנושיות כוללות מאפיינים מקומיים רבים, שיטה זו מתאימה לייצוג הבדלים בין תמונות פנים שונות.

לאחר חישוב תבניות ה- LBP התמונה מחולקת לאזורים קטנים. עבור כל אזור מחושבת היסטוגרמה, המתארת את שכיחות הערכים המקומיים שהתקבלו באותו אזור. לאחר מכן מחוברות ההיסטוגרמות של כל האזורים לוקטור מאפיינים אחד, המייצג את תמונת הפנים כולה.

בשלב הזיהוי, וקטור המאפיינים של תמונת פנים חדשה מושווה לוקטורי מאפיינים שנשמרו בשלב האימון. ההשוואה מתבצעת באמצעות מדד מרחק, כגון מרחק אוקלידי או מדד דומה. ככל שהמרחק בין הוקטורים קטן יותר, כך הדמיון בין תמונות הפנים גבוה יותר. אם נמצא דמיון מספק למודל שנשמר, ניתן לשייך את תמונת הפנים לאדם מסוים.

יתרונו של LBPH הוא בפשטות היחסית שלו וביכולתו לפעול גם עם כמות אימון לא גדולה במיוחד. בנוסף, האלגוריתם מתבסס על תבניות מקומיות, ולכן הוא עשוי להיות עמיד יחסית לשינויים מסוימים בתאורה ובמרקם התמונה. עם זאת, איכות הזיהוי עדיין תלויה בגורמים כגון איכות התמונה, זווית הפנים, תנאי התאורה, כמות תמונות האימון וגיוון התמונות במאגר.



איור 19 דיאגרמה עבור אלגורתים LBPH

LBPH נחשב לאלגוריתם מתאים למערכות זיהוי פנים בסיסיות ובינוניות, שבהן נדרש איזון בין פשטות מימוש לבין יכולת זיהוי מעשית. השיטה מאפשרת לאמן מודל על תמונות פנים ידועות, ולאחר מכן להשתמש במודל לצורך השוואה וזיהוי של תמונות חדשות.

4.12. בוט טלגרם

Telegram היא פלטפורמת תקשורת המאפשרת שליחת הודעות, תמונות, קבצים ועדכונים בין משתמשים. בנוסף לשימוש הרגיל באפליקציה Telegram, מאפשרת יצירת בוטים באמצעות ממשק תכנות יישומים הנקרא Telegram Bot API בוט הוא רכיב תוכנה המסוגל לשלוח ולקבל הודעות באופן אוטומטי בהתאם ללוגיקה שהוגדרה מראש.

Telegram Bot API מאפשר למערכות תוכנה לתקשר עם משתמשים דרך שרתי Telegram לאחר יצירת בוט מתקבל מזהה ייחודי הנקרא Token המשמש לזיהוי הבוט מול השרתים. באמצעות Token זה ניתן לשלוח בקשות אל ממשק ה- APIכגון שליחת הודעת טקסט, שליחת תמונה, קבלת עדכונים או ביצוע פעולות נוספות.

כדי לשלוח הודעה למשתמש מסוים נדרש מזהה שיחה הנקרא Chat ID מזהה זה מאפשר לבוט לדעת לאיזו שיחה או לאיזה משתמש יש לשלוח את ההודעה. שילוב בין Token לבין Chat ID מאפשר למערכת חיצונית לשלוח התראות באופן אוטומטי למשתמש המתאים.

אחד היתרונות המרכזיים של שימוש ב- Telegram Bot API הוא הפשטות היחסית של שליחת התראות בזמן אמת. מערכת תוכנה יכולה לשלוח בקשת HTTP לשרתי Telegram וכתוצאה מכך מתקבלת הודעה באפליקציה של המשתמש. בדרך זו ניתן להעביר מידע על אירועים, תקלות, מצבי מערכת או התראות אבטחה ללא צורך בפיתוח אפליקציה ייעודית.

בנוסף להודעות טקסט Telegram Bot API, מאפשר שליחת תמונות וקבצים. יכולת זו חשובה במערכות שבהן נדרש להעביר למשתמש מידע חזותי או תיעוד של אירוע מסוים. שליחת תמונה מאפשרת למשתמש לקבל מידע עשיר יותר מאשר הודעת טקסט בלבד, ולכן היא נפוצה במערכות ניטור, אבטחה ובקרה.

עם זאת, בשימוש ב־ Telegram Bot API קיימת חשיבות לשמירה על אבטחת המידע. ה־ Token של הבוט משמש כמפתח גישה, ולכן יש לשמור עליו ולא לחשוף אותו בקוד ציבורי או במסמכים שאינם מוגנים. במקרה שבו ה־ Token נחשף, יש להחליפו באמצעות BotFather כדי למנוע שימוש לא מורשה בבוט.

לכן Telegram Bot API ,מהווה כלי יעיל לשילוב תקשורת והתראות במערכות משובצות ובמערכות תוכנה. הוא מאפשר שליחת מידע למשתמש בזמן אמת, תומך בהודעות טקסט ובתמונות, ומספק דרך פשוטה לחיבור בין מערכת אוטומטית לבין משתמש קצה.

4.13. מערכת systemd

systemd הוא מנהל שירותים ותהליכים במערכות הפעלה מבוססות Linux תפקידו המרכזי הוא לנהל את תהליך האתחול של מערכת ההפעלה, להפעיל שירותים באופן אוטומטי, לפקח על תהליכים פעילים ולספק מנגנון מסודר לניהול שירותי מערכת.

במערכות Linux קיימים שירותים רבים אשר נדרשים לפעול ברקע, כגון שירותי רשת, שירותי תקשורת, שירותי קבצים, שרתי תוכנה ותהליכי בקרה שונים systemd מאפשר להגדיר עבור כל שירות קובץ תצורה ייעודי, שבו נקבעים פרטי השירות, תנאי ההפעלה, פקודת ההרצה, אופן ההתנהגות במקרה של כשל ותלות בשירותים אחרים.

קובץ שירות של systemd כולל בדרך כלל מספר חלקים מרכזיים. חלק ה־ Unit משמש להגדרת שם השירות, תיאורו ותלויותיו. חלק ה־ Service מגדיר את פקודת ההפעלה, תיקיית העבודה, המשתמש שמריץ את השירות, אופן ההפעלה והתגובה במקרה של עצירה או כשל. חלק ה־ Install מגדיר באיזה שלב של עליית המערכת השירות יופעל, והאם הוא ייטען באופן אוטומטי בעת אתחול.

אחד היתרונות המרכזיים של systemd הוא היכולת להפעיל תוכנה באופן אוטומטי לאחר עליית מערכת ההפעלה. יכולת זו חשובה במיוחד במערכות משובצות מחשב, שבהן נדרש שהמערכת תתחיל לפעול ללא צורך בהתערבות ידנית של המשתמש. כאשר השירות מוגדר ומופעל כראוי, ניתן לאתחל את המערכת, והתוכנה תורץ באופן עצמאי לאחר סיום תהליך האתחול.

בנוסף systemd ,מאפשר לעקוב אחר מצב השירות. באמצעות פקודות מתאימות ניתן לבדוק האם השירות פעיל, האם הוא נכשל, מתי הופעל לאחרונה ומה הודפס ביומני המערכת. מערכת היומנים של Linux מאפשרת לצפות בלוגים של השירות, לאתר שגיאות ולבצע תהליך ניפוי תקלות מסודר.

יתרון נוסף של systemd הוא האפשרות להגדיר הפעלה מחדש אוטומטית במקרה של כשל. כאשר שירות מוגדר עם מדיניות הפעלה מחדש, מערכת ההפעלה יכולה להפעיל אותו מחדש אם הוא נעצר באופן בלתי צפוי. תכונה זו משפרת את אמינות המערכת ומאפשרת לה להתמודד טוב יותר עם תקלות זמניות.

במערכות משובצות, שימוש ב־ systemd מאפשר להפוך תוכנה רגילה לשירות מערכת עצמאי. כך ניתן להפעיל תוכניות בקרה, תוכניות תקשורת, מערכות ניטור או תהליכים אוטומטיים מיד לאחר עליית המערכת, תוך שמירה על יכולת בקרה, ניטור ותיעוד.

5. מימוש המערכת

בפרק זה תוארה מערכת המנעול החכם כמערכת משובצת מחשב, אשר שילבה בין חומרה דיגיטלית, תוכנה, חיישנים, עיבוד תמונה ומנגנוני אבטחה. מטרת המערכת הייתה לאפשר בקרת כניסה חכמה לדלת, כך שפתיחת המנעול בוצעה רק לאחר זיהוי משתמש מורשה או לאחר הזנת סיסמה תקינה בלוח המקשים.

המערכת התבססה על כרטיס הפיתוח PYNQ-Z2 אשר שימש כיחידת הבקרה המרכזית. בכרטיס זה שולבו שני חלקים עיקריים: חלק תוכנתי המבוסס על מעבד ARM והרצת קוד Python וחלק חומרתי המבוסס על FPGA שבו מומשה חומרה ייעודית בשפת VHDL. באמצעות שילוב זה התאפשרה חלוקה ברורה בין פעולות תוכנה מורכבות, כגון עיבוד תמונה, זיהוי פנים, ניהול לוגים ושליחת התראות, לבין פעולות חומרה דיגיטליות, כגון הפעלת מנוע הצעד, קריאת חיישנים והפעלת רכיבי חיווי.

מערכת המנעול החכם כללה מספר תתי־מערכות מרכזיות. תת־מערכת זיהוי התנועה התבססה על חיישן PIR, אשר זיהה תנועה באזור הדלת והפעיל את תהליך הצילום והזיהוי. תת־מערכת הצילום התבססה על מצלמת USB מסוג Logitech Brio 300 אשר שימשה לצילום אזור הכניסה לאחר זיהוי תנועה. לאחר צילום התמונה, בוצע עיבוד תמונה באמצעות OpenCV, ובו תחילה בוצע גילוי פנים באמצעות Haar Cascade ולאחר מכן בוצע זיהוי פנים באמצעות אלגוריתם LBPH

בנוסף למנגנון זיהוי הפנים, שולב במערכת לוח מקשים מטריציוני 4 על 4 אשר שימש כאמצעי גיבוי לפתיחת המנעול באמצעות סיסמה. מנגנון זה נועד לאפשר פתיחה גם במקרים שבהם זיהוי הפנים לא בוצע בהצלחה, לדוגמה עקב תנאי תאורה, זווית צילום או כשל בזיהוי. כמו כן, שולב חיישן דלת מגנטי מסוג MC-38 אשר שימש לזיהוי מצב הדלת ואפשר למערכת לדעת האם הדלת פתוחה או סגורה לפני ביצוע נעילה.

מנגנון הפתיחה והנעילה מומש באמצעות מנוע צעד יחד עם דרייבר ULN2003 מנוע הצעד שימש לייצוג מנגנון הנעילה הפיזי, כאשר סיבוב בכיוון אחד ייצג פתיחת מנעול וסיבוב בכיוון ההפוך ייצג נעילה. השליטה במנוע בוצעה באמצעות חומרה שתוכננה ב־FPGA בשפת VHDL והופעלה מתוך תוכנת Python באמצעות ממשקי AXI GPIO

לצורך התאמה לתנאי סביבה, שולב במערכת חיישן אור דיגיטלי מסוג LM393 חיישן זה שימש לזיהוי תנאי תאורה באזור הדלת. כאשר זוהתה חשכה, הופעל ממסר אשר שימש להפעלת תאורה חיצונית, במטרה לשפר את איכות התמונה ולאפשר גילוי וזיהוי פנים בתנאי תאורה ירודים.

מערכת האבטחה כללה גם מנגנון לניהול ניסיונות גישה שגויים. כאשר בוצעו ניסיונות פתיחה כושלים, המערכת ספרה את מספר הניסיונות וניהלה את מצב האבטחה בהתאם. במקרה של ניסיון גישה חריג, נשלחה התראה בזמן אמת באמצעות Telegram, ובמידת הצורך נשלחה גם תמונה של האדם שנמצא באזור הדלת. בנוסף, פעולות המערכת תועדו בקובצי לוג, לצורך בדיקה, מעקב וניתוח תוצאות.

בסך הכול, המערכת תוכננה כמערכת משולבת שבה כל רכיב ביצע תפקיד מוגדר בתהליך בקרת הכניסה. החיישנים שימשו לזיהוי אירועים פיזיים בסביבת הדלת, התוכנה שימשה לעיבוד המידע ולקבלת החלטות, והחומרה הדיגיטלית שימשה להפעלת מנגנון הנעילה והחיווי. שילוב זה אפשר יצירת מערכת מנעול חכם בעלת יכולת פעולה אוטומטית, יכולת גיבוי באמצעות לוח מקשים, תיעוד אירועים ושליחת התראות בזמן אמת.

5.1. רשימת ציוד

טבלה 9 רשימת ציוד חומרה

מספר	שם הרכיב / הציוד	סוג הרכיב	תפקיד בפרויקט
1	PYNQ-Z2	כרטיס פיתוח FPGA	יחידת הבקרה המרכזית של המערכת
2	Logitech Brio 300	מצלמת USB	צילום אזור הכניסה לצורך גילוי וזיהוי פנים
3	חיישן PIR	חיישן תנועה	זיהוי תנועה באזור הדלת והפעלת תהליך הצילום
4	חיישן אור LM393	חיישן תאורה דיגיטלי	זיהוי תנאי תאורה בסביבת הדלת
5	חיישן דלת MC-38	חיישן מגנטי	זיהוי מצב הדלת, פתוחה או סגורה
6	לוח מקשים 4x4	רכיב קלט	הזנת סיסמה לפתיחת המנעול כאמצעי גיבוי
7	מנוע צעד 28BYJ-48	רכיב הנעה	ביצוע פעולת הפתיחה והנעילה
8	דרייבר ULN2003	דרייבר למנוע צעד	הפעלת מנוע הצעד באמצעות זרם מתאים
9	ממסר	רכיב מיתוג	הפעלת תאורה חיצונית בתנאי חשכה
10	נורות LED	רכיב חיווי	חיווי מצבי פעולה כגון פתוח, נעול, תנועה ושגיאה
11	זמזם	רכיב חיווי קולי	מתן חיווי קולי בעת לחיצה, פתיחה או אירוע חריג
12	כבלי Jumper	ציוד חיבור	חיבור החיישנים והרכיבים אל כרטיס הפיתוח
13	Breadboard	ציוד עזר להרכבה	ביצוע חיבורים זמניים ובדיקות רכיבים
14	חיבור Ethernet	תקשורת	חיבור הכרטיס לרשת לצורך וגישה מרחוק Telegram
15	מחשב אישי	סביבת פיתוח ובקרה	עבודה מול Jupyter, Vivado ובדיקות המערכת
16	כרטיס microSD	אמצעי אחסון	הפעלת מערכת PYNQ ושמירת קבצי הפרויקט

טבלה 13 רשימת ציוד תוכנה

מספר	שם הכלי / הספרייה	סוג	תפקיד בפרויקט
1	Vivado	סביבת פיתוח חומרה	תכנון חומרה ב-VHDL, יצירת bit והפקת קובץ Block Design
2	Jupyter Notebook	סביבת פיתוח תוכנה	הרצת קוד Python על גבי כרטיס PYNQ-Z2
3	Python	שפת תכנות	מימוש לוגיקת המערכת, קריאת חיישנים וניהול התראות
4	OpenCV	ספריית עיבוד תמונה	ביצוע גילוי פנים וזיהוי פנים מתוך תמונות ווידאו
5	Haar Cascade	אלגוריתם גילוי פנים	איתור פנים בתמונה לפני שלב זיהוי הפנים
6	LBPH	אלגוריתם זיהוי פנים	זיהוי משתמש מורשה לפי מודל שאומן מראש
7	Telegram Bot API	שירות התראות	שליחת התראות ותמונות במקרה של ניסיון גישה חריג
8	systemd	מנגנון הפעלה אוטומטית	הפעלת מערכת המנעול כשירות אוטומטי בעת עליית המערכת

5.2. הפעולה הכללית של המערכת

מערכת המנעול החכם תוכננה כמערכת בקרת כניסה עצמאית, אשר קיבלה החלטות פתיחה ונעילה בהתאם למידע שהתקבל מסביבת הדלת. תפיסת הפעולה של המערכת התבססה על הפעלה לפי אירועים, ולא על הפעלה רציפה של כלל הרכיבים בכל רגע נתון. בצורה זו הופעלו תהליכי הצילום, עיבוד התמונה וזיהוי הפנים רק כאשר התקבל אירוע מתאים, כגון זיהוי תנועה באזור הדלת או הזנת קוד בלוח המקשים.

המערכת כללה שני מסלולי גישה עיקריים. מסלול הגישה הראשון היה מסלול אוטומטי, שבו בוצע ניסיון לפתיחת המנעול לאחר זיהוי משתמש מורשה באמצעות זיהוי פנים. מסלול הגישה השני היה מסלול ידני, שבו התאפשרה פתיחת המנעול באמצעות הזנת סיסמה בלוח המקשים. שילוב שני מסלולי הגישה נועד לשפר את אמינות המערכת, כך שגם במקרה שבו זיהוי הפנים לא בוצע בהצלחה, עדיין נשמרה אפשרות לפתיחת המנעול באמצעות קוד גישה.

במצב ההתחלתי של המערכת המנעול הוגדר כנעול, וכל רכיבי הבקרה הוגדרו למצב המתנה. כאשר זוהתה תנועה באזור הדלת, הופעל תהליך צילום, ולאחר מכן בוצעה בדיקה האם קיימות פנים בתמונה. אם לא נמצאו פנים, האירוע לא אושר כאירוע כניסה, והמערכת חזרה למצב המתנה. אם נמצאו פנים, בוצע תהליך זיהוי מול מודל שאומן מראש. כאשר התקבל זיהוי תקין של משתמש מורשה, נשלחה פקודת פתיחה למנגנון הנעילה.

לאחר פתיחת המנעול, המערכת לא ביצעה נעילה מיידית באופן עיוור, אלא המשיכה לעקוב אחר מצב הדלת. נעילה מחדש בוצעה רק לאחר שהתקבלה אינדיקציה לכך שהדלת נסגרה. בדרך זו נמנעה הפעלת מנגנון הנעילה בזמן שהדלת עדיין פתוחה, ונשמר רצף פעולה תקין בין שלבי הפתיחה, המעבר והנעילה.

בנוסף, שולב במערכת מנגנון אבטחה לניהול ניסיונות גישה כושלים. כאשר בוצעה הזנת סיסמה שגויה או כאשר התקבל ניסיון גישה לא מורשה, עודכן מונה ניסיונות הכניסה. במקרה שבו מספר הניסיונות הכושלים הגיע לערך שהוגדר מראש, הופעלה תגובת אבטחה שכללה תיעוד האירוע ושליחת התראה למשתמש. כך נוצרה מערכת שלא רק פתחה ונעלה את הדלת, אלא גם ניהלה מצבי חריגה בצורה מבוקרת.

5.3. מצבי פעולה של המערכת

פעולת המערכת חולקה למספר מצבים עיקריים, כאשר כל מצב ייצג שלב מוגדר בתהליך בקרת הכניסה. חלוקה זו אפשרה לנהל את המערכת בצורה מסודרת, למנוע פעולות לא רצויות, ולהבטיח שכל פעולה תתבצע רק לאחר קבלת התנאים המתאימים.

מצב ברירת המחדל של המערכת היה מצב נעול. במצב זה המנעול היה סגור, והמערכת המתינה לאירוע כניסה. בזמן ההמתנה נדגמו הקלטים הרלוונטיים, ובהם חיישן התנועה ולוח המקשים. כל עוד לא התקבל אירוע מתאים, המערכת נשארה במצב זה ולא הופעלה פעולה נוספת.

כאשר התקבלה אינדיקציה לתנועה באזור הדלת, המערכת עברה למצב בדיקה אוטומטית. במצב זה בוצע צילום של אזור הכניסה, ולאחר מכן בוצע עיבוד ראשוני לתמונה. מטרת שלב זה הייתה לבדוק האם קיימות פנים בתמונה, ורק במקרה שבו נמצאו פנים בוצע מעבר לשלב הזיהוי.

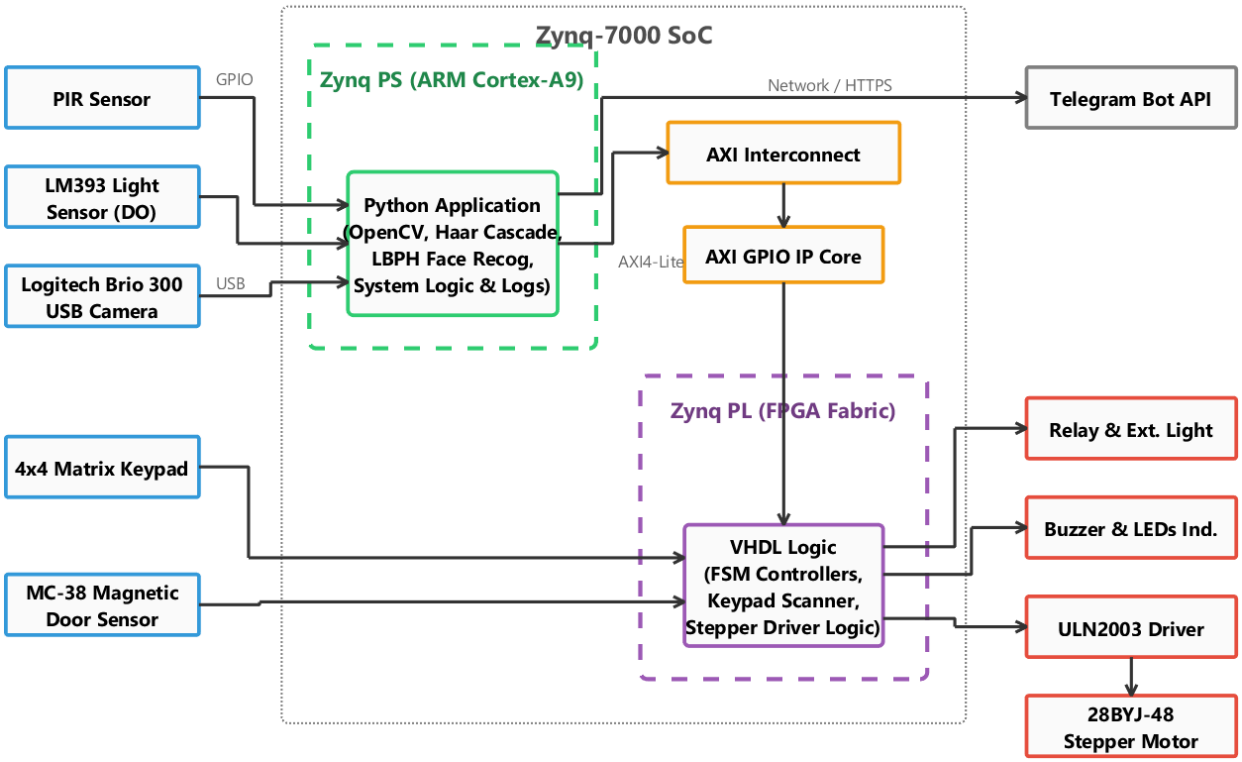
במצב זיהוי הפנים בוצעה השוואה בין תמונת הפנים שנקלטה לבין המודל שאומן מראש. אם התקבל זיהוי תקין של משתמש מורשה, המערכת עברה למצב פתיחה. אם לא התקבל זיהוי תקין, האירוע הוגדר כניסיון גישה לא מזוהה, והמערכת חזרה למצב המתנה או הפעילה תגובת אבטחה בהתאם למספר הניסיונות הכושלים.

במקביל למסלול האוטומטי, המערכת אפשרה מעבר למצב הזנת סיסמה. במצב זה נקלטו לחיצות מלוח המקשים, והקוד שהוזן הושווה לסיסמה שהוגדרה במערכת. אם הסיסמה הייתה תקינה, המערכת עברה למצב פתיחה. אם הסיסמה הייתה שגויה, עודכן מונה הניסיונות הכושלים.

במצב פתיחה הופעל מנגנון הנעילה בכיוון הפתיחה, ובוצע חיווי מתאים למשתמש. לאחר מכן עברה המערכת למצב המתנה לסגירת הדלת. במצב זה המערכת לא חזרה מיד למצב נעול, אלא המתינה לקבלת אינדיקציה מחיישן הדלת. רק לאחר שהדלת זוהתה כסגורה, הופעלה פעולת הנעילה.

במצב נעילה הופעל מנגנון הנעילה בכיוון ההפוך, ולאחר סיום הפעולה עודכנו חיווי המערכת בהתאם. בסיום שלב זה חזרה המערכת למצב ברירת המחדל, כלומר מצב נעול והמתנה לאירוע הבא.

5.4. סכמת בלוקים



איור 20 סכמת בלוקים של כלל המערכת

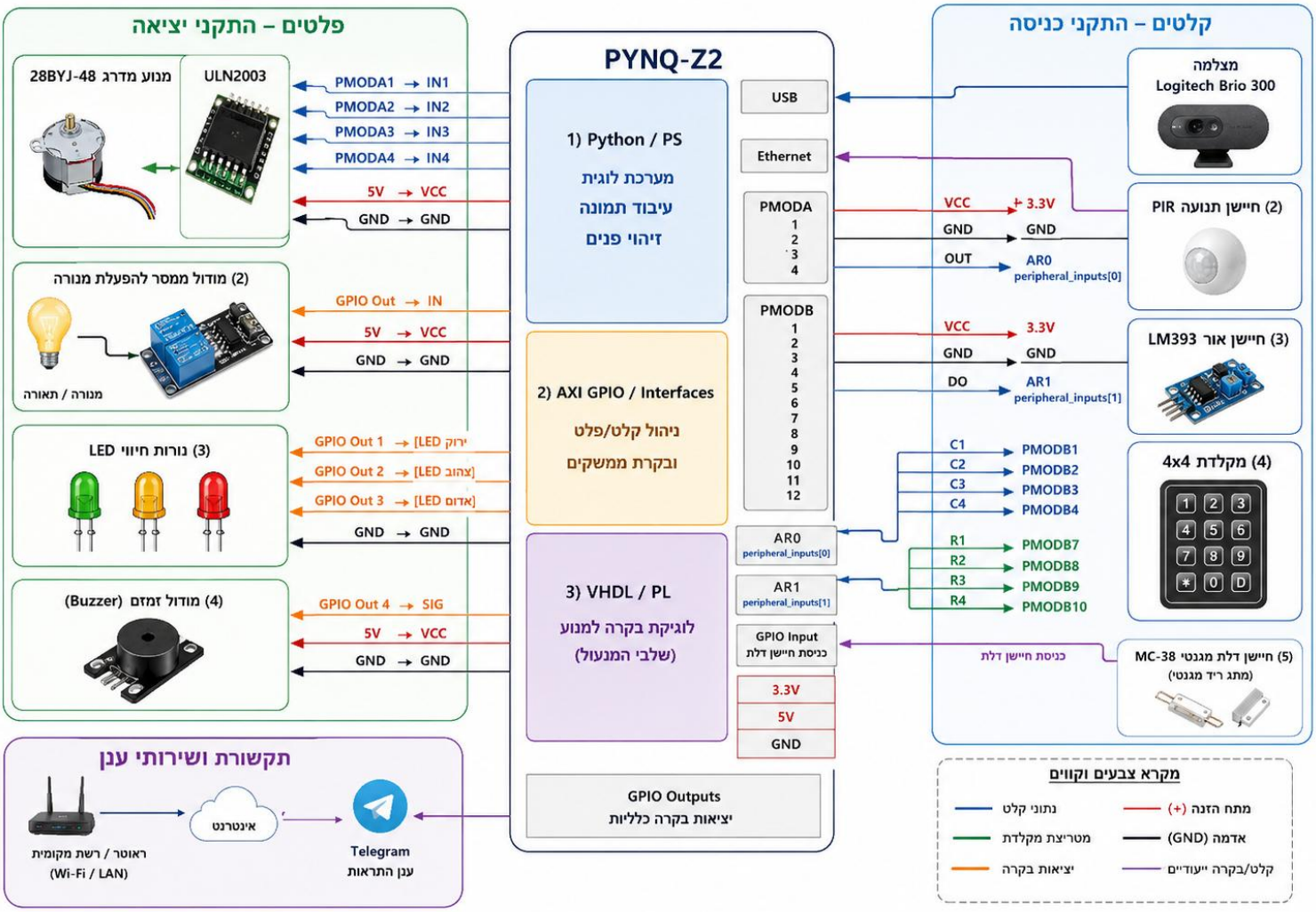
6. ארכיטקטורת החומרה

מטרת פרק זה הייתה לתאר את מבנה החומרה של מערכת המנעול החכם ואת אופן חיבור רכיבי הקלט והפלט אל כרטיס הפיתוח PYNQ-Z2 בפרק הקודם הוצגה המערכת ברמת־על, ואילו בפרק זה הוצגו החיבורים הפיזיים, תפקידי הכניסות והיציאות, ואופן שילוב הרכיבים החיצוניים עם יחידת הבקרה המרכזית.

המערכת התבססה על כרטיס PYNQ-Z2 אשר שימש כמרכז הבקרה של כלל רכיבי החומרה. אל הכרטיס חוברו רכיבי קלט, אשר סיפקו מידע על סביבת הדלת, ורכיבי פלט, אשר שימשו להפעלה, חיווי והתראה. רכיבי הקלט כללו חיישן תנועה, חיישן תאורה, חיישן דלת, לוח מקשים ומצלמת USB רכיבי הפלט כללו מנוע צעד, דרייבר מנוע, ממסר, נורות חיווי וזמזם.

החיבורים בין הרכיבים החיצוניים לבין כרטיס הפיתוח בוצעו באמצעות ממשקי PMOD וכניסות/יציאות דיגיטליות. אותות הקלט נקראו על ידי המערכת לצורך קבלת החלטות, ואילו אותות הפלט הופעלו בהתאם למצב הפעולה של המערכת. בנוסף, מנוע הצעד הופעל באמצעות דרייבר ULN2003 אשר שימש כשלב תיווך בין אותות הבקרה לבין זרמי ההפעלה הנדרשים למנוע.

6.1. תרשים חיבורים של המערכת



איור 21 תרשים חיבורים כללי של המערכת

6.2. טבלת חיבורים של המערכת

טבלה 17 חיבורים של המערכת

מספר	רכיב	נקודת חיבור בכרטיס	חיבורי הזנה	תפקיד במערכת
1	חיישן PIR	OUT → AR0 / peripheral_inputs[0]	VCC → 5V, GND → GND	זיהוי תנועה באזור הדלת והפעלת תהליך הצילום
2	חיישן אור LM393	DO → AR1 / peripheral_inputs[1]	VCC → 3.3V, GND → GND	זיהוי תנאי תאורה בסביבת הדלת
3	חיישן דלת MC-38	Signal → AR2	משותף בהתאם GND לחיבור החיישן	זיהוי מצב פתיחה או סגירה של הדלת
4	מצלמת Logitech Brio 300	USB	הזנה דרך USB	צילום אזור הכניסה לצורך גילוי וזיהוי פנים
5	לוח מקשים 4x4	C1-C4 → PMODB1-PMODB4, R1-R4 → PMODB7-PMODB10	ללא הזנת מתח נפרדת	סריקת מטריצת המקשים לצורך הזנת סיסמה
6	דרייבר ULN2003	IN1-IN4 → PMODA1-PMODA4	VCC → 5V, GND → GND	שליחת אותות בקרה לארבע כניסות הדרייבר
7	מנוע צעד 28BYJ-48	מחובר למחבר המנוע בדרייבר ULN2003	הזנה דרך דרייבר ULN2003	ביצוע פעולת הפתיחה והנעילה של מנגנון המנעול
8	ממסר תאורה	IN / Signal → AR7	VCC → 5V, GND → GND	הפעלת תאורה חיצונית בתנאי חשכה
9	ירוק LED	Signal → AR5	דרך נגד מתאים אל GND	חיווי מצב פתיחה תקין של המנעול
10	אדום LED	Signal → AR6	דרך נגד מתאים אל GND	חיווי מצב נעילה או מצב מערכת סגור
11	זמזם	Signal → AR4	VCC → 5V, GND → GND	חיווי קולי בעת לחיצה, פתיחה או אירוע חריג
12	Telegram Bot	Ethernet → Router / Internet	חיבור רשת קווי	שליחת התראות ותמונות למשתמש בזמן אמת
13	קווי הזנה ואדמה	מחברי ההזנה בכרטיס PYNQ-Z2	3.3V / 5V / GND	אספקת מתח ואדמה לרכיבי המערכת

7. מימוש החומרה

בפרק זה תואר תכנון החומרה המתוכנתת במערכת המנעול החכם. מערכת זו התבססה על כרטיס PYNQ-Z2 הכולל שילוב בין מעבד ARM בצד ה־ PS לבין רכיב FPGA בצד ה־ PL בצד התוכנה בוצעו ניהול המערכת, עיבוד התמונה, זיהוי הפנים וקבלת ההחלטות. בצד החומרה המתוכנתת מומשה לוגיקת הבקרה שנועדה להפעיל את מנגנון הנעילה, ובפרט את מנוע הצעד ששימש לביצוע פעולת הפתיחה והנעילה.

מטרת תכנון ה־ FPGA הייתה לאפשר הפעלה מדויקת ומסודרת של מנוע הצעד באמצעות אותות דיגיטליים. מנוע הצעד דרש הפעלה מחזורית של ארבע פאזות, ולכן נדרש תכנון לוגי אשר יצר סדרת אותות מתאימה. לוגיקה זו נכתבה בשפת VHDL ולאחר מכן שולבה במערכת Vivado כחלק ממבנה החומרה של הפרויקט.

במערכת נעשה שימוש בממשקי AXI GPIO לצורך תקשורת בין התוכנה לבין החומרה. ממשקים אלו אפשרו לקוד ה־ Python שרץ על גבי ה־ PS לשלוח פקודות בקרה אל הלוגיקה שב־ PL ולקבל מידע מרכיבי הקלט. בצורה זו נוצרה מערכת משולבת, שבה החלטות התוכנה תורגמו לפעולות חומרה בפועל.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity keypad_controller is
    generic(
        CLK_FREQ_HZ    : integer := 100000000;
        SCAN_FREQ_HZ   : integer := 1000;
        DEBOUNCE_FRAMES : integer := 20
);
    port(
        clk        : in  std_logic;
        reset_n    : in  std_logic;
        keypad_rows : in  std_logic_vector(3 downto 0);
        keypad_cols : out std_logic_vector(3 downto 0);
        key_status  : out std_logic_vector(4 downto 0)
);
end keypad_controller;
architecture Behavioral of keypad_controller is
    constant TICK_MAX : integer := CLK_FREQ_HZ / SCAN_FREQ_HZ;
    signal tick_counter   : integer range 0 to TICK_MAX := 0;
    signal scan_tick      : std_logic := '0';
    signal col_index      : integer range 0 to 3 := 0;
    signal frame_pressed  : std_logic := '0';
    signal frame_code     : std_logic_vector(3 downto 0) := (others => '0');
    signal raw_pressed    : std_logic := '0';
    signal raw_code       : std_logic_vector(3 downto 0) := (others => '0');
    signal stable_pressed : std_logic := '0';
    signal stable_code    : std_logic_vector(3 downto 0) := (others => '0');
    signal debounce_count : integer range 0 to DEBOUNCE_FRAMES := 0;
begin
    key_status <= stable_pressed & stable_code;
    process(clk)
    begin
        if rising_edge(clk) then
```

CLK_FREQ_HZ תדר שעון הכניסה, כאן 100MHz
SCAN_FREQ_HZ קצב הסריקה של לוח המקשים, כאן 1000 **Hz**
DEBOUNCE_FRAMES מספר מחזורי סריקה יציבים הנדרשים לפני אישור לחיצה.

```
    if reset_n = '0' then
      tick_counter <= 0;
      scan_tick <= '0';
    else
      if tick_counter = TICK_MAX - 1 then
        tick_counter <= 0;
        scan_tick <= '1';
      else
        tick_counter <= tick_counter + 1;
        scan_tick <= '0';
      end if;
    end if;
  end if;
end process;
process(col_index)
begin
  case col_index is
    when 0<=
      keypad_cols <= "1110" ;
    when 1<=
      keypad_cols <= "1101" ;
    when 2<=
      keypad_cols <= "1011" ;
    when others<=
      keypad_cols <= "0111" ;
  end case;
end process;
process(clk)
  variable detected_pressed : std_logic;
  variable detected_code    : std_logic_vector(3 downto 0);
  variable next_pressed     : std_logic;
  variable next_code        : std_logic_vector(3 downto 0);
begin
  if rising_edge(clk) then
    if reset_n = '0' then
      col_index <= 0;
```

תהליך זה אחראי על יצירת קצב הסריקה של לוח המקשים. מאחר ששעון המערכת מהיר מאוד, נעשה שימוש במונה פנימי שמחלק את קצב העבודה. כאשר המונה מגיע לערך שנקבע מראש, נוצר האות scan_tick למשך מחזור שעון אחד. אות זה משמש כהוראה לבקר לבצע את שלב הסריקה הבא של לוח המקשים.

תהליך זה אחראי על סריקת ארבע עמודות לוח המקשים. בכל רגע מופעלת עמודה אחת בלבד במצב לוגי נמוך, בעוד שאר העמודות נשארות במצב גבוה. הערך של col_index קובע איזו עמודה פעילה בכל שלב.

```
    frame_pressed <= '0';
    frame_code <= (others => '0');
    raw_pressed <= '0';
    raw_code <= (others => '0');
    stable_pressed <= '0';
    stable_code <= (others => '0');
    debounce_count <= 0;
elsif scan_tick = '1' then
    detected_pressed := '0';
    detected_code := (others => '0');
    if keypad_rows(0) = '0' then
        detected_pressed := '1' ;
        case col_index is
            when 0 => detected_code := x"1" ;
            when 1 => detected_code := x"2" ;
            when 2 => detected_code := x"3" ;
            when others => detected_code := x"A" ;
        end case;
    elsif keypad_rows(1) = '0' then
        detected_pressed := '1' ;
        case col_index is
            when 0 => detected_code := x"4" ;
            when 1 => detected_code := x"5" ;
            when 2 => detected_code := x"6" ;
            when others => detected_code := x"B" ;
        end case;
    elsif keypad_rows(2) = '0' then
        detected_pressed := '1' ;
        case col_index is
            when 0 => detected_code := x"7" ;
            when 1 => detected_code := x"8" ;
            when 2 => detected_code := x"9" ;
            when others => detected_code := x"C" ;
        end case;
    elsif keypad_rows(3) = '0' then
        detected_pressed := '1' ;
```

תהליך זה אחראי על עיקר פעולת בקר לוח המקשים. בכל scan_tick נקרא מצב ארבע השורות בהתאם לעמודה הפעילה באותו רגע. אם אחת השורות נמצאת במצב לוגי נמוך, הבקר מזהה את צירוף השורה והעמודה וממיר אותו לקוד של המקש המתאים.

לאחר סריקת כל ארבע העמודות נשמרת תוצאת הלחיצה ומופעל מנגנון ה־Debounce, שבודק שהמקש נשאר יציב במשך מספר מחזורי סריקה רצופים. רק לאחר שהלחיצה אושרה כיציבה, הקוד מועבר ל־ stable_code והלחיצה מסומנת כתקפה באמצעות stable_pressed

```
        case col_index is
            when 0 => detected_code := x"E" ;
            when 1 => detected_code := x"0" ;
            when 2 => detected_code := x"F" ;
            when others => detected_code := x"D" ;
        end case;
    end if;
    next_pressed := frame_pressed;
    next_code := frame_code;
    if detected_pressed = '1' then
        next_pressed := '1';
        next_code := detected_code;
    end if;
    if col_index = 3 then
        if (next_pressed = raw_pressed) and (next_code = raw_code) then
            if debounce_count < DEBOUNCE_FRAMES then
                debounce_count <= debounce_count + 1;
            end if;
        else
            debounce_count <= 0;
            raw_pressed <= next_pressed;
            raw_code <= next_code;
        end if;
        if debounce_count >= DEBOUNCE_FRAMES - 1 then
            stable_pressed <= next_pressed;
            stable_code <= next_code;
        end if;
        frame_pressed <= '0' ;
        frame_code <= (others => '0');
        col_index <= 0;
    else
        frame_pressed <= next_pressed;
        frame_code <= next_code;
        col_index <= col_index + 1;
    end if;
end if;
```

```
    end if;
  end process;
end Behavioral;
```

הסבר על הקוד:
קוד ה־ VHDL של בקר לוח המקשים נועד לבצע סריקה מחזורית של לוח מקשים מטריציוני בגודל 4×4, לזהות את המקש שנלחץ, לבצע סינון של רעידות המגע ולספק למערכת קוד יציב המייצג את המקש שנקלט.

בקוד הוגדרו שלושה פרמטרים מרכזיים. הפרמטר CLK_FREQ_HZ הגדיר את תדר שעון המערכת, הפרמטר SCAN_FREQ_HZ הגדיר את קצב סריקת לוח המקשים, והפרמטר DEBOUNCE_FRAMES הגדיר את מספר מחזורי הסריקה הרצופים שבהם נדרש לקבל את אותה תוצאה לפני שהלחיצה אושרה כלחיצה תקינה.

כניסת clk שימשה לסנכרון פעולת הבקר, וכניסת reset_n שימשה לאיפוס המערכת. ארבע הכניסות keypad_rows חוברו לשורות לוח המקשים, וארבע היציאות keypad_cols חוברו לעמודות. היציאה key_status הוגדרה ברוחב של 5 ביטים, כאשר ביט 4 שימש לציון קיום לחיצה תקינה, וארבעת הביטים הנמוכים שימשו לייצוג קוד המקש שזוהה.

לצורך יצירת קצב הסריקה הוגדר מונה פנימי בשם tick_counter המונה ספר מחזורי שעון עד לערך שנקבע באמצעות TICK_MAX המחושב לפי היחס בין תדר שעון המערכת לבין תדר הסריקה. כאשר המונה הגיע לערכו המרבי הוא אופס, והאות scan_tick הועלה למשך מחזור שעון אחד. אות זה שימש להפעלת שלב הסריקה הבא של לוח המקשים.

סריקת העמודות בוצעה באמצעות המשתנה col_index אשר קיבל ערכים בין 0 ל־3. בהתאם לערכו הופעלה בכל פעם עמודה אחת בלבד במצב לוגי נמוך, בעוד שאר העמודות נשארו במצב לוגי גבוה. רצף היציאות שהתקבל היה 1110, 1101 1011 ו־ 0111 לאחר סריקת העמודה הרביעית חזר הבקר לעמודה הראשונה והחל מחזור סריקה חדש.

השורות הוגדרו עם נגדי Pull-Up ולכן במצב שבו לא נלחץ אף מקש התקבל בהן ערך לוגי גבוה. כאשר נלחץ מקש, נוצר חיבור בין השורה לבין העמודה המתאימה. בזמן שבו אותה עמודה הופעלה במצב נמוך, גם השורה המתאימה ירדה למצב לוגי נמוך. באמצעות שילוב בין מספר השורה לבין ערך col_index נקבע המקש שנלחץ.

לכל צירוף של שורה ועמודה הוגדר קוד מתאים. לדוגמה, כאשר זוהתה לחיצה בשורה השלישית ובעמודה השלישית, הוגדר detected_code לערך x״9״ באופן דומה הוגדרו יתר הספרות, האותיות והמפתחות * ו־ #

מכיוון שכל מחזור סריקה כלל מעבר על ארבע העמודות, נשמרה תוצאת הזיהוי הזמנית באמצעות frame_pressed ו־ frame_code אם במהלך אחת מארבע העמודות זוהה מקש, נשמר המידע עד להשלמת מחזור הסריקה המלא. לאחר סיום סריקת העמודה הרביעית הועברה התוצאה למנגנון סינון הלחיצה.

מנגנון ה־ Debounce נועד למנוע זיהוי שגוי של מספר לחיצות כתוצאה מרעידות מכניות של מגעי המקש. התוצאה החדשה הושוותה לערכים raw_pressed ו־ raw_code מהסריקה הקודמת. כאשר התקבלה אותה תוצאה במספר מחזורי סריקה רצופים, הוגדל debounce_count. רק לאחר שמספר הסריקות היציבות הגיע לערך שהוגדר ב־ DEBOUNCE_FRAMES הועברה התוצאה אל stable_pressed ו־ stable_code

בסיום התהליך נבנתה היציאה key_status באמצעות חיבור בין ביט תקפות הלחיצה לבין קוד המקש:

```
key_status <= stable_pressed & stable_code;
```

לכן, כאשר זוהתה לחיצה יציבה, ביט 4 של key_status קיבל את הערך 1, וארבעת הביטים הנמוכים הכילו את קוד המקש. לדוגמה, עבור לחיצה על הספרה 9 התקבל הקוד 11001 שבו הביט העליון ציין כי קיימת לחיצה תקפה והערך 1001 ייצג את הספרה 9.

באופן זה בוצעו ב־ PL סריקת לוח המקשים, זיהוי השורה והעמודה, פענוח המקש וסינון רעידות המגע. התוצאה הסופית הועברה דרך ממשק AXI GPIO אל תוכנת ה־ Python בצד ה־ PS שבה בוצעו בניית הסיסמה, בדיקת תקינותה וניהול ניסיונות הגישה.

7.2. תכנון בקר מנוע הצעד

מנוע הצעד במערכת שימש כמנגנון הפתיחה והנעילה. לצורך הפעלתו נדרש ליצור רצף אותות לארבע פאזות, כאשר כל פאזה הפעילה כניסה אחרת בדרייבר ULN2003 שינוי סדר הפאזות אפשר לקבוע את כיוון הסיבוב של המנוע.

תכנון בקר מנוע הצעד כלל שלושה חלקים עיקריים:

1. מחלק תדר ליצירת שעון איטי יותר להפעלת המנוע .
2. מכונת מצבים לסיבוב בכיוון פתיחה .
3. מכונת מצבים לסיבוב בכיוון נעילה .

מחלק התדר נדרש מכיוון ששעון המערכת של הכרטיס מהיר בהרבה מהקצב הדרוש להפעלת מנוע הצעד. לכן נוצר אות שעון איטי יותר, אשר קבע את קצב מעבר המצבים במכונת המצבים. בכל מעבר מצב הופעלה פאזה אחרת של המנוע, וכך נוצרה תנועה הדרגתית ומבוקרת.

מכונת המצבים לסיבוב בכיוון פתיחה יצרה את רצף הפאזות המתאים לסיבוב בכיוון אחד. לעומת זאת, מכונת המצבים לסיבוב בכיוון נעילה יצרה את אותו רצף פאזות בסדר הפוך. באמצעות שני רצפים אלו ניתן היה לפתוח ולנעול את המנגנון בהתאם לפקודה שהתקבלה מהתוכנה.

7.2.1. קומפוננט של מחלק התדר

קומפוננטת מחלק התדר תוכננה במטרה להמיר את שעון המערכת של כרטיס PYNQ-Z2 לאות שעון איטי יותר, אשר התאים להפעלת מנוע הצעד. שעון המערכת המקורי פעל בתדר גבוה, ולכן לא ניתן היה להשתמש בו ישירות להפעלת המנוע. שינוי מהיר מדי של אותות הפאזות היה גורם לכך שהמנוע לא היה מספיק להגיב לכל שינוי, ולכן נדרש קצב פעולה נמוך ומבוקר יותר.

בקומפוננטה זו התקבל אות השעון הראשי של המערכת כקלט, ובוצעה ספירה של מחזורי שעון באמצעות מונה פנימי. כאשר המונה הגיע לערך שנקבע מראש, מוצא השעון שינה את מצבו הלוגי והמונה אופס. פעולה זו יצרה אות שעון חדש בתדר נמוך יותר, אשר שימש בהמשך להפעלת מכונות המצבים של מנוע הצעד.

מוצא מחלק התדר שימש כבסיס הזמן של בקר המנוע. בכל מחזור של השעון המחולק, מכונת המצבים התקדמה למצב הבא, ובהתאם לכך הופעלה פאזה אחרת של מנוע הצעד. בצורה זו התקבלה תנועה הדרגתית ומבוקרת של המנוע.

להלן קוד VHDL עבור מחלק תדר מ125MHZ ל500HZ :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity clk_divider is
  port(
    clk_system_125MHZ : in  std_logic;
    reset             : in  std_logic;
    clk_500HZ         : out std_logic
```

הספריות הקוד

בקטע זה הוגדרו כניסות ויציאת הקומפוננטה של מחלק התדר.
הכניסה clk_system_125MHZ שימשה לקבלת שעון המערכת הראשי בתדר 125 MHz הכניסה reset שימשה לאיפוס הקומפוננטה, והיציאה clk_500HZ שימשה להוצאת השעון המחולק בתדר 500Hz

```
;(
end clk_divider;

architecture B of clk_divider is
   constant C_MAX_COUNT : integer := 125000;
   signal counter  : integer range 0 to C_MAX_COUNT - 1 := 0;
   signal clk_state : std_logic := '0';
begin
   process (clk_system_125MHZ)
   begin
      if rising_edge(clk_system_125MHZ) then
         if reset = '1' then
            counter  <= 0;
            clk_state <= '0';
         elsif counter = C_MAX_COUNT - 1 then
            counter  <= 0;
            clk_state <= not clk_state;
         else
            counter <= counter + 1;
         end if;
      end if;
   end process;

   clk_500HZ <= clk_state;
end B;
```

בקטע זה הוגדר ערך הספירה
C_MAX_COUNT = 125000

הערך חושב לפי:

125MHz / (2 · 500Hz) = 125000

החלוקה ב־2 נדרשה משום שמחזור שעון מלא כולל שני שינויי מצב .counter סופר עד ערך זה, ו־clk_stateשומר את מצב השעון המחולק.

בקטע זה מומש תהליך סינכרוני שפועל בכל עליית שעון של clk_system_125MHZ תחילה נבדק אות האיפוס reset כאשר האיפוס פעיל, המונה counter מתאפס ל־0, וגם מצב השעון המחולק clk_state מתאפס ל־0.

כאשר האיפוס אינו פעיל, המונה מתחיל לספור את מחזורי שעון המערכת. כל עוד המונה לא הגיע לערך הסופי C_MAX_COUNT - 1, הוא גדל באחד בכל עליית שעון. כאשר המונה מגיע לערך הסופי, הוא מתאפס, והאות clk_state משנה את מצבו הלוגי. שינוי מצב זה יוצר את פעולת חלוקת התדר.

בסיום הקוד, האות הפנימי clk_state מחובר ליציאה clk_500HZ לכן מתקבל במוצא שעון מחולק בתדר 500Hz

7.2.2. קומפונטה של סיבוב המנוע עם כיוון השעון

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity motor_clockwise is
   port(
      clk           : in  std_logic;
      reset         : in  std_logic;
      enable_cw     : in  std_logic;
      stepper_phases : out std_logic_vector(3 downto 0)
   );
end motor_clockwise;

architecture Behavioral of motor_clockwise is
   type state_type is (IDLE, PHASE_1, PHASE_2, PHASE_3, PHASE_4);
   signal current_state : state_type := IDLE;
   signal next_state    : state_type := IDLE;
begin
   process(clk)
   begin
      if rising_edge(clk) then
         if reset = '1' then
            current_state <= IDLE;
         else
            current_state <= next_state;
         end if;
      end if;
```

clk- שעון לסנכרון מעבר המצבים
reset- איפוס המערכת למצב IDLE
enable_cw- הפעלת סיבוב עם כיוון השעון
stepper_phases- יציאת 4 סיביות להפעלת פאזות המנוע דרך ULN2003

הגדרת סיגנלים פנימיים:

current_state-המצב הנוכחי של המכונה.
next_state -המצב הבא של המכונה.
state_type טיפוס המכיל את המצבים השונים

תהליך סינכרוני הפועל בעליית השעון ומעדכן את המצב הנוכחי למצב הבא. בעת reset המערכת חוזרת למצב IDLE

```
end process;

process(current_state, enable_cw)
begin
   stepper_phases <= "0000";
   next_state <= IDLE;

   case current_state is
      when IDLE =>
         stepper_phases <= "0000";
         if enable_cw = '1' then
            next_state <= PHASE_1;
         else
            next_state <= IDLE;
         end if;

      when PHASE_1 =>
         stepper_phases <= "0001";
         if enable_cw = '1' then
            next_state <= PHASE_2;
         else
            next_state <= IDLE;
         end if;

      when PHASE_2 =>
         stepper_phases <= "0010";
         if enable_cw = '1' then
            next_state <= PHASE_3;
         else
            next_state <= IDLE;
         end if;

      when PHASE_3 =>
         stepper_phases <= "0100";
         if enable_cw = '1' then
            next_state <= PHASE_4;
         else
            next_state <= IDLE;
         end if;

      when PHASE_4 =>
         stepper_phases <= "1000";
         if enable_cw = '1' then
            next_state <= PHASE_1;
         else
            next_state <= IDLE;
         end if;

      when others =>
         stepper_phases <= "0000";
         next_state <= IDLE;
   end case;
end process;
```

התהליך האסינכרוני קובע את המצב הבא ואת יציאת פאזות המנוע בהתאם למצב הנוכחי current_state ולאותenable_cw כאשר enable_cw =1 המערכת מתקדמת מהמצב הנוכחי למצב הבא ברצף PHASE_1 עד PHASE_4 ובכל מצב מופעלת פאזה אחרת של המנוע. כאשר enable_cw=0 המערכת חוזרת למצב IDLE והמוצא נקבע ל־ "0000"

```
end Behavioral;
```

7.2.3. קומפונטה של המנוע נגד כיוון השעון

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity motor_counterclockwise is
   port)
      clk           : in  std_logic;
      reset         : in  std_logic;
      enable_ccw    : in  std_logic;
      stepper_phases : out std_logic_vector(3 downto 0)
);
end motor_counterclockwise;

architecture Behavioral of motor_counterclockwise is
   type state_type is (IDLE, PHASE_1, PHASE_2, PHASE_3, PHASE_4);
   signal current_state : state_type := IDLE;
   signal next_state    : state_type := IDLE;
begin
   process(clk)
   begin
      if rising_edge(clk) then
         if reset = '1' then
            current_state <= IDLE;
         else
            current_state <= next_state;
         end if;
      end if;
   end process;

   process(current_state, enable_ccw)
   begin
      stepper_phases <= "0000" ;
      next_state <= IDLE;

      case current_state is
         when IDLE<=
            stepper_phases <= "0000" ;
            if enable_ccw = '1' then
               next_state <= PHASE_4;
            else
               next_state <= IDLE;
            end if;
```

```
        when PHASE_1<=
          stepper_phases <= "1000″ ;
          if enable_ccw = '1' then
            next_state <= PHASE_2;
          else
            next_state <= IDLE;
          end if;

        when PHASE_2<=
          stepper_phases <= "0100″ ;
          if enable_ccw = '1' then
            next_state <= PHASE_3;
          else
            next_state <= IDLE;
          end if;

        when PHASE_3<=
          stepper_phases <= "0010″ ;
          if enable_ccw = '1' then
            next_state <= PHASE_4;
          else
            next_state <= IDLE;
          end if;

        when PHASE_4<=
          stepper_phases <= "0001″ ;
          if enable_ccw = '1' then
            next_state <= PHASE_1;
          else
            next_state <= IDLE;
          end if;

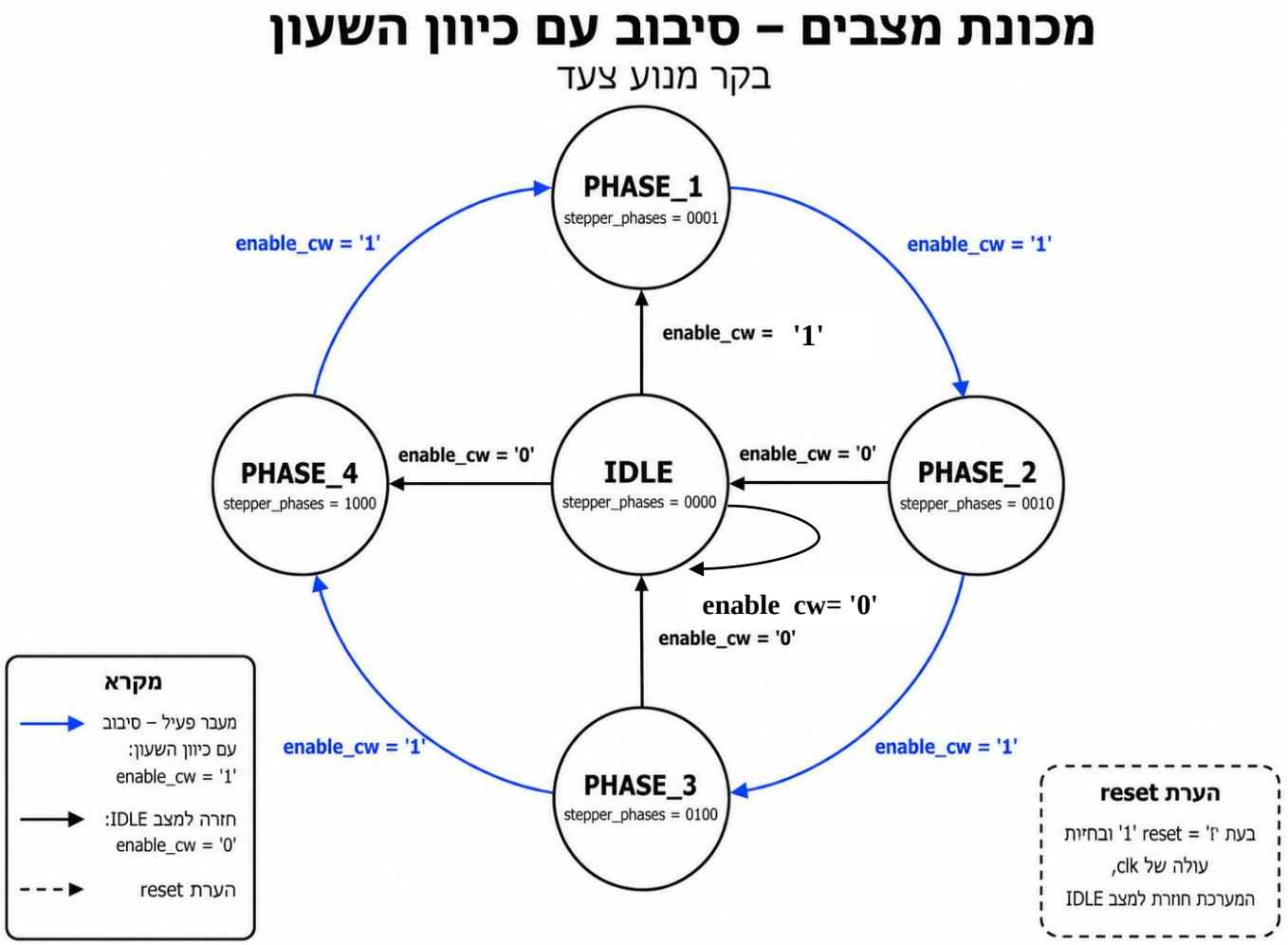
        when others<=
          stepper_phases <= "0000″ ;
          next_state <= IDLE;
      end case;
    end process;
end Behavioral;
```

קוד הVHDL של הסיבוב נגד כיוון השעון מבוסס על אותו מבנה של קוד הסיבוב עם כיוון השעון. בשני הקודים נעשה שימוש באותה מכונת מצבים הכוללת את המצבים IDLE ו־ PHASE_1 עד PHASE_4 וכן באותו תהליך סינכרוני לעדכון המצב הנוכחי.

ההבדל העיקרי נמצא בסדר הפעלת הפאזות. בסיבוב עם כיוון השעון הפאזות מופעלות ברצף אחד, ואילו בסיבוב נגד כיוון השעון אותו רצף מופעל בסדר הפוך. שינוי סדר הפעלת ארבע הפאזות משנה את כיוון השדה המגנטי במנוע, ולכן גורם לציר המנוע להסתובב בכיוון ההפוך.

כאשר enable_ccw נמצא במצב לוגי גבוה, מכונת המצבים מתקדמת בין הפאזות לפי הסדר שהוגדר לסיבוב נגד כיוון השעון. כאשר enable_ccw יורד למצב לוגי נמוך, המערכת חוזרת למצב IDLE והיציאה stepper_phases נקבעת ל־0000″ כך שהמנוע מפסיק לפעול.

להלן סכמה של מכונת המצבים עבור סיבוב עם כיוון השעון:



כאשר ׳0׳ = enable_cw המערכת לא המשיכה ברצף הסיבוב, אלא חזרה למצב IDLE בנוסף, כאשר התקבל ׳1׳ = reset בעלייה של אות השעון, המערכת חזרה גם כן למצב IDLE ללא תלות במצב הנוכחי. לכן, מכונת המצבים קבעה בצורה ברורה מתי המנוע מסתובב, באיזה סדר מופעלות הפאזות, ומתי המערכת חוזרת למצב מנוחה.

עבור סיבוב נגד כיוון השעון, עקרון הפעולה נשאר זהה והסרטוט נשאר אותו סרטוט, פרט לכך שכיוון החיצים הכחולים מתהפך. כלומר, אותם מצבים ואותם ערכי מוצא נשמרים, אך סדר המעבר בין הפאזות מתבצע בכיוון ההפוך, וכך מתקבל סיבוב נגד כיוון השעון.

7.2.4. קוד הראשי של מנוע הצעד

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity step_motor_controller is
   port (
      clk_system_125MHZ : in  std_logic;
      reset             : in  std_logic;
      enable_cw         : in  std_logic;
      enable_ccw        : in  std_logic;
      stepper_phases    : out std_logic_vector(3 downto 0)
);
end step_motor_controller;

architecture Behavioral of step_motor_controller is
   component clk_divider is
      port (
         clk_system_125MHZ : in  std_logic;
         reset             : in  std_logic;
         clk_500HZ         : out std_logic
)       ;

   end component;

   component motor_clockwise is
      port (
         clk           : in  std_logic;
         reset         : in  std_logic;
         enable_cw     : in  std_logic;
         stepper_phases : out std_logic_vector(3 downto 0)
)       ;

   end component;

   component motor_counterclockwise is
      port (
         clk           : in  std_logic;
         reset         : in  std_logic;
         enable_ccw    : in  std_logic;
         stepper_phases : out std_logic_vector(3 downto 0)
)       ;

   end component;

   signal clk_500hz : std_logic;
   signal phases_cw : std_logic_vector(3 downto 0);
   signal phases_ccw : std_logic_vector(3 downto 0);

begin
   u1: clk_divider
```

הספריות הקוד

בקטע זה הוגדרו כול הכניסות של מערכת המנוע :
clk_system_125MHZ - כניסת שעון
reset - כניסת איפוס
enable_cw - כניסת איפשור עבור סיבוב עם כיוון השעון
enable_ccw - כניסת איפשור עבור סיבוב נגד כיוון השעון
stepper_phases- יציאת 4 סיביות להפעלת פאזות המנוע דרך ULN2003

פורטי הקומפונטה של מחלק התדר

פורטי הקומפונטה של סיבוב עם כיוון השעון

פורטי הקומפונטה של סיבוב נגד כיוון השעון

סיגנלים עבור כניסות המנוע

```
    port map (
      clk_system_125MHZ => clk_system_125MHZ,
      reset          => reset,
      clk_500HZ      => clk_500hz
);    ;
```

שיוך עבור מחלק התדר

```
  u2: motor_clockwise
    port map (
      clk          => clk_500hz,
      reset        => reset,
      enable_cw    => enable_cw,
      stepper_phases => phases_cw
);    ;
```

שיוך עבור סיבוב עם כיוון השעון

```
  u3: motor_counterclockwise
    port map (
      clk          => clk_500hz,
      reset        => reset,
      enable_ccw   => enable_ccw,
      stepper_phases => phases_ccw
);    ;
```

שיוך עבור סיבוב נגד כיוון השעון

```
  stepper_phases <= phases_cw  when enable_cw = '1' else
            phases_ccw when enable_ccw = '1' else
"0000"              ;

end Behavioral;
```

בחירת המוצא

המוצא stepper_phases נקבע בהתאם לאות ההפעלה. כאשר enable_cw = '1' נבחר המוצא phases_cw כאשר enable_ccw = '1' נבחר phases_ccw וכאשר שני אותות ההפעלה אינם פעילים המוצא נקבע ל-"0000"

7.3. השלבים ליצירת המערכת ב VIVADO

לאחר השלמת שלושת חלקי הבקר, הם שולבו בתוכנית ראשית בשם step_motor_controller התוכנית הראשית חיברה בין מחלק התדר לבין שתי מכונות המצבים, וקבעה איזה רצף פאזות יועבר למוצא בהתאם לפקודת כיוון הסיבוב שהתקבלה.

לאחר השלמת כתיבת הקוד ובדיקת מבנה הבקר, בוצעה אריזת התכנון כרכיב IP עצמאי. לצורך כך נבחרה בסביבת Vivado האפשרות:

Tools → Create and Package New IP

באמצעות תהליך זה נארז בקר מנוע הצעד כיחידת חומרה אחת, כך שניתן היה להוסיף אותו בהמשך כבלוק עצמאי ל-Block Design של מערכת המנעול החכם. בשלב האריזה נבדקו רכיבי ה-IP, אותות הכניסה והיציאה והקבצים שנכללו בתכנון.

אחרי השלמת הבדיקה הסתיים תהליך האריזה, ובקר מנוע הצעד היה זמין לשילוב במערכת החומרה הכללית ב־Vivado. לאחר השלמת תכנון בקר מנוע הצעד ואריזתו כרכיב IP בוצע שילובו במערכת החומרה המלאה של המנעול החכם. תהליך בניית המערכת בסביבת Vivado בוצע בשלבים הבאים:

1. יצירת פרויקט חדש ב־Vivado

נפתח פרויקט חדש בסביבת Vivado ונבחר כרטיס הפיתוח PYNQ-Z2 בחירת הכרטיס אפשרה להתאים את סביבת הפיתוח לרכיב ה־ Zynq-7000 ולמשאבים הקיימים בכרטיס.

2. יצירת Block Design

לאחר יצירת הפרויקט נוצר Block Design חדש. סביבת ה־ Block Design שימשה לבניית מערכת החומרה באמצעות חיבור בין רכיבי IP סטנדרטיים לבין רכיב ה־ IP של בקר מנוע הצעד שתוכנן בשפת VHDL

3. הוספת Zynq Processing System

למערכת נוסף רכיב Zynq Processing System אשר ייצג את צד ה־ PS של כרטיס PYNQ-Z2 צד זה שימש בהמשך להרצת תוכנת ה־ Python ולשליטה ברכיבי החומרה שתוכננו בצד ה־ PL

4. ביצוע Run Block Automation

לאחר הוספת רכיב ה־ Zynq Processing System בוצעה הפעולה Run Block Automation פעולה זו הגדירה את החיבורים הבסיסיים וההגדרות הדרושות לשילוב רכיב העיבוד עם כרטיס PYNQ-Z2

5. הוספת ממשקי AXI GPIO

ל־ Block Design נוספו רכיבי AXI GPIO רכיבים אלו שימשו כממשק בין התוכנה שפעלה בצד ה־ PS לבין אותות הקלט והפלט שהוגדרו בצד ה־ PL

6. הגדרת ממשק GPIO עבור רכיבי המערכת ההיקפיים

אחד מממשקי ה־ AXI GPIO שימש להעברת המידע בין תוכנת הבקרה לבין רכיבי הקלט והפלט של מערכת המנעול החכם. באמצעות ממשק זה התאפשרה קריאת אותות מהחיישנים והפעלת רכיבי הפלט בהתאם להחלטות שהתקבלו בתוכנה.

7. הגדרת ממשק GPIO עבור בקר מנוע הצעד

ממשק AXI GPIO נוסף שימש להעברת פקודות הבקרה אל בקר מנוע הצעד. באמצעותו נשלחו האותות שקבעו את כיוון סיבוב המנוע, ובהתאם לכך בוצעה פעולת הפתיחה או הנעילה.

8. הוספת ה־ IP של בקר מנוע הצעד

למערכת נוסף רכיב ה־ IP מסוג step_motor_controller אשר תוכנן ונארז קודם לכן. רכיב זה ריכז את מחלק התדר ואת מכונות המצבים של שני כיווני הסיבוב תחת יחידת חומרה אחת.

9. חיבור אותות הבקרה לבקר מנוע הצעד

אותות הבקרה שהתקבלו מממשק ה־ AXI GPIO חוברו לכניסות enable_cw ו־ enable_ccw של בקר המנוע. באמצעות אותות אלו נקבע איזה רצף פאזות יופעל ובהתאם לכך נקבע כיוון הסיבוב.

10. חיבור שעון המערכת

אות שעון המערכת בתדר 125MHz חובר לכניסה clk_system_125MHZ של בקר מנוע הצעד. בתוך הבקר בוצעה חלוקת התדר לצורך יצירת קצב מתאים למעבר בין פאזות המנוע.

11. הגדרת יציאות מנוע הצעד

ארבע יציאות stepper_phases של בקר המנוע הוגדרו כיציאות חיצוניות של המערכת. יציאות אלו נועדו להתחבר לכניסות הדרייבר ULN2003 ולהפעיל את סלילי מנוע הצעד בהתאם לרצף שנוצר בבקר.

12. הגדרת אותות הקלט והפלט החיצוניים

אותות החיישנים ורכיבי הפלט של המערכת הוגדרו כאותות חיצוניים של ה־ .Block Design פעולה זו אפשרה בשלב מאוחר יותר לשייך כל אות לפין הפיזי המתאים בכרטיס PYNQ-Z2

13. ביצוע Run Connection Automation

לאחר הוספת רכיבי ה־ AXI בוצעה הפעולה Run Connection Automation באמצעות פעולה זו נוצרו החיבורים

הדרושים בין ה־ Zynq Processing System לבין ממשקי ה־ AXI GPIO וכך התאפשרה תקשורת בין צד ה־ PS לבין צד ה־ PL

14. בדיקת המערכת באמצעות Validate Design

לאחר השלמת החיבורים בוצעה הפעולה Validate Design מטרת הבדיקה הייתה לוודא שמבנה המערכת תקין, שהממשקים הדרושים חוברו ושלא קיימות שגיאות מבניות המונעות את המשך תהליך המימוש.

15. יצירת HDL Wrapper

לאחר שהמערכת עברה את בדיקת התקינות בוצעה הפעולה Create HDL Wrapper פעולה זו יצרה מעטפת HDL עבור ה־ Block Design והגדירה אותו כיחידת חומרה שניתן להעביר לשלבי הסינתזה והמימוש.

16. יצירת קובץ Constraints

לפרויקט נוסף קובץ Constraints מסוג .XDC בקובץ זה בוצע שיוך בין האותות החיצוניים שהוגדרו בתכנון לבין הפינים הפיזיים של כרטיס PYNQ-Z2 באמצעות שיוך זה נקבעו החיבורים בין ה־ FPGA לבין החיישנים, רכיבי הפלט ודרייבר מנוע הצעד.

17. ביצוע Synthesis

לאחר השלמת התכנון בוצע תהליך Synthesis בשלב זה תיאור החומרה ומבנה ה־ Block Design הומרו למימוש לוגי המתאים למשאבים הקיימים ב־ FPGA

18. ביצוע Implementation

לאחר השלמת הסינתזה בוצע תהליך Implementation בשלב זה בוצע מיקום של המשאבים הלוגיים בתוך ה־ FPGA וניתוב החיבורים ביניהם בהתאם לתכנון.

19. יצירת Bitstream

לאחר השלמת שלב ה־ Implementation בוצעה הפעולה Generate Bitstream באמצעות פעולה זו נוצר קובץ ה־ Bitstream אשר הכיל את תצורת החומרה הסופית שנועדה להיטען אל אזור ה־ PL של כרטיס PYNQ-Z2

20. העברת קובצי החומרה אל כרטיס PYNQ-Z2

בסיום תהליך הבנייה התקבלו קובצי ה־ bit וה־ hwh קובץ ה־ bit הכיל את תצורת החומרה שנועדה להיטען אל ה־ FPGA ואילו קובץ ה־ hwh הכיל מידע על מבנה החומרה ורכיבי ה־ IP

21. טעינת ה־ Overlay מתוך Python

לאחר העברת הקבצים אל כרטיס PYNQ-Z2 ניתן היה לטעון את ה־ Overlay מתוך תוכנת ה־ Python לאחר הטעינה התאפשרה גישה לממשקי ה־ AXI GPIO קריאת מצב החיישנים, הפעלת רכיבי הפלט ושליחת פקודות אל בקר מנוע הצעד.

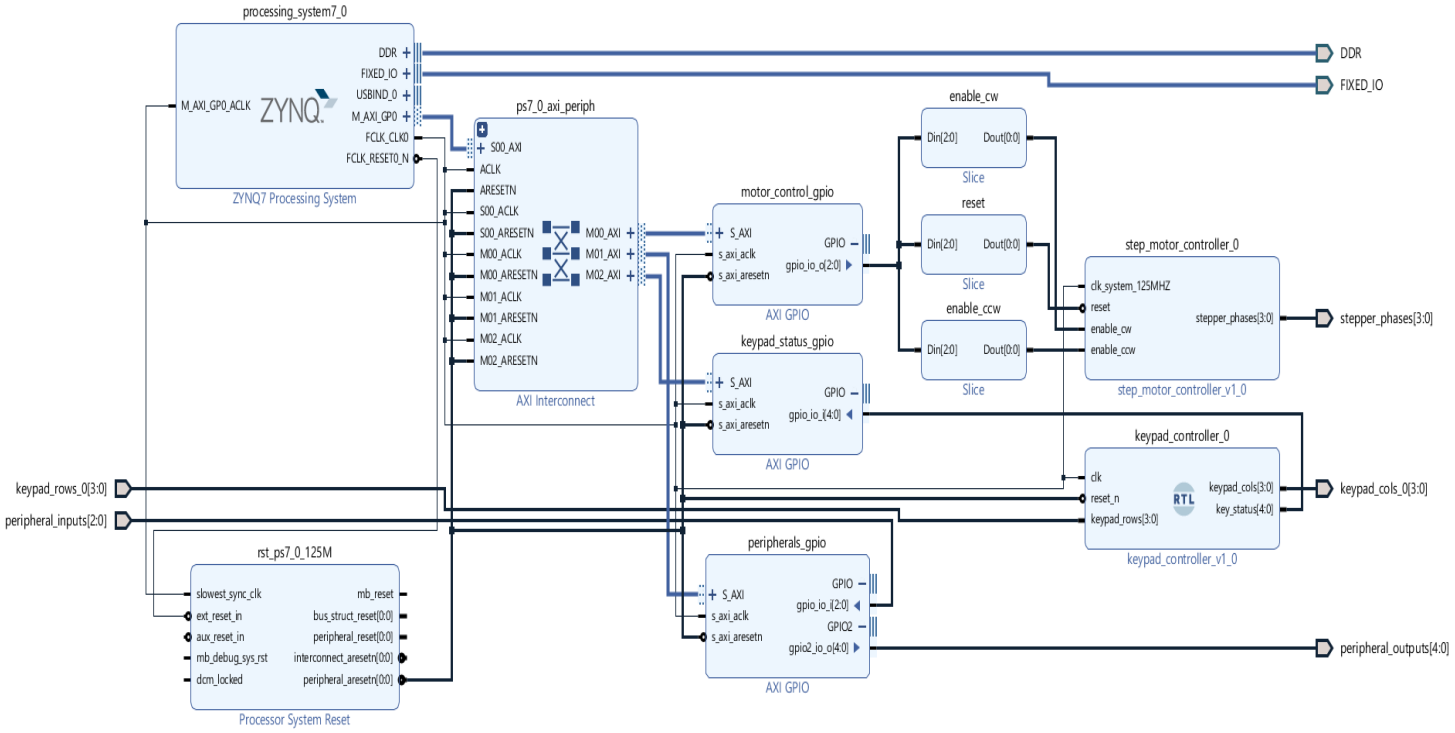
בסיום תהליך תכנון החומרה נבנתה ב־ Vivado מערכת מלאה אשר שילבה בין צד ה־ PS של כרטיס PYNQ-Z2 לבין רכיבי החומרה שמומשו בצד ה־ PL במערכת שולבו רכיב ה־ Zynq Processing System ממשקי AXI GPIO בקר מנוע הצעד שתוכנן ונארז כ־ IP וכן ממשקי הקלט והפלט ששימשו לחיבור הרכיבים החיצוניים.

באמצעות ממשקי ה־ AXI GPIO התאפשרה העברת מידע ופקודות בין תוכנת הבקרה לבין החומרה. אותות הקלט מהחיישנים נקראו על ידי המעבד, ואילו אותות הפלט שימשו להפעלת נורות החיווי, הזמזם, הממסר ובקר מנוע הצעד. בנוסף, לוח המקשים ומנוע הצעד חוברו דרך הממשקים הפיזיים המתאימים בכרטיס.

לאחר השלמת החיבורים ב־ Block Design בוצעה בדיקת תקינות למערכת נוצר HDL Wrapper והוגדר קובץ XDC אשר קישר בין האותות הלוגיים לבין הפינים הפיזיים בכרטיס. לאחר מכן בוצעו שלבי ה־ Synthesis וה־ Implementation ולבסוף נוצר קובץ ה־ Bitstream

בסיום התהליך התקבלו קובצי ה־ bit וה־ hwh אשר שימשו בהמשך לטעינת ה־ Overlay בסביבת PYNQ לאחר טעינת ה־ Overlay ניתן היה לשלוט ברכיבי החומרה מתוך תוכנת ה־ Python ולשלב את כלל תתי־המערכות במסגרת מערכת המנעול החכם.

באיור הבא מוצג ה־ Block Design הסופי של מערכת החומרה, הכולל את כלל רכיבי ה־ IP והחיבורים ביניהם.



איור 22 מבנה מערכת החומרה בVivado

באיור 22 הוצג מבנה מערכת החומרה שנבנתה בסביבת Vivado ניתן לראות את שילוב ה־ Zynq Processing System ממשקי AXI GPIO ולוגיקת ה־ VHDL הייעודית להפעלת מנוע הצעד והלוח מקשים. מבנה זה אפשר לקוד התוכנה לגשת אל רכיבי הקלט והפלט ולשלוט בפעולת המנוע באמצעות ממשקי AXI .

להלן קובץ ה xdc של המערכת:

```
##Stepper Motor Outputs - PMODA

set_property -dict { PACKAGE_PIN Y18 IOSTANDARD LVCMOS33 } [get_ports { stepper_phases[0]}]

set_property -dict { PACKAGE_PIN Y19 IOSTANDARD LVCMOS33 } [get_ports { stepper_phases[1]}]

set_property -dict { PACKAGE_PIN Y16 IOSTANDARD LVCMOS33 } [get_ports { stepper_phases[2]}]

set_property -dict { PACKAGE_PIN Y17 IOSTANDARD LVCMOS33 } [get_ports { stepper_phases[3]}]


##Peripheral Inputs - Arduino Header

##AR0 -> PIR motion sensor
```

```
set_property -dict { PACKAGE_PIN T14 IOSTANDARD LVCMOS33 } [get_ports { peripheral_inputs[0]}]

##AR1 -> LM393 digital light sensor

set_property -dict { PACKAGE_PIN U12 IOSTANDARD LVCMOS33 } [get_ports { peripheral_inputs[1]}]

##AR2 -> MC-38 magnetic door sensor

set_property -dict { PACKAGE_PIN U13 IOSTANDARD LVCMOS33 } [get_ports { peripheral_inputs[2]}]



##Peripheral Outputs - Arduino Header

##AR3 -> PIR motion indicator LED

set_property -dict { PACKAGE_PIN V13 IOSTANDARD LVCMOS33 } [get_ports { peripheral_outputs[0]}]

##AR4 -> Buzzer

set_property -dict { PACKAGE_PIN V15 IOSTANDARD LVCMOS33 } [get_ports { peripheral_outputs[1]}]

##AR5 -> Door unlocked LED

set_property -dict { PACKAGE_PIN T15 IOSTANDARD LVCMOS33 } [get_ports { peripheral_outputs[2]}]

##AR6 -> Door locked LED

set_property -dict { PACKAGE_PIN R16 IOSTANDARD LVCMOS33 } [get_ports { peripheral_outputs[3]}]

##AR7 -> Lighting relay

set_property -dict { PACKAGE_PIN U17 IOSTANDARD LVCMOS33 } [get_ports { peripheral_outputs[4]}]



##x4 Keypad Controller - PMODB

##Columns - outputs from VHDL controller

set_property -dict { PACKAGE_PIN W14 IOSTANDARD LVCMOS33 } [get_ports { keypad_cols_0[0]}]

set_property -dict { PACKAGE_PIN Y14 IOSTANDARD LVCMOS33 } [get_ports { keypad_cols_0[1]}]

set_property -dict { PACKAGE_PIN T11 IOSTANDARD LVCMOS33 } [get_ports { keypad_cols_0[2]}]

set_property -dict { PACKAGE_PIN T10 IOSTANDARD LVCMOS33 } [get_ports { keypad_cols_0[3]}]
```

```
##Rows - inputs to VHDL controller, with internal pull-up

set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 PULLUP true } [get_ports { keypad_rows_0[0]}]

set_property -dict { PACKAGE_PIN W16 IOSTANDARD LVCMOS33 PULLUP true } [get_ports { keypad_rows_0[1]}]

set_property -dict { PACKAGE_PIN V12 IOSTANDARD LVCMOS33 PULLUP true } [get_ports { keypad_rows_0[2]}]

set_property -dict { PACKAGE_PIN W13 IOSTANDARD LVCMOS33 PULLUP true } [get_ports { keypad_rows_0[3]}]
```

8. מימוש התוכנה

מימוש התוכנה של מערכת המנעול החכם בוצע בשפת Python בסביבת Jupyter Notebook הפועלת על גבי כרטיס PYNQ-Z2 התוכנה שימשה כיחידת הבקרה המרכזית של המערכת, וקישרה בין רכיבי החומרה, מנגנון זיהוי הפנים, לוח המקשים, מערכת ההתראות ומנגנון תיעוד האירועים.

באמצעות התוכנה נקראו מצבי החיישנים, התקבלו נתונים מלוח המקשים, הופעלו רכיבי החיווי, הממסר והזמזם, ונשלחו פקודות פתיחה ונעילה אל בקר מנוע הצעד שמומש בצד ה־ FPGA בנוסף, בוצעו באמצעות התוכנה צילום תמונות, גילוי פנים, זיהוי משתמש מורשה, ניהול ניסיונות גישה שגויים ושליחת התראות באמצעות Telegram

מבנה התוכנה תוכנן כך שכל תת־מערכת טופלה באמצעות פונקציות ויחידות פעולה ייעודיות, בעוד שהלוגיקה הראשית ניהלה את המעבר בין מצבי המערכת בהתאם לאירועים שהתקבלו מהחיישנים ומהמשתמש. בדרך זו התאפשר שילוב מסודר בין פעולות החומרה לבין תהליכי העיבוד וקבלת ההחלטות.

בשלב הראשון של הפעלת התוכנה נטען ה־ Overlay שנוצר בסביבת Vivado באמצעות טעינת ה־ Overlay הוגדרה החומרה בצד ה־ PL ולאחר מכן התאפשרה גישה מתוך Python לרכיבי ה־ IP ולממשקי ה־ GPIO שנכללו בתכנון.

לאחר טעינת ה־ Overlay הוגדרו ממשקי הקלט והפלט של המערכת. כניסות ה־ GPIO שימשו לקריאת חיישן התנועה PIR חיישן האור LM393 וחיישן הדלת MC-38 יציאות ה־ GPIO שימשו להפעלת נורות החיווי, הזמזם והממסר. בנוסף, הוגדר ממשק נפרד להעברת פקודות ההפעלה אל בקר מנוע הצעד, וכן ממשק עבור בקר לוח המקשים.

לאחר השלמת הגדרות החומרה בוצע אתחול של רכיבי התוכנה הנוספים, ובהם המצלמה, מודל Haar Cascade לגילוי פנים, מודל LBPH לזיהוי פנים, מערכת הלוגים ומנגנון ההתראות. לאחר שכל הרכיבים אותחלו בהצלחה, המערכת הייתה מוכנה לעבור למצב הפעולה הראשי ולקבל אירועי כניסה.

8.1. ייבוא ספריות והגדרת סביבת העבודה

```
 1. from pynq import Overlay
 2. from pynq.lib import AxiGPIO
 3.
 4. import cv2
 5. import os
 6. import time
 7. import csv
 8. import queue
 9. import threading
10. import requests
11.
12. from datetime import datetime
13. from enum import Enum
14. from pathlib import Path
```

תפקידי הספריות שהוגדרו היו:

- Overlay שימשה לטעינת תצורת החומרה שנוצרה ב־ Vivado אל אזור ה־ PL של כרטיס PYNQ-Z2
- AxiGPIO שימשה לגישה מתוך Python לרכיבי ה־ AXI GPIO שהוגדרו במערכת החומרה.
- cv2 ספריית OpenCV ששימשה להפעלת המצלמה, קריאת ושמירת תמונות, המרה לגווני אפור, גילוי פנים וזיהוי פנים.
- os שימשה לבדיקת קיומם של קבצים ונתיבים הנדרשים להפעלת המערכת.
- time שימשה למדידת זמנים, יצירת השהיות ולניהול זמני הפעולה של מנגנוני המערכת.
- csv שימשה ליצירת קובץ הלוג ולשמירת אירועי הגישה והמערכת בפורמט CSV
- queue שימשה ליצירת תור עבור הודעות והתראות Telegram כך ששליחת ההתראות לא תחסום את פעולת הלולאה הראשית.
- threading שימשה להפעלת תהליך נפרד שטיפל בשליחת הודעות ותמונות ל־ Telegram במקביל לפעולת המערכת.
- requests שימשה לשליחת בקשות HTTP אל Telegram Bot API לצורך שליחת הודעות ותמונות.
- datetime שימשה להוספת תאריך ושעה לאירועים שנשמרו בלוג ולהודעות שנשלחו למשתמש.
- Enum שימשה להגדרת מצבי הפעולה של מכונת המצבים הראשית של מערכת המנעול החכם.
- Path שימשה לניהול נתיבי הקבצים והתיקיות של הפרויקט בצורה מסודרת ונוחה.

8.2. הגדרת פרמטריים מרכזיים

```
 1. BITSTREAM_FILE = "design_1.bit"
 2. PASSWORD = "952"
 3. AUTHORIZED_USER_ID = 1
 4. CONFIDENCE_THRESHOLD = 60.0
 5. TRAINER_PATH = "/home/xilinx/jupyter_notebooks/FINAL_PROJECT/TESTS/camera_test/Face
Recognition/trainer.yml"
 6. CAPTURED_IMAGE_PATH = "smart_lock_captured.jpg"
 7. RESULT_IMAGE_PATH = "smart_lock_result.jpg"
 8. UNKNOWN_IMAGE_PATH = "unknown_person.jpg"
 9. KEYPAD_INTRUDER_IMAGE_PATH = "keypad_intruder.jpg"
10. LOG_FILE = "smart_lock_access_log.csv"
```

- BITSTREAM_FILE הוגדר שם קובץ ה־ Bitstream שנוצר ב־ Vivado ושימש להגדרת החומרה בצד ה־ PL
- PASSWORD הוגדרה הסיסמה שאפשרה פתיחת המנעול באמצעות לוח המקשים .
- AUTHORIZED_USER_ID הוגדר מספר הזיהוי של המשתמש המורשה כפי שנשמר במודל זיהוי הפנים .
- CONFIDENCE_THRESHOLD הוגדר סף ההחלטה עבור אלגוריתם LBPH תוצאת זיהוי עם ערך Confidence הנמוך או שווה לסף נחשבה להתאמה אפשרית למשתמש המורשה .
- TRAINER_PATH הוגדר הנתיב לקובץ trainer.yml שבו נשמר מודל ה־ LBPH שאומן מראש .
- CAPTURED_IMAGE_PATH הוגדר שם הקובץ שבו נשמרה התמונה הראשונית שצולמה מהמצלמה .
- RESULT_IMAGE_PATH הוגדר שם הקובץ שבו נשמרה התמונה לאחר ביצוע גילוי וזיהוי הפנים וסימון התוצאה .
- UNKNOWN_IMAGE_PATH הוגדר שם הקובץ שבו נשמרה תמונה במקרה שבו זוהה אדם שאינו מורשה .
- KEYPAD_INTRUDER_IMAGE_PATH הוגדר שם הקובץ שבו נשמרה תמונה שצולמה לאחר שלושה ניסיונות שגויים באמצעות לוח המקשים .
- LOG_FILE הוגדר שם קובץ ה־ CSVשבו נשמרו אירועי הגישה והפעולות המרכזיות של המערכת.

8.3. הגדרת מערכת ההתראות על ידי טלגרם

```
1. TELEGRAM_ENABLED = True
2. TELEGRAM_BOT_TOKEN = "                                          "
3. TELEGRAM_CHAT_ID = "         "
```

- TELEGRAM_ENABLED קבע האם מנגנון ההתראות באמצעות Telegram יהיה פעיל. כאשר ערכו היה True ניתן היה לבצע שליחת הודעות ותמונות .
- TELEGRAM_BOT_TOKEN הכיל את מפתח הגישה של הבוט, שבאמצעותו התבצעה ההתקשרות מול Telegram Bot API
- TELEGRAM_CHAT_ID הכיל את מזהה השיחה שאליה נשלחו ההתראות.

8.4. הגדרת קבועי וזמני המערכת

```
 1. DARK_VALUE = 1
 2. BUZZER_FREQ_HZ = 3000
 3. KEYPAD_BEEP_SEC = 0.035
 4. KEYPAD_DEBOUNCE_SEC = 0.18
 5. MOTOR_STOP = 0b000
 6. MOTOR_OPEN = 0b001
 7. MOTOR_CLOSE = 0b010
 8. MOTOR_RESET = 0b100
 9. MOTOR_RUN_TIME_SEC = 4.096
10. DOOR_OPEN_TIMEOUT_SEC = 15.0
11. DOOR_CLOSE_TIMEOUT_SEC = 60.0
12. DOOR_POLL_INTERVAL_SEC = 0.05
13. DOOR_STABLE_TIME_SEC = 0.20
14. DOOR_LOCK_DELAY_SEC = 3.0
15. RELAY_PRE_LIGHT_SEC = 1.0
16. MOTION_COOLDOWN_SEC = 3.0
17. MAX_FAILED_ATTEMPTS = 3
18. MAX_FAILED_FACE_RECOGNITION = 3
19. LOCKOUT_TIME_SEC = 30
```

- DARK_VALUE הוגדר הערך שמייצג מצב חשוך בחיישן האור LM393 בהתאם לבדיקות שבוצעו, הערך 1 הצביע על סביבה חשוכה .
- BUZZER_FREQ_HZ הוגדר תדר העבודה של הזמזם, ונקבע ל־ 3000 Hz
- KEYPAD_BEEP_SEC הוגדר משך הצפצוף הקצר שהופעל לאחר לחיצה על מקש בלוח המקשים .
- KEYPAD_DEBOUNCE_SEC הוגדר זמן ההשהיה ששימש למניעת זיהוי כפול של אותה לחיצה בלוח המקשים .
- MOTOR_STOP הוגדרה פקודת הבקרה שעצרה את מנוע הצעד .
- MOTOR_OPEN הוגדרה פקודת הבקרה ששימשה להפעלת המנוע בכיוון פתיחת המנעול .
- MOTOR_CLOSE הוגדרה פקודת הבקרה ששימשה להפעלת המנוע בכיוון נעילת המנעול .
- MOTOR_RESET הוגדרה פקודת האיפוס של בקר מנוע הצעד .
- MOTOR_RUN_TIME_SEC הוגדר משך הזמן שבו מנוע הצעד הופעל לצורך ביצוע פעולת פתיחה או נעילה מלאה .
- DOOR_OPEN_TIMEOUT_SEC הוגדר פרק הזמן המרבי שבו המערכת המתינה לפתיחת הדלת לאחר פתיחת המנעול .
- DOOR_CLOSE_TIMEOUT_SEC הוגדר פרק הזמן המרבי שבו המערכת המתינה לסגירת הדלת .
- DOOR_POLL_INTERVAL_SEC הוגדר מרווח זמן של 0.05 שניות עבור בדיקה מחזורית של מצב חיישן הדלת .
- DOOR_STABLE_TIME_SEC הוגדר משך הזמן שבו מצב חיישן הדלת נדרש להישאר יציב לפני שהמערכת אישרה שהתרחש שינוי אמיתי במצב הדלת .
- DOOR_LOCK_DELAY_SEC הוגדרה השהיה של 3 שניות לאחר זיהוי סגירת הדלת ולפני תחילת פעולת הנעילה .
- RELAY_PRE_LIGHT_SEC הוגדר זמן המתנה של שנייה אחת לאחר הפעלת התאורה ובטרם בוצע צילום באמצעות המצלמה .

- MOTION_COOLDOWN_SEC הוגדר פרק זמן מינימלי בין שני אירועי תנועה עוקבים של חיישן ה־PIR כדי למנוע הפעלה חוזרת ומהירה של תהליך הזיהוי .
- MAX_FAILED_ATTEMPTS הוגדר מספר ניסיונות הגישה הכושלים המרבי לפני הפעלת מנגנון הנעילה הזמנית .
- MAX_FAILED_FACE_RECOGNITION הוגדר מספר ניסיונות זיהוי הפנים הכושלים שנדרשו לפני שהם נספרו כניסיון גישה כושל אחד .
- LOCKOUT_TIME_SEC הוגדר משך הזמן שבו המערכת נשארה במצב Lockout לאחר הגעה למספר המרבי של ניסיונות גישה כושלים.

8.5. מיפוי הביטים של רכיבי הפלט והקלט

```
1. PIR_BIT = 0
2. LIGHT_SENSOR_BIT = 1
3. DOOR_SENSOR_BIT = 2
4. DOOR_CLOSED_VALUE = 0
5. MOTION_LED_BIT = 0
6. BUZZER_BIT = 1
7. UNLOCKED_LED_BIT = 2
8. LOCKED_LED_BIT = 3
9. RELAY_BIT = 4
```

- PIR_BIT הוגדר הביט שבו נקרא חיישן התנועה PIR החיישן חובר לביט 0 של ממשק הקלט .
- LIGHT_SENSOR_BIT הוגדר הביט שבו נקרא חיישן האור LM393 החיישן חובר לביט 1 של ממשק הקלט .
- DOOR_SENSOR_BIT הוגדר הביט שבו נקרא חיישן הדלת MC-38 החיישן חובר לביט 2 של ממשק הקלט .
- DOOR_CLOSED_VALUE הוגדר הערך שמייצג מצב שבו הדלת סגורה. בהתאם לחיבור ולבדיקות שבוצעו, הערך 0הצביע על דלת סגורה .
- MOTION_LED_BIT הוגדר הביט ששימש להפעלת נורת החיווי של זיהוי תנועה .
- BUZZER_BIT הוגדר הביט ששימש להפעלת הזמזם .
- UNLOCKED_LED_BIT הוגדר הביט ששימש להפעלת נורת החיווי שהצביעה על מצב שבו המנעול פתוח .
- LOCKED_LED_BIT הוגדר הביט ששימש להפעלת נורת החיווי שהצביעה על מצב שבו המנעול נעול .
- RELAY_BIT הוגדר הביט ששימש להפעלת הממסר של מערכת התאורה.

8.6. הגדרת מכונת המצבים של המערכת באמצעות Enum

```
1. class SmartLockState(Enum):
2. locked = "LOCKED"
3. motor_opening = "MOTOR_OPENING"
4. waiting_for_door_open = "WAITING_FOR_DOOR_OPEN"
5. holding_door_open = "HOLDING_DOOR_OPEN"
6. waiting_for_door_close = "WAITING_FOR_DOOR_CLOSE"
7. locking_delay = "LOCKING_DELAY"
8. motor_closing = "MOTOR_CLOSING"
```

השימוש ב־ Enum אפשר לייצג כל מצב בשם ברור במקום להשתמש בערכים מספריים. בכך ניתן היה לנהל בצורה מסודרת את המעבר בין שלבי מחזור הפעולה של המנעול, ולוודא שכל פעולה בוצעה רק כאשר המערכת נמצאה במצב המתאים.

מכונת המצבים הייתה חשובה במיוחד משום שהמערכת לא הסתמכה על רצף אחד ארוך וחוסם, אלא בדקה בכל מחזור של הלולאה הראשית מהו המצב הנוכחי וביצעה בהתאם את הפעולה הנדרשת. כך ניתן היה להמשיך לטפל בחיישנים ובאירועים נוספים גם בזמן תהליך הפתיחה והנעילה.

8.7. הגדרת פונקציות עזר כלליות

- now_string() מחזירה את התאריך והשעה הנוכחיים כמחרוזת בפורמט YYYY-MM-DD HH:MM:SS פונקציה זו שימשה בהמשך לצורך הוספת חותמת זמן לקובץ הלוג ולהודעות שנשלחו באמצעות Telegram
- bit_is_set(value, bit_index) בודקת האם ביט מסוים בתוך ערך בינארי נמצא במצב 1 תחילה הערך מוזז ימינה בהתאם למיקום הביט המבוקש, ולאחר מכן מתבצעת פעולת AND עם הערך 1 הפונקציה מחזירה True אם הביט פעיל ו False אם אינו פעיל .
- set_bit(value, bit_index) משנה ביט מסוים לערך 1 מבלי לשנות את שאר הביטים. פעולה זו בוצעה באמצעות הזזה של 1 למיקום הרצוי וביצוע פעולת OR עם הערך הקיים .
- clear_bit(value, bit_index) משנה ביט מסוים לערך 0 מבלי להשפיע על שאר הביטים. לצורך כך נוצרה מסכה עבור הביט הרצוי, בוצע עליה היפוך באמצעות ~ ולאחר מכן בוצעה פעולת AND עם הערך המקורי.

פונקציות אלו אפשרו לבצע פעולות חוזרות בצורה מרוכזת וברורה. במקום לכתוב בכל פעם מחדש את פעולות הביטים או את יצירת חותמת הזמן, ניתן היה לקרוא לפונקציה המתאימה וכך לשמור על קוד מסודר וקריא יותר.

8.8. הגדרת המחלקה הראשית `SmartLockSystem` ופונקציית האתחול `__init__`

```
class SmartLockSystem:
    def __init__(self):
        if "__file__" in globals():
            self.base_dir = Path(__file__).resolve().parent
        else:
            self.base_dir = Path("/home/xilinx/jupyter_notebooks/FINAL_PROJECT/AUTO_RUN")
        self.overlay = None
        self.motor_gpio = None
        self.peripherals_gpio = None
        self.keypad_gpio = None
        self.motor_channel = None
        self.inputs_channel = None
        self.outputs_channel = None
        self.keypad_channel = None
        self.motor_mask = None
        self.inputs_mask = None
        self.outputs_mask = None
        self.keypad_mask = None
        self.output_state = 0
        self.failed_attempts = 0
        self.failed_face_recognition_count = 0
        self.lockout_until = 0
        self.face_detector = None
        self.face_recognizer = None
        self.last_keypad_key = None
        self.last_keypad_time = 0
        self.keypad_buffer = ""
        self.state = SmartLockState.locked
        self.motor_deadline = 0
        self.active_motor_command = MOTOR_STOP
        self.close_sequence = []
        self.close_step_index = 0
        self.door_wait_target = None
        self.door_wait_started_at = 0
        self.door_wait_timeout = 0
        self.door_wait_stable_time = DOOR_STABLE_TIME_SEC
        self.door_wait_stable_since = None
        self.door_wait_last_value = None
        self.hold_open_until = 0
        self.lock_delay_until = 0
        self.password = PASSWORD
        self.telegram_bot_token = TELEGRAM_BOT_TOKEN
        self.telegram_chat_id = TELEGRAM_CHAT_ID
        self.telegram_enabled = (
            TELEGRAM_ENABLED and
            bool(self.telegram_bot_token) and
            bool(self.telegram_chat_id)
        )
        self.telegram_queue = queue.Queue(maxsize=20)
        self.telegram_worker = threading.Thread(
            target=self._telegram_worker,
            name="smart_lock_telegram_worker",
            daemon=True
        )
        self.telegram_worker.start()
```

פונקציית **__init__** שימשה למעשה כשלב אתחול מבנה התוכנה .בשלב זה עדיין לא נטען ה־ Overlay ולא הופעלו החיישנים והמנוע, אלא הוכנו כל המשתנים והאובייקטים שבהם נעשה שימוש בהמשך. בדרך זו, כאשר הופעלה המערכת, כל תתי־המערכות התחילו ממצב ידוע ומוגדר.

8.9. ניהול נתיבי הקבצים

```
1. def runtime_path(self, path_value):
2.     path = Path(path_value)
3.     if path.is_absolute():
4.     return path
5.     return self.base_dir / path
```

- runtime_path(self, path_value) מקבלת נתיב של קובץ או תיקייה ומחזירה את הנתיב שבו יש להשתמש בפועל בזמן הרצת המערכת .
- path = Path(path_value) ממירה את הנתיב שהתקבל לאובייקט מסוג Path שימוש ב־ Path מאפשר לבצע פעולות על נתיבים בצורה מסודרת ונוחה יותר מאשר עבודה עם מחרוזות רגילות .
- path.is_absolute() בודקת האם הנתיב שהתקבל הוא נתיב מוחלט, כלומר נתיב שמתחיל ממיקום מלא במערכת הקבצים .
- return path אם הנתיב כבר מוחלט, הוא מוחזר כפי שהוא ללא שינוי .
- return self.base_dir / path אם הנתיב הוא יחסי, הוא מחובר לתיקיית הבסיס של הפרויקט שנקבעה קודם לכן ב־ self.base_dir

8.10. ניהול קובץ הלוג

```
 1. def init_log_file(self):
 2.     log_path = self.runtime_path(LOG_FILE)
 3.
 4.     if not log_path.exists():
 5.         with log_path.open(mode="w", newline="") as file:
 6.             writer = csv.writer(file)
 7.
 8.             writer.writerow([
 9.                 "Timestamp",
10.                 "Source",
11.                 "Result",
12.                 "Confidence",
13.                 "FailedAttempts",
14.                 "FaceFailedCount",
15.                 "SystemState",
16.                 "ImagePath"
17.             ])
```

- log_path = self.runtime_path(LOG_FILE) נבנה הנתיב המלא לקובץ הלוג באמצעות הפונקציה runtime_path() שהוגדרה קודם לכן .
- if not log_path.exists(): נבדק האם קובץ הלוג כבר קיים. פעולה זו מנעה יצירה מחדש של הקובץ בכל הפעלה של המערכת .
- with log_path.open(mode="w", newline="") as file: אם הקובץ לא היה קיים, הוא נפתח במצב כתיבה ("w") לצורך יצירתו .
- writer = csv.writer(file) נוצר אובייקט כתיבה של ספריית csv, שבאמצעותו ניתן היה לשמור נתונים בפורמט CSV.
- writer.writerow([...]) נכתבה השורה הראשונה בקובץ, ששימשה ככותרת עבור העמודות השונות.

לאחר יצירת קובץ הלוג הוגדרה הפונקציה ששימשה להוספת אירועים חדשים:

```
def write_log(self, source, result, confidence="", system_state="LOCKED", image_path=""):
    log_path = self.runtime_path(LOG_FILE)

    with log_path.open(mode="a", newline="") as file:
        writer = csv.writer(file)

        writer.writerow([
            now_string(),
            source,
            result,
            confidence,
            self.failed_attempts,
            self.failed_face_recognition_count,
            system_state,
            image_path
        ])
```

- source מציין את מקור האירוע .
- result מציין את תוצאת האירוע .
- confidence="" מאפשר להעביר ערך Confidence במקרה של זיהוי פנים. אם האירוע אינו קשור לזיהוי פנים, נשמר ערך ריק .
- system_state="LOCKED" קובע כברירת מחדל שהמערכת נמצאת במצב נעול, אלא אם נשלח לפונקציה מצב אחר .
- image_path="" מאפשר לשמור את הנתיב לתמונה הקשורה לאירוע. כאשר לא הייתה תמונה, נשמר ערך ריק .
- log_path = self.runtime_path(LOG_FILE) מתקבל הנתיב המלא לקובץ הלוג .
- mode="a" הקובץ נפתח במצב Append כלומר הנתונים החדשים נוספו לסוף הקובץ מבלי למחוק את המידע שכבר נשמר .
- writer = csv.writer(file) נוצר אובייקט כתיבה לקובץ CSV
- now_string() נשמרו התאריך והשעה הנוכחיים בזמן כתיבת האירוע .
- self.failed_attempts נשמר מספר ניסיונות הגישה הכושלים שהיו קיימים באותו רגע .
- self.failed_face_recognition_count נשמר מספר אירועי זיהוי הפנים הכושלים שהצטברו.

באמצעות שתי הפונקציות נוצר מנגנון תיעוד רציף של פעולת המערכת. בכל אירוע משמעותי ניתן היה לקרוא ל־**write_log()** ולשמור את מקור האירוע, תוצאתו, מצב האבטחה, מצב המערכת ותמונה הקשורה לאירוע. כך התקבל תיעוד מסודר שאפשר לעקוב לאחר מכן אחר היסטוריית הפעולות של המנעול.

8.11. מנגנון ההודעות של Telegram

שליחת הודעה:

```
def send_telegram_message(self, message):
    if not self.telegram_enabled:
        return

    self.enqueue_telegram_task({
        "kind": "message",
        "message": message
    })
```

- Message מכיל את תוכן ההודעה שאותה נדרש לשלוח למשתמש .
- if not self.telegram_enabled: נבדק האם מנגנון Telegram פעיל והאם הוגדרו פרטי החיבור הנדרשים .
- return אם Telegram אינו פעיל, הפונקציה מסתיימת ללא ניסיון לשלוח הודעה .
- self.enqueue_telegram_task({...}) במקום לשלוח את ההודעה באופן ישיר, נוצרה משימה חדשה והוכנסה לתור ה־ Telegram.
- "kind": "message" מציין כי סוג המשימה הוא שליחת הודעת טקסט .
- "message": message שומר בתוך המשימה את תוכן ההודעה שנדרש לשלוח .

בדרך זו הלולאה הראשית לא נדרשה להמתין עד לסיום התקשורת עם שרתי Telegram

שליחת תמונה:

```
 1. def send_telegram_photo(self, image_path, caption=""):
 2.     if not self.telegram_enabled:
 3.         return
 4.
 5.     image_file = self.runtime_path(image_path)
 6.
 7.     if not image_file.exists():
 8.         print("Telegram photo was not sent. Image does not exist:", image_path)
 9.         return
10.
11.     self.enqueue_telegram_task({
12.         "kind": "photo",
13.         "image_path": str(image_file),
14.         "caption": caption
15.     })
```

- image_path מכיל את הנתיב לתמונה שאותה נדרש לשלוח .
- caption="" מאפשר לצרף טקסט לתמונה. אם לא נשלח טקסט, נעשה שימוש במחרוזת ריקה .
- if not self.telegram_enabled: נבדק האם שירות Telegram פעיל .
- image_file = self.runtime_path(image_path) הנתיב שהתקבל הומר לנתיב מלא באמצעות הפונקציה runtime_path().
- if not image_file.exists(): נבדק האם קובץ התמונה אכן קיים לפני ניסיון השליחה .
- print(...) אם התמונה אינה קיימת, מודפסת הודעת שגיאה לצורך מעקב ובדיקה .
- return פעולת השליחה מופסקת במקרה שבו הקובץ אינו קיים .
- self.enqueue_telegram_task({...}) אם התמונה קיימת, נוצרת משימת Telegram חדשה ומוכנסת לתור .
- "kind": "photo" מציין שמדובר במשימה מסוג שליחת תמונה .
- "image_path": str(image_file) נשמר הנתיב המלא של התמונה .
- "caption": caption נשמר הטקסט שיצורף לתמונה.

הכנסת משימה חדשה:

```
1. def enqueue_telegram_task(self, task):
2.     try:
3.         self.telegram_queue.put_nowait(task)
4.
5.     except queue.Full:
6.         print("Telegram queue is full. Alert was dropped.")
```

- task מכיל את המשימה שנדרש לבצע, לדוגמה שליחת הודעה או תמונה .
- self.telegram_queue.put_nowait(task) המשימה מוכנסת לתור ללא המתנה. כלומר, אם ניתן להכניס אותה לתור, הפעולה מתבצעת מיד .
- try מאפשר לטפל בצורה בטוחה במקרה שבו לא ניתן להוסיף משימה חדשה .
- except queue.Full: אם התור הגיע לגודל המרבי שהוגדר קודם לכן, מתקבלת חריגה מסוג queue.Full
- print(...) במקרה כזה מודפסת הודעה המציינת שההתראה לא הוכנסה לתור.

השימוש ב-**put_nowait**() היה חשוב משום שמטרת התור הייתה למנוע חסימה של פעולת המערכת הראשית.

העובד של הטלגרם:

```
def _telegram_worker(self):
    while True:
        task = self.telegram_queue.get()

        try:
            if not self.telegram_enabled:
                continue

            if task.get("kind") == "message":
                self._send_telegram_message_now(task.get("message", ""))

            elif task.get("kind") == "photo":
                self._send_telegram_photo_now(
                    task.get("image_path", ""),
                    task.get("caption", "")
                )

        finally:
            self.telegram_queue.task_done()
```

- while True: ה־Worker פועל באופן רציף כל עוד התוכנית פעילה .
- task = self.telegram_queue.get() נלקחת המשימה הבאה מתוך תור ה־ Telegram אם אין משימות, ה־ Thread ממתין עד שתתקבל משימה חדשה .
- if not self.telegram_enabled: מתבצעת בדיקה נוספת שמנגנון Telegram עדיין פעיל .
- task.get("kind") נקרא סוג המשימה שנשמר בתור .
- אם סוג המשימה הוא "message" נקראת הפונקציה _send_telegram_message_now() לצורך שליחת הודעת טקסט .
- אם סוג המשימה הוא "photo" נקראת הפונקציה _send_telegram_photo_now() ומועברים אליה נתיב התמונה והטקסט המצורף .
- finally: מבטיח שהפעולה הבאה תתבצע גם אם התרחשה שגיאה בזמן הטיפול במשימה .
- self.telegram_queue.task_done() מסמן לתור שהטיפול במשימה שהוצאה ממנו הסתיים.

8.12. השליחה בפועל ל־ Telegram Bot API

שליחת הודעה בפועל:

```
def _send_telegram_message_now(self, message):
    try:
        url = "https://api.telegram.org/bot{}/sendMessage".format(self.telegram_bot_token)

        data = {
            "chat_id": self.telegram_chat_id,
            "text": message
        }

        response = requests.post(
            url,
            data=data,
            timeout=10
        )

        if response.status_code != 200:
            print("Telegram message failed:", response.text)

    except Exception as e:
        print("Telegram message error:", e)
```

- message מכיל את תוכן הודעת הטקסט שנדרש לשלוח .
- try: תחילת אזור מוגן שמאפשר לטפל בשגיאות תקשורת מבלי לגרום להפסקת פעולת המערכת כולה .
- url = ... נבנה קישור הגישה ל־ Telegram Bot API. הקישור כולל את ה־ Bot Token של המערכת ואת הפקודה sendMessage, שמיועדת לשליחת הודעות טקסט .
- data = {...} נוצר מבנה נתונים המכיל את הפרטים שנשלחו ל־ Telegram
- "chat_id" מציין את מזהה השיחה שאליה נשלחה ההודעה .
- "text" מכיל את תוכן ההודעה עצמה .
- requests.post() נשלחה בקשת HTTP מסוג POST אל שרתי Telegram.
- url כתובת היעד שאליה נשלחה הבקשה .
- data=data הנתונים שהועברו בבקשה .
- timeout=10 הוגדר זמן המתנה מרבי של 10 שניות לתגובה מהשרת. אם לא התקבלה תגובה בזמן זה, הופעלה חריגה במקום להמתין ללא הגבלה .
- response.status_code נבדק קוד התגובה שהתקבל משרת Telegram
- 200 קוד HTTP שמציין שהבקשה בוצעה בהצלחה .
- if response.status_code != 200: אם התקבל קוד אחר, הודפסה הודעת שגיאה יחד עם התגובה שהתקבלה מהשרת .
- except Exception as e: במקרה של תקלה, לדוגמה ניתוק מהרשת או חריגת זמן, השגיאה נתפסה והודפסה במקום לעצור את התוכנית.

שליחת תמונה בפועל:

```
1. def _send_telegram_photo_now(self, image_path, caption=""):
2.     image_file = Path(image_path)
3.
4.     if not image_file.exists():
5.         print("Telegram photo was not sent. Image does not exist:", image_path)
6.         return
7.
8.     try:
9.         url = "https://api.telegram.org/bot{}/sendPhoto".format(self.telegram_bot_token)
10.
11.        with image_file.open("rb") as file:
12.            files = {
13.                "photo": file
14.            }
15.
16.            data = {
17.                "chat_id": self.telegram_chat_id,
18.                "caption": caption
19.            }
20.
21.            response = requests.post(
22.                url,
23.                data=data,
24.                files=files,
25.                timeout=20
26.            )
27.
28.        if response.status_code != 200:
29.            print("Telegram photo failed:", response.text)
30.
31.    except Exception as e:
32.        print("Telegram photo error:", e)
```

- image_path מכיל את הנתיב לתמונה שנדרש לשלוח .
- caption="" מכיל טקסט אופציונלי שניתן לצרף לתמונה .
- image_file = Path(image_path) הנתיב לתמונה הומר לאובייקט Path כדי לאפשר בדיקה ופתיחה נוחה של הקובץ .
- if not image_file.exists(): נבדק שקובץ התמונה אכן קיים לפני ניסיון השליחה .
- return אם הקובץ אינו קיים, הפונקציה מסתיימת מבלי לבצע בקשת רשת .
- url = ... נבנתה כתובת ה־Telegram Bot API, הפעם באמצעות הפעולה sendPhoto
- with image_file.open("rb") as file: קובץ התמונה נפתח במצב קריאה בינארי (rb), הדרוש לצורך שליחת קובצי תמונה דרך HTTP.
- files = {"photo": file} נוצר מבנה המכיל את קובץ התמונה שנשלח ל־Telegram
- data הכיל את פרטי השיחה ואת הטקסט המצורף לתמונה .
- "chat_id" קבע את השיחה שאליה תישלח התמונה .
- "caption" הכיל את הטקסט שיופיע יחד עם התמונה .
- requests.post() נשלחה בקשת POST לשרת Telegram
- files=files צורף קובץ התמונה לבקשה .
- timeout=20 הוגדר זמן המתנה של עד 20 שניות, ארוך יותר משליחת הודעת טקסט משום שהעלאת תמונה דורשת העברת כמות גדולה יותר של נתונים .
- if response.status_code != 200: אם השליחה לא הסתיימה בהצלחה, הודפסה הודעת שגיאה .
- except Exception as e: כל תקלה בתהליך פתיחת הקובץ או התקשורת עם Telegram טופלה מבלי לעצור את פעולת המערכת.

שתי פונקציות אלו הן השלב שבו המשימות שהוכנסו קודם לתור מתורגמות בפועל לבקשות HTTP אל שרתי Telegram.
ההפרדה בין יצירת המשימה לבין ביצוע השליחה אפשרה למערכת הראשית להמשיך לפעול גם בזמן תקשורת עם הרשת.

8.13. טעינת ה־Overlay ואתחול ממשקי ה־AXI GPIO

בחלק זה בוצעה טעינת ה־Overlay של המערכת ואתחול ממשקי ה־AXI GPIO ששימשו לתקשורת בין תוכנת ה־Python לבין רכיבי החומרה שמומשו בצד ה־PL.

תחילה נטען קובץ ה־Bitstream באמצעות המחלקה Overlay לאחר טעינת הקובץ התקבלה גישה לרכיבי ה־IP שהוגדרו ב־Vivado, והתוכנה הדפיסה את שמות הרכיבים הזמינים לצורך בדיקה שהמערכת נטענה בצורה תקינה.

לאחר מכן נוצרו שלושה ממשקי AxiGPIO עיקריים. ממשק אחד שימש לשליחת פקודות אל בקר מנוע הצעד, ממשק נוסף שימש לקריאת החיישנים ולהפעלת רכיבי הפלט, וממשק שלישי שימש לקבלת נתונים מבקר לוח המקשים.

מתוך ממשקי ה־GPIO הוגדרו הערוצים המתאימים. ערוץ אחד שימש לבקרת המנוע, ערוץ אחד לקריאת כניסות החיישנים, ערוץ נוסף להפעלת רכיבי הפלט, וערוץ נוסף לקריאת מצב לוח המקשים.

בשלב האחרון חושבו מסכות הביטים עבור כל אחד מהערוצים. המסכות נקבעו בהתאם לרוחב ממשקי ה־GPIO שהוגדרו בתכנון החומרה, ואפשרו לוודא שבפעולות הקריאה והכתיבה נעשה שימוש רק בביטים השייכים לאותו ממשק.

באמצעות שלב זה נוצר למעשה החיבור בין תוכנת הבקרה לבין החומרה של המערכת, ולאחר השלמתו ניתן היה לקרוא את מצב החיישנים, לקבל נתונים מלוח המקשים, להפעיל את רכיבי הפלט ולשלוח פקודות אל בקר מנוע הצעד.

```python
def load_system_overlay(self):
    print("Loading overlay...")

    self.overlay = Overlay(BITSTREAM_FILE)

    print("Overlay loaded successfully.")
    print("Available IP blocks:")

    for ip_name in self.overlay.ip_dict.keys():
        print("-", ip_name)

    self.motor_gpio = AxiGPIO(self.overlay.ip_dict["motor_control_gpio"])
    self.peripherals_gpio = AxiGPIO(self.overlay.ip_dict["peripherals_gpio"])
    self.keypad_gpio = AxiGPIO(self.overlay.ip_dict["keypad_status_gpio"])

    self.motor_channel = self.motor_gpio.channel1
    self.inputs_channel = self.peripherals_gpio.channel1
    self.outputs_channel = self.peripherals_gpio.channel2
    self.keypad_channel = self.keypad_gpio.channel1

    self.motor_mask = self.confirm_gpio_mask(
        ip_name="motor_control_gpio",
        channel_number=1,
        channel=self.motor_channel,
        channel_name="motor_control_gpio.channel1"
    )

    self.inputs_mask = self.confirm_gpio_mask(
        ip_name="peripherals_gpio",
        channel_number=1,
        channel=self.inputs_channel,
        channel_name="peripherals_gpio.channel1"
    )

    self.outputs_mask = self.confirm_gpio_mask(
        ip_name="peripherals_gpio",
        channel_number=2,
        channel=self.outputs_channel,
        channel_name="peripherals_gpio.channel2"
    )

    self.keypad_mask = self.confirm_gpio_mask(
        ip_name="keypad_status_gpio",
        channel_number=1,
        channel=self.keypad_channel,
        channel_name="keypad_status_gpio.channel1"
    )

    print("GPIO blocks initialized successfully.")
```

8.14. זיהוי רוחב ממשקי ה־ GPIO, יצירת מסכות הביטים ואתחול יציאות המערכת

```
def confirm_gpio_mask(self, ip_name, channel_number, channel, channel_name):
    width = self.get_hwh_gpio_width(
        ip_name,
        channel_number
    )

    if width is None:
        for attribute_name in ("length", "width", "_width"):
            width_value = getattr(
                channel,
                attribute_name,
                None
            )

            if isinstance(width_value, int) and width_value > 0:
                width = width_value
                break

    if width is None or width <= 0:
        raise RuntimeError("Could not confirm GPIO width for " + channel_name)

    mask = (1 << width) - 1

    print(
        "GPIO mask confirmed for {}: width = {}".format(
            channel_name,
            width
        )
    )

    return mask
```

```
def get_hwh_gpio_width(self, ip_name, channel_number):
    ip_info = self.overlay.ip_dict.get(
        ip_name,
        {}
    )

    parameters = ip_info.get(
        "parameters",
        {}
    )

    if channel_number == 1:
        keys = [
            "C_GPIO_WIDTH",
            "GPIO_WIDTH"
        ]

    else:
        keys = [
            "C_GPIO2_WIDTH",
            "GPIO2_WIDTH"
        ]

    for key in keys:
        if key in parameters:
            try:
                return int(
                    str(parameters[key]),
                    0
                )

            except Exception:
                return None
    return None
```

בחלק זה נקבע רוחב הביטים של כל אחד מממשקי ה־ GPIO ונבנתה עבורו מסכה מתאימה. מטרת המסכה הייתה להגביל את פעולות הקריאה והכתיבה רק לביטים השייכים בפועל לאותו ממשק.

תחילה נעשה ניסיון לקרוא את רוחב הממשק מתוך המידע שנשמר בקובץ ה־ HWH של ה־Overlay קובץ זה מכיל מידע על רכיבי ה־ IP והפרמטרים שהוגדרו עבורם ב־ Vivado בהתאם לערוץ שנבחר, נקרא הפרמטר המתאר את מספר הביטים שהוגדרו עבור אותו ערוץ GPIO

אם לא ניתן היה לקבל את רוחב הממשק ישירות מתוך נתוני ה־ HWH התוכנית ניסתה לקבל אותו מתוך מאפייני אובייקט ה־ GPIO עצמו. בדרך זו נוצר מנגנון גיבוי שאפשר לוודא שרוחב הערוץ זוהה בצורה תקינה.

לאחר שנקבע מספר הביטים, נוצרה מסכה בינארית שבה כל הביטים הפעילים בממשק קיבלו את הערך 1 לדוגמה, עבור ממשק ברוחב 3 ביטים התקבלה המסכה 0b111 מסכה זו שימשה בהמשך בעת קריאה וכתיבה ל־ GPIO כדי למנוע שימוש בביטים שאינם חלק מהממשק שהוגדר.

אם לא ניתן היה לזהות את רוחב הממשק, הופעלה שגיאה והמשך פעולת המערכת נעצר. בדיקה זו נועדה למנוע מצב שבו התוכנה מנסה לשלוט בחומרה באמצעות הגדרה שגויה של מספר הביטים.

לאחר קביעת המסכות בוצע אתחול מצב היציאות:

```
def init_outputs(self):
    self.output_state = 0

    self.write_outputs()

    self.set_locked_led(True)
    self.set_unlocked_led(False)
    self.set_motion_led(False)
    self.set_relay(False)
    self.set_buzzer(False)

    self.motor_write(MOTOR_STOP)

    print("Outputs initialized. System state: LOCKED")
```

בחלק זה אותחלו כל רכיבי הפלט של המערכת למצב התחלתי ידוע ובטוח. בתחילת הפעולה אופס ערך יציאות ה־ GPIO ולאחר מכן הוגדרו רכיבי החיווי וההפעלה בהתאם למצב ההתחלתי של המנעול.

נורת החיווי של מצב נעול הופעלה, בעוד שנורת המצב הפתוח, נורת זיהוי התנועה, הממסר והזמזם כובו. בנוסף נשלחה לבקר מנוע הצעד פקודת MOTOR_STOP כדי לוודא שהמנוע אינו פועל בזמן אתחול המערכת.

באמצעות שלב זה המערכת התחילה תמיד ממצב מוגדר של דלת נעולה, ללא הפעלה לא רצויה של המנוע או של אחד מרכיבי הפלט.

```
def write_outputs(self):
    self.outputs_channel.write(
        self.output_state,
        self.outputs_mask
    )

def set_output_bit(self, bit_index, state):
    if state:
        self.output_state = set_bit(
            self.output_state,
            bit_index
        )

    else:
        self.output_state = clear_bit(
            self.output_state,
            bit_index
        )

    self.write_outputs()

def read_inputs(self):
    return self.inputs_channel.read() & self.inputs_mask

def read_pir(self):
    value = self.read_inputs()

    return 1 if bit_is_set(
        value,
        PIR_BIT
    ) else 0

def read_light_sensor(self):
    value = self.read_inputs()

    return 1 if bit_is_set(
        value,
        LIGHT_SENSOR_BIT
    ) else 0

def read_door_sensor(self):
    value = self.read_inputs()

    return 1 if bit_is_set(
        value,
        DOOR_SENSOR_BIT
    ) else 0

def is_door_closed(self):
    return self.read_door_sensor() == DOOR_CLOSED_VALUE

def get_door_state(self):
    return "CLOSED" if self.is_door_closed() else "OPEN"

def is_dark(self):
    return self.read_light_sensor() == DARK_VALUE

def motor_write(self, command):
    self.active_motor_command = command

    self.motor_channel.write(
        command,
        self.motor_mask
    )
```

בחלק זה הוגדרו הפונקציות שאחראיות על התקשורת השוטפת בין תוכנת ה־ Python לבין רכיבי החומרה המחוברים דרך ממשקי ה־ GPIO

עבור רכיבי הפלט נשמר מצב כולל במשתנה output_state כאשר נדרש להפעיל או לכבות רכיב מסוים, שונה רק הביט המתאים לו, ולאחר מכן הערך המעודכן נכתב אל ערוץ הפלט. בדרך זו ניתן היה לשלוט בכל רכיב בנפרד מבלי לשנות את מצב שאר היציאות.

עבור רכיבי הקלט נקרא הערך המלא של ערוץ ה־ GPIO ומתוכו חולצו הביטים המתאימים לחיישן התנועה PIR לחיישן האור LM393 ולחיישן הדלת MC-38 בנוסף הוגדרו פונקציות עזר שהמירו את הערכים הגולמיים למצבים ברורים יותר, כגון בדיקה האם הדלת סגורה או האם הסביבה חשוכה.

בחלק האחרון הוגדרה הפונקציה motor_write() ששימשה לשליחת פקודות הבקרה אל בקר מנוע הצעד שמומש בצד ה־ FPGA הפונקציה שמרה את הפקודה הפעילה ושלחה אותה דרך ערוץ ה־ GPIO הייעודי למנוע.

באמצעות פונקציות אלו נוצרה שכבת גישה מסודרת לחומרה, כך ששאר התוכנית לא נדרשה לבצע קריאה או כתיבה ישירה ל־ GPIO אלא השתמשה בפונקציות הייעודיות שהוגדרו כאן.

8.16. שליטה בנורות החיווי, בממסר ובזמזם

```
 1. def set_motion_led(self, state):
 2.     self.set_output_bit(
 3.         MOTION_LED_BIT,
 4.         state
 5.     )
 6.
 7. def set_buzzer(self, state):
 8.     self.set_output_bit(
 9.         BUZZER_BIT,
10.         state
11.     )
12.
13. def set_unlocked_led(self, state):
14.     self.set_output_bit(
15.         UNLOCKED_LED_BIT,
16.         state
17.     )
18.
19. def set_locked_led(self, state):
20.     self.set_output_bit(
21.         LOCKED_LED_BIT,
22.         state
23.     )
24.
25. def set_relay(self, state):
26.     self.set_output_bit(
27.         RELAY_BIT,
28.         state
29.     )
30.
31. def beep(self, duration_sec=0.15):
32.     period = 1.0 / BUZZER_FREQ_HZ
33.     half_period = period / 2.0
34.     end_time = time.monotonic() + duration_sec
35.
36.     while time.monotonic() < end_time:
37.         self.set_buzzer(True)
38.
39.         time.sleep(
40.             half_period
41.         )
42.
43.         self.set_buzzer(False)
44.
45.         time.sleep(
46.             half_period
47.         )
48.
49. def keypad_beep(self):
50.     self.beep(
51.         KEYPAD_BEEP_SEC
```

```
52.     )
53.
54. def success_buzzer(self):
55.     self.beep(
56.         0.12
57.     )
58.
59.     time.sleep(
60.         0.08
61.     )
62.
63.     self.beep(
64.         0.12
65.     )
66.
67. def denied_buzzer(self):
68.     self.beep(
69.         0.35
70.     )
71.
72. def lockout_buzzer(self):
73.     for _ in range(3):
74.         self.beep(
75.             0.25
76.         )
77.
78.         time.sleep(
79.             0.15
80.         )
```

בחלק זה הוגדרו הפונקציות שאחראיות על הפעלת רכיבי החיווי וההתראה של המערכת. עבור כל אחד מהרכיבים הוגדרה פונקציה ייעודית, אשר הפעילה או כיבתה את הביט המתאים בממשק הפלט באמצעות הפונקציה set_output_bit() שהוגדרה קודם לכן.

באמצעות פונקציות אלו נשלטו נורת החיווי של זיהוי התנועה, נורת המצב הפתוח, נורת המצב הנעול, הממסר והזמזם. הפרדה זו אפשרה לשאר חלקי התוכנית להפעיל כל רכיב באמצעות פקודה ברורה, מבלי לגשת ישירות למיפוי הביטים.

בנוסף הוגדרה הפונקציה beep() ששימשה ליצירת אות מחזורי עבור הזמזם. התדר נקבע בהתאם לערך BUZZER_FREQ_HZ והזמזם הופעל וכובה לסירוגין עד לסיום משך הזמן שנדרש. כך נוצר צפצוף בתדר קבוע למשך זמן מוגדר.

על בסיס פונקציה זו נבנו מספר סוגי חיווי קולי keypad_beep() הפעילה צפצוף קצר לאחר לחיצה על מקש , success_buzzer() יצרה שני צפצופים קצרים לציון גישה מאושרת denied_buzzer(), יצרה צפצוף ממושך יותר במקרה של דחיית גישה, ו־ lockout_buzzer() הפעילה שלושה צפצופים לציון כניסת המערכת למצב Lockout

באמצעות דפוסי הצפצוף השונים ניתן היה לספק למשתמש משוב מיידי על הפעולה שהתבצעה, גם ללא צורך להסתכל על נורות החיווי.

```
def reset_motor_controller(self):
    print("Resetting motor controller...")

    self.motor_write(
        MOTOR_RESET
    )

    time.sleep(
        0.2
    )

    self.motor_write(
        MOTOR_STOP
    )

    time.sleep(
        0.2
    )

    print("Motor controller reset completed.")

def start_open_lock(self):
    print("Opening lock...")

    self.set_locked_led(False)
    self.set_unlocked_led(True)

    self.motor_write(
        MOTOR_OPEN
    )

    self.motor_deadline = time.monotonic() + MOTOR_RUN_TIME_SEC
    self.state = SmartLockState.motor_opening

def finish_open_lock(self):
    self.motor_write(
        MOTOR_STOP
    )

    print("Lock opened.")

    initial_raw = self.read_door_sensor()
    initial_state = "CLOSED" if initial_raw == 0 else "OPEN"

    print(
        "Door state after unlocking: raw = {} | state = {}".format(
            initial_raw,
            initial_state
        )
    )

    print("Waiting for the door to open...")

    self.start_door_wait(
        state=SmartLockState.waiting_for_door_open,
        target_value=1,
        timeout_sec=DOOR_OPEN_TIMEOUT_SEC,
        stable_time_sec=DOOR_STABLE_TIME_SEC
    )

def start_close_lock(self):
    if not self.is_door_closed():
        print("Door reopened before locking.")
        print("Waiting for the door to close again...")

        self.set_locked_led(False)
        self.set_unlocked_led(True)

        self.start_door_wait(
            state=SmartLockState.waiting_for_door_close,
            target_value=DOOR_CLOSED_VALUE,
```

```
            timeout_sec=DOOR_CLOSE_TIMEOUT_SEC,
            stable_time_sec=0.50
        )

        return

    print("Closing lock...")

    self.set_unlocked_led(False)
    self.set_locked_led(True)

    self.close_sequence = [
        (MOTOR_STOP, 0.10, ""),
        (MOTOR_RESET, 0.20, ""),
        (MOTOR_STOP, 0.20, ""),
        (MOTOR_CLOSE, MOTOR_RUN_TIME_SEC, "Sending MOTOR_CLOSE command..."),
        (MOTOR_STOP, 0.10, "")
    ]

    self.close_step_index = 0
    self.state = SmartLockState.motor_closing

    self.run_next_close_step()

def run_next_close_step(self):
    if self.close_step_index >= len(self.close_sequence):
        print("Lock closed.")

        self.state = SmartLockState.locked
        self.close_sequence = []
        self.close_step_index = 0

        return

    command, duration_sec, message = self.close_sequence[self.close_step_index]

    if command == MOTOR_CLOSE and not self.is_door_closed():
        print("Door reopened before MOTOR_CLOSE.")
        print("Waiting for the door to close again...")

        self.motor_write(
            MOTOR_STOP
        )

        self.set_locked_led(False)
        self.set_unlocked_led(True)

        self.start_door_wait(
            state=SmartLockState.waiting_for_door_close,
            target_value=DOOR_CLOSED_VALUE,
            timeout_sec=DOOR_CLOSE_TIMEOUT_SEC,
            stable_time_sec=0.50
        )

        return

    if message:
        print(message)

    self.motor_write(
        command
    )

    self.motor_deadline = time.monotonic() + duration_sec
    self.close_step_index += 1
```

בחלק זה הוגדר מנגנון השליטה בתהליך הפתיחה והנעילה של המנעול באמצעות מנוע הצעד.

בתחילת העבודה בוצע איפוס לבקר המנוע באמצעות reset_motor_controller() תחילה נשלחה פקודת MOTOR_RESET ולאחר השהיה קצרה נשלחה פקודת MOTOR_STOP פעולה זו הביאה את בקר המנוע למצב התחלתי ידוע לפני תחילת פעולת המערכת.

בעת פתיחת המנעול נקראה הפונקציה start_open_lock() הפונקציה עדכנה את נורות החיווי, שלחה את הפקודה MOTOR_OPEN לבקר המנוע והגדירה את הזמן שבו פעולת הפתיחה אמורה להסתיים. במקביל עודכן מצב מכונת המצבים ל- motor_opening

עם סיום זמן פעולת המנוע נקראה finish_open_lock() בשלב זה המנוע נעצר, נקרא מצבו הנוכחי של חיישן הדלת, והמערכת עברה למצב המתנה לפתיחת הדלת. מכאן והלאה המשך תהליך הנעילה נקבע לפי מצב חיישן ה- MC-38 ולא רק לפי זמן קבוע.

תהליך הנעילה החל באמצעות start_close_lock() לפני הפעלת המנוע בוצעה בדיקה נוספת שהדלת אכן סגורה. אם הדלת הייתה פתוחה, פעולת הנעילה לא בוצעה והמערכת חזרה להמתין לסגירתה. בדיקה זו נועדה למנוע הפעלת מנגנון הנעילה כאשר הדלת אינה נמצאת במקומה.

כאשר הדלת הייתה סגורה, הוגדר רצף פקודות עבור בקר המנוע: עצירה, איפוס, עצירה נוספת, פקודת סגירה ולבסוף עצירה. הפונקציה run_next_close_step() ביצעה בכל פעם שלב אחד מתוך הרצף ושמרה את הזמן שבו עליו להסתיים. כך פעולת הנעילה בוצעה בצורה הדרגתית כחלק ממכונת המצבים, ללא רצף ארוך שחוסם את פעולת התוכנית.

בנוסף, לפני שליחת פקודת MOTOR_CLOSE בוצעה בדיקה נוספת של חיישן הדלת. אם זוהתה פתיחה מחדש של הדלת, המנוע נעצר מיד והמערכת חזרה למצב המתנה לסגירת הדלת. בכך שולבה בדיקת בטיחות גם ממש לפני פעולת הנעילה עצמה.

8.18. התחלת מחזור הדלת והמתנה לשינוי במצב החיישן

```
def unlock_then_auto_close(self):
    """Start a non-blocking 0 -> 1 -> 0 door cycle, then lock."""

    if self.is_access_cycle_active():
        print("Access cycle is already active. New unlock request ignored.")
        return

    self.start_open_lock()


def is_access_cycle_active(self):
    return self.state != SmartLockState.locked


def start_door_wait(self, state, target_value, timeout_sec, stable_time_sec):
    self.state = state
    self.door_wait_target = target_value
    self.door_wait_started_at = time.monotonic()
    self.door_wait_timeout = timeout_sec
    self.door_wait_stable_time = stable_time_sec
    self.door_wait_stable_since = None
    self.door_wait_last_value = None


def poll_door_wait(self):
    now = time.monotonic()

    if now - self.door_wait_started_at >= self.door_wait_timeout:
        return False

    raw_value = self.read_door_sensor()

    if raw_value != self.door_wait_last_value:
        state = "CLOSED" if raw_value == 0 else "OPEN"

        print(
            "Door sensor changed: raw = {} | state = {}".format(
                raw_value,
                state
            )
        )

        self.door_wait_last_value = raw_value

    if raw_value == self.door_wait_target:
        if self.door_wait_stable_since is None:
```

```
47.             self.door_wait_stable_since = now
48.
49.         elif now - self.door_wait_stable_since >= self.door_wait_stable_time:
50.             return True
51.
52.     else:
53.         self.door_wait_stable_since = None
54.
55.     return None
```

בחלק זה הוגדר מנגנון ההמתנה לשינוי במצב הדלת לאחר שהתקבלה הרשאת גישה. המנגנון פעל כחלק ממכונת המצבים של המערכת ואפשר לעקוב אחר חיישן ה־ MC-38 מבלי לעצור את פעולת הלולאה הראשית.

הפונקציה unlock_then_auto_close() התחילה מחזור חדש של פתיחת המנעול רק כאשר לא היה כבר מחזור פתיחה או נעילה פעיל. בדיקה זו מנעה מצב שבו מתקבלת בקשת פתיחה נוספת בזמן שהמערכת עדיין מטפלת בבקשה קודמת.

הפונקציה start_door_wait() שימשה להכנת המערכת להמתנה למצב מסוים של חיישן הדלת. בכל הפעלה נשמר מצב היעד שאליו המערכת ממתינה, זמן תחילת ההמתנה, זמן ההמתנה המרבי ומשך הזמן שבו הקריאה חייבת להישאר יציבה לפני שהיא נחשבת תקינה.

הבדיקה עצמה בוצעה באמצעות poll_door_wait() בכל קריאה לפונקציה נקרא מצבו העדכני של חיישן ה־ MC-38 ונבדק האם התקבל מצב היעד. אם המערכת המתינה לפתיחת הדלת, מצב היעד היה 1, ואם המתינה לסגירתה, מצב היעד היה 0

כדי למנוע תגובה לשינוי רגעי או לרעש באות, לא התקבלה קריאה חדשה מיד עם הופעתה. הערך נדרש להישאר יציב במשך הזמן שהוגדר ב־ door_wait_stable_time רק לאחר מכן הוחזר הערך True אשר ציין שהשינוי במצב הדלת אושר.

אם זמן ההמתנה המרבי הסתיים לפני שהתקבל המצב המבוקש, הוחזר False כל עוד ההמתנה עדיין פעילה ולא התקבלה החלטה, הוחזר None

באמצעות שיטה זו ניתן היה לבצע מעקב רציף אחר מצב הדלת בצורה לא־חוסמת, כאשר הלולאה הראשית המשיכה לפעול ובכל מחזור בדקה מחדש האם הדלת נפתחה או נסגרה.

8.19. השהיית הנעילה וטיפול במצבי Timeout של הדלת

```
 1. def start_lock_delay(self):
 2.     print("Door closing confirmed.")
 3.
 4.     print(
 5.         "Waiting {} seconds before locking...".format(
 6.             int(DOOR_LOCK_DELAY_SEC)
 7.         )
 8.     )
 9.
10.     self.lock_delay_until = time.monotonic() + DOOR_LOCK_DELAY_SEC
11.     self.state = SmartLockState.locking_delay
12.
13.
14. def update_lock_delay(self):
15.     if self.read_door_sensor() != DOOR_CLOSED_VALUE:
16.         print("Door reopened during the locking delay.")
17.         print("Waiting for the door to close again...")
18.
19.         self.start_door_wait(
20.             state=SmartLockState.waiting_for_door_close,
21.             target_value=DOOR_CLOSED_VALUE,
22.             timeout_sec=DOOR_CLOSE_TIMEOUT_SEC,
23.             stable_time_sec=0.50
24.         )
25.
26.         return
27.
28.     if time.monotonic() >= self.lock_delay_until:
29.         print(
```

```
            "Door remained closed for {} seconds.".format(
                int(DOOR_LOCK_DELAY_SEC)
            )
        )

        self.start_close_lock()


def handle_door_open_timeout(self):
    print("Door was not opened within the allowed time.")
    print("Closing the lock automatically because the door remained closed.")

    self.write_log(
        source="DOOR_SENSOR",
        result="DOOR_NOT_OPENED",
        confidence="",
        system_state="UNLOCKED",
        image_path=""
    )

    if self.is_door_closed():
        self.start_close_lock()

    else:
        self.handle_door_left_open()
        self.state = SmartLockState.locked


def handle_door_left_open(self):
    print("Door remained open beyond the allowed time.")
    print("The lock will remain open to avoid locking while the door is open.")

    self.write_log(
        source="DOOR_SENSOR",
        result="DOOR_LEFT_OPEN",
        confidence="",
        system_state="DOOR_OPEN",
        image_path=""
    )

    self.send_telegram_message(
        "Smart Lock Alert\n"
        "The door remained open for more than {} seconds\n"
        "The lock was not closed\n"
        "Time: {}".format(
            int(DOOR_CLOSE_TIMEOUT_SEC),
            now_string()
        )
    )
```

בחלק זה הוגדר מנגנון הבטיחות שמנהל את הזמן שבין זיהוי סגירת הדלת לבין הפעלת המנוע לצורך נעילתה, וכן את הטיפול במקרים שבהם הדלת אינה נפתחת או אינה נסגרת בזמן שהוגדר.

לאחר שחיישן ה־ MC-38 אישר שהדלת נסגרה, הפונקציה start_lock_delay() העבירה את המערכת למצב locking_delay והפעילה השהיה של 3 שניות לפני תחילת הנעילה. ההשהיה נועדה לוודא שהדלת נשארה במצב סגור ולא נסגרה רק באופן רגעי.

במהלך ההשהיה הפונקציה update_lock_delay() המשיכה לבדוק את מצב חיישן הדלת. אם הדלת נפתחה מחדש, תהליך הנעילה בוטל והמערכת חזרה להמתין לסגירת הדלת. רק כאשר הדלת נשארה סגורה לאורך כל זמן ההשהיה הופעלה הפונקציה start_close_lock() והחל תהליך נעילת המנעול.

בנוסף הוגדר טיפול במקרה שבו לאחר פתיחת המנעול המשתמש אינו פותח את הדלת בזמן המותר. במקרה זה נשמר אירוע מסוג DOOR_NOT_OPENED בקובץ הלוג, ואם הדלת עדיין סגורה המערכת מתחילה מחדש את פעולת הנעילה.

במקרה שבו הדלת נפתחה אך נשארה פתוחה מעבר לזמן המרבי שהוגדר, המערכת אינה מפעילה את מנוע הנעילה. במקום זאת נשמר אירוע מסוג DOOR_LEFT_OPEN בקובץ הלוג ונשלחת התראה באמצעות Telegram. פעולה זו נועדה למנוע ניסיון לנעול את המנגנון כאשר הדלת עדיין פתוחה ולשמור על פעולה בטוחה של המערכת.

```
def update_access_cycle(self):
    now = time.monotonic()

    if self.state == SmartLockState.motor_opening:
        if now >= self.motor_deadline:
            self.finish_open_lock()

        return

    if self.state == SmartLockState.waiting_for_door_open:
        door_opened = self.poll_door_wait()

        if door_opened is True:
            print("Door opening confirmed.")

            self.hold_open_until = time.monotonic() + 0.75
            self.state = SmartLockState.holding_door_open

        elif door_opened is False:
            self.handle_door_open_timeout()

        return

    if self.state == SmartLockState.holding_door_open:
        if now >= self.hold_open_until:
            print("Waiting for the door to close...")

            self.start_door_wait(
                state=SmartLockState.waiting_for_door_close,
                target_value=0,
                timeout_sec=DOOR_CLOSE_TIMEOUT_SEC,
                stable_time_sec=0.50
            )

        return

    if self.state == SmartLockState.waiting_for_door_close:
        door_closed = self.poll_door_wait()

        if door_closed is True:
            self.start_lock_delay()

        elif door_closed is False:
            self.handle_door_left_open()
            self.state = SmartLockState.locked

        return

    if self.state == SmartLockState.locking_delay:
        self.update_lock_delay()

        return

    if self.state == SmartLockState.motor_closing:
        if self.active_motor_command == MOTOR_CLOSE and not self.is_door_closed():
            print("Door opened while MOTOR_CLOSE was active.")
            print("Stopping motor and waiting for the door to close again...")

            self.motor_write(
                MOTOR_STOP
            )

            self.set_locked_led(False)
            self.set_unlocked_led(True)

            self.start_door_wait(
                state=SmartLockState.waiting_for_door_close,
                target_value=DOOR_CLOSED_VALUE,
                timeout_sec=DOOR_CLOSE_TIMEOUT_SEC,
                stable_time_sec=0.50
```

```
            )

            return

        if now >= self.motor_deadline:
            self.run_next_close_step()
```

בחלק זה מומשה הפונקציה המרכזית שמנהלת את **מחזור הפתיחה והנעילה של המנעול בהתאם למצב הנוכחי של מכונת המצבים**.

בכל הפעלה של הפונקציה נבדק המצב שבו המערכת נמצאת, ובהתאם לכך מבוצעת הפעולה הנדרשת. כאשר המנוע נמצא בתהליך פתיחה, נבדק האם זמן פעולת המנוע הסתיים. לאחר מכן המערכת עוברת להמתנה לפתיחת הדלת באמצעות חיישן ה־MC-38

כאשר מזוהה שהדלת נפתחה, המערכת עוברת למצב ביניים קצר ולאחריו מתחילה להמתין לסגירת הדלת. אם הדלת נסגרת בזמן שהוגדר, מתחיל שלב ההשהיה שלפני הנעילה. אם הדלת נשארת פתוחה מעבר לזמן המרבי, מופעל מנגנון הטיפול בדלת שנשארה פתוחה.

לאחר שהדלת נסגרת ונשארת סגורה לאורך זמן ההשהיה, מתחיל רצף פעולת הנעילה של מנוע הצעד. גם בזמן פעולת הסגירה ממשיכה המערכת לבדוק את חיישן הדלת. אם הדלת נפתחת מחדש בזמן שפקודת MOTOR_CLOSE פעילה, המנוע נעצר מיד והמערכת חוזרת להמתין לסגירת הדלת.

המימוש בנוי בצורה לא־חוסמת, כלומר במקום לבצע את כל תהליך הפתיחה והנעילה ברצף אחד עם זמני המתנה ארוכים, הפונקציה בודקת בכל מחזור רק את המצב הנוכחי ומעדכנת אותו בהתאם. כך נשמרת שליטה רציפה במערכת וניתן להגיב לשינויים במצב הדלת גם במהלך תהליך הנעילה.

8.21. טעינת מודל גילוי הפנים ומודל זיהוי הפנים

```
def find_cascade_path(self):
    for path in CASCADE_PATHS:
        if os.path.exists(path):
            return path

    return None


def load_face_recognition_models(self):
    cascade_path = self.find_cascade_path()

    if cascade_path is None:
        raise FileNotFoundError("Haar Cascade file was not found.")

    if not os.path.exists(TRAINER_PATH):
        raise FileNotFoundError("LBPH trainer file was not found: " + TRAINER_PATH)

    print("Loading Haar Cascade face detector...")

    self.face_detector = cv2.CascadeClassifier(
        cascade_path
    )

    if self.face_detector.empty():
        raise RuntimeError("Failed to load Haar Cascade.")

    print("Face detector loaded successfully.")
    print("Loading LBPH face recognition model...")

    self.face_recognizer = cv2.face.LBPHFaceRecognizer_create()

    self.face_recognizer.read(
        TRAINER_PATH
    )

    print("LBPH model loaded successfully.")
```

בחלק זה בוצעה טעינת שני המודלים ששימשו בתהליך זיהוי הפנים: מסווג Haar Cascade לצורך גילוי מיקום הפנים בתמונה, ומודל LBPH שאומן מראש לצורך זיהוי זהות האדם.

תחילה התבצעה חיפוש אחר קובץ ה־ Haar Cascade מתוך רשימת הנתיבים שהוגדרה בתחילת התוכנית. הפונקציה find_cascade_path() עברה על הנתיבים האפשריים ובחרה את הנתיב הראשון שבו נמצא קובץ תקין. בדרך זו ניתן היה להתמודד עם מצב שבו קובץ ה־ Cascade נמצא במיקום שונה בהתאם לסביבת ההפעלה.

לאחר שנמצא הקובץ, נוצר אובייקט CascadeClassifier של OpenCV ששימש בהמשך לגילוי פנים בתמונות שצולמו באמצעות המצלמה. בנוסף בוצעה בדיקה שהמסווג נטען בצורה תקינה, ובמקרה של כשל הופסק תהליך האתחול ונוצרה הודעת שגיאה.

בשלב הבא נבדקה זמינותו של קובץ trainer.yml שבו נשמר מודל ה־ LBPH שאומן מראש על תמונות המשתמש המורשה. לאחר מכן נוצר אובייקט LBPHFaceRecognizer והמודל המאומן נטען לתוכו באמצעות הפונקציה read()

בסיום שלב זה היו זמינים בתוכנה שני הכלים הדרושים לתהליך: תחילה גילוי האזור בתמונה שבו נמצאות פנים באמצעות Haar Cascade ולאחר מכן השוואת הפנים שנמצאו למודל המאומן באמצעות LBPH לצורך קבלת מזהה המשתמש וערך ה־ Confidence

8.22. צילום תמונה מהמצלמה

```python
def capture_image(self, image_path=CAPTURED_IMAGE_PATH):
    print("Opening camera...")

    cap = None
    output_path = self.runtime_path(image_path)

    try:
        cap = cv2.VideoCapture(0)

        if not cap.isOpened():
            print("Camera 0 failed. Trying camera 1...")

            cap.release()
            cap = cv2.VideoCapture(1)

        if not cap.isOpened():
            print("Camera could not be opened.")
            return None

        time.sleep(
            0.5
        )

        ret, frame = cap.read()

        if not ret or frame is None:
            print("Image capture failed.")
            return None

        cv2.imwrite(
            str(output_path),
            frame
        )

        print(
            "Image captured successfully:",
            str(output_path)
        )

        return str(output_path)

    finally:
        if cap is not None:
            cap.release()
```

בחלק זה מומשה פעול ת צילום התמונה באמצעות המצלמה שחוברה למערכת. הפונקציה capture_image() פתחה את המצלמה באמצעות OpenCV צילמה תמונה בודדת ושמרה אותה בקובץ לצורך המשך תהליך גילוי וזיהוי הפנים.

תחילה נעשה ניסיון לפתוח את המצלמה באמצעות האינדקס 0 המייצג בדרך כלל את התקן המצלמה הראשון במערכת. אם פתיחת המצלמה נכשלה, בוצע ניסיון נוסף באמצעות האינדקס 1 מנגנון זה אפשר להתמודד עם מצב שבו מערכת ההפעלה הקצתה למצלמה מספר התקן שונה.

לאחר פתיחת המצלמה בוצעה השהיה קצרה של חצי שנייה, שאפשרה למצלמה להתייצב לפני צילום התמונה. לאחר מכן נקראה תמונה אחת באמצעות cap.read() אם פעולת הצילום נכשלה או שלא התקבלה תמונה תקינה, הפונקציה הסתיימה והחזירה None

כאשר הצילום הצליח, התמונה נשמרה באמצעות cv2.imwrite() בנתיב שהוגדר מראש. הפונקציה החזירה את הנתיב המלא של הקובץ שנשמר, כדי שניתן יהיה להעביר אותו ישירות לשלב הבא של גילוי וזיהוי הפנים.

בסיום הפעולה שוחררה המצלמה באמצעות cap.release() השחרור בוצע בתוך בלוק finally ולכן הוא התבצע גם במקרה שבו אירעה שגיאה או שהפונקציה הסתיימה מוקדם. כך נמנע מצב שבו המצלמה נשארת תפוסה ולא ניתן להשתמש בה מחדש בהמשך פעולת המערכת.

8.23. גילוי וזיהוי פנים

```
1. def detect_and_recognize_face(self, image_path):
2.     if image_path is None:
3.         print("Image does not exist.")
4.         return False, None, None, ""
5.
6.     image_file = Path(image_path)
7.
8.     if not image_file.exists():
9.         print("Image does not exist.")
10.        return False, None, None, ""
11.
12.    image = cv2.imread(
13.        str(image_file)
14.    )
15.
16.    if image is None:
17.        print("Failed to read image.")
18.        return False, None, None, ""
19.
20.    gray = cv2.cvtColor(
21.        image,
22.        cv2.COLOR_BGR2GRAY
23.    )
24.
25.    faces = self.face_detector.detectMultiScale(
26.        gray,
27.        scaleFactor=1.2,
28.        minNeighbors=5,
29.        minSize=(60, 60)
30.    )
31.
32.    print(
33.        "Number of faces detected:",
34.        len(faces)
35.    )
36.
37.    if len(faces) == 0:
38.        return False, None, None, str(image_file)
39.
40.    best_confidence = None
41.    best_id = None
42.
43.    for (x, y, w, h) in faces:
44.        face_roi = gray[y:y+h, x:x+w]
45.
46.        predicted_id, confidence = self.face_recognizer.predict(
```

```
47.             face_roi
48.         )
49.
50.         if best_confidence is None or confidence < best_confidence:
51.             best_confidence = confidence
52.             best_id = predicted_id
53.
54.         label = "ID: {} Conf: {:.2f}".format(
55.             predicted_id,
56.             confidence
57.         )
58.
59.         cv2.rectangle(
60.             image,
61.             (x, y),
62.             (x+w, y+h),
63.             (0, 255, 0),
64.             2
65.         )
66.
67.         cv2.putText(
68.             image,
69.             label,
70.             (x, y - 10),
71.             cv2.FONT_HERSHEY_SIMPLEX,
72.             0.7,
73.             (0, 255, 0),
74.             2
75.         )
76.
77.     result_path = self.runtime_path(
78.         RESULT_IMAGE_PATH
79.     )
80.
81.     cv2.imwrite(
82.         str(result_path),
83.         image
84.     )
85.
86.     print(
87.         "Best predicted ID:",
88.         best_id
89.     )
90.
91.     print(
92.         "Best confidence:",
93.         best_confidence
94.     )
95.
96.     print(
97.         "Result image saved:",
98.         str(result_path)
99.     )
100.
101.    return True, best_id, best_confidence, str(result_path)
102.
103.
104. def is_authorized_face(self, predicted_id, confidence):
105.     return predicted_id == AUTHORIZED_USER_ID and confidence <= CONFIDENCE_THRESHOLD
```

בחלק זה בוצע התהליך המרכזי של **גילוי הפנים וזיהוי המשתמש** מתוך התמונה שצולמה.

תחילה נבדק שקובץ התמונה קיים וניתן לקריאה. לאחר מכן התמונה נטענה באמצעות OpenCV והומרה לגווני אפור, מכיוון שגם Haar Cascade וגם מודל ה־ LBPH פועלים על מידע בהירויות ולא נדרשים לערוצי הצבע המלאים.

בשלב הבא הופעל מסווג Haar Cascade באמצעות הפונקציה detectMultiScale() שרקה את התמונה ואיתרה אזורים שבהם קיימת הסתברות להופעת פנים. לכל פנים שהתגלו התקבל מלבן המתאר את מיקומן וגודלן בתמונה.

עבור כל אזור פנים שנמצא, נחתך אזור הפנים מתוך התמונה בגווני אפור והועבר למודל ה־ LBPH המודל החזיר שני נתונים: מזהה המשתמש שנמצא כמתאים ביותר וערך Confidence המתאר את מידת ההתאמה למודל המאומן.

כאשר התגלו מספר פנים, נשמרה התוצאה בעלת ערך ה־ Confidence הנמוך ביותר, משום שב־ LBPH ערך נמוך יותר מצביע על התאמה טובה יותר.

בנוסף, סביב כל פנים שזוהו צויר מלבן על גבי התמונה, ונוספו מזהה המשתמש וערך ה־ Confidence התמונה המעובדת נשמרה בקובץ נפרד לצורך תיעוד או שליחה בהמשך.

בסיום התהליך הוחזרו ארבעה נתונים: האם נמצאו פנים, מזהה המשתמש המתאים ביותר, ערך ה־ Confidence והנתיב לתמונה המעובדת.

הפונקציה is_authorized_face() ביצעה את ההחלטה הסופית האם המשתמש מורשה. הזיהוי נחשב תקין רק כאשר מזהה המשתמש שהתקבל תאם ל־ AUTHORIZED_USER_ID וגם ערך ה־ Confidence היה קטן או שווה לסף שהוגדר ב־CONFIDENCE_THRESHOLD

8.24. קריאת לוח המקשים וטיפול בסיסמה

```
def read_keypad_status(self):
    return self.keypad_channel.read() & self.keypad_mask

def decode_key(self, key_code):
    key_map = {
        0x0: "0",
        0x1: "1",
        0x2: "2",
        0x3: "3",
        0x4: "4",
        0x5: "5",
        0x6: "6",
        0x7: "7",
        0x8: "8",
        0x9: "9",
        0xA: "A",
        0xB: "B",
        0xC: "C",
        0xD: "D",
        0xE: "*",
        0xF: "#"
    }

    return key_map.get(
        key_code,
        "?"
    )

def read_keypad_key(self):
    status = self.read_keypad_status()

    key_valid = (status >> 4) & 0x1
    key_code = status & 0xF

    current_time = time.monotonic()

    if key_valid == 1:
        key = self.decode_key(
            key_code
        )

        if key != self.last_keypad_key or (current_time - self.last_keypad_time) > KEYPAD_DEBOUNCE_SEC:
            self.last_keypad_key = key
            self.last_keypad_time = current_time

            return key

    else:
```

```
            self.last_keypad_key = None

        return None

    def handle_keypad(self):
        if self.is_access_cycle_active():
            return

        key = self.read_keypad_key()

        if key is None:
            return

        self.keypad_beep()

        print(
            "Key pressed:",
            key
        )

        if self.is_system_locked_out():
            remaining = self.get_lockout_remaining_time()

            print(
                "System is locked out. Remaining time:",
                remaining,
                "seconds."
            )

            self.denied_buzzer()

            return

        if key == "*":
            self.keypad_buffer = ""

            print("Keypad buffer cleared.")

            return

        if key == "#":
            entered_password = self.keypad_buffer
            self.keypad_buffer = ""

            print("Keypad password submitted.")

            if entered_password == self.password:
                self.handle_access_granted(
                    source="KEYPAD"
                )

            else:
                self.handle_access_denied(
                    source="KEYPAD"
                )

            return

        if key.isdigit():
            self.keypad_buffer += key

            print(
                "Current keypad buffer length:",
                len(self.keypad_buffer)
            )

            if len(self.keypad_buffer) > 8:
                self.keypad_buffer = ""

                print("Keypad buffer cleared because it was too long.")

            return

        print(
```

```
123.         "Unsupported key ignored:",
124.         key
125.     )
```

בחלק זה מומש מנגנון קריאת לוח המקשים 4×4 והטיפול בסיסמה שהוזנה על ידי המשתמש.

תחילה נקרא ערך הסטטוס שמתקבל מבקר לוח המקשים בצד החומרה. מתוך ערך זה מופרדים שני חלקים: ביט המציין האם זוהתה לחיצה תקפה, וקוד בן ארבעה ביטים המייצג את המקש שנלחץ. קוד המקש מתורגם באמצעות טבלת key_map לתו המתאים, הכולל את הספרות 0–9 המקשים A–D, הכוכבית * והסולמית #

כדי למנוע מצב שבו לחיצה אחת מזוהה מספר פעמים, הוגדר מנגנון Debounce התוכנית שומרת את המקש האחרון ואת זמן זיהויו, ומאפשרת לקבל מחדש את אותו מקש רק לאחר שעבר פרק הזמן שהוגדר ב־ KEYPAD_DEBOUNCE_SEC

לאחר זיהוי מקש תקין מופעל צפצוף קצר לצורך משוב למשתמש. אם המערכת נמצאת במצב Lockout הקלט אינו משמש לבדיקת סיסמה ונשמע חיווי של גישה שנדחתה.

הספרות שנלחצו נשמרות באופן מצטבר במשתנה keypad_buffer לחיצה על * מאפסת את הקלט שהוזן ומאפשרת להתחיל מחדש. לחיצה על # משמשת כאישור לסיום הזנת הסיסמה. בשלב זה המחרוזת שהצטברה מושווית לסיסמה שהוגדרה במערכת.

אם הסיסמה נכונה, מופעל מנגנון אישור הגישה ומתחיל מחזור פתיחת המנעול. אם הסיסמה שגויה, מופעל מנגנון דחיית הגישה ונרשם ניסיון כושל. בנוסף הוגבלה מחרוזת הקלט לאורך מרבי של שמונה ספרות, כדי למנוע הצטברות בלתי מוגבלת של נתונים.

8.25. איפוס מנגנון האבטחה ובדיקת מצב Lockout

```
 1. def reset_security_state(self):
 2.     self.failed_attempts = 0
 3.     self.failed_face_recognition_count = 0
 4.     self.lockout_until = 0
 5.
 6.     print("Security state reset.")
 7.
 8.     print(
 9.         "Failed attempts:",
10.         self.failed_attempts
11.     )
12.
13.     print(
14.         "Failed face recognition count:",
15.         self.failed_face_recognition_count
16.     )
17.
18.
19. def is_system_locked_out(self):
20.     if self.lockout_until == 0:
21.         return False
22.
23.     if time.monotonic() < self.lockout_until:
24.         return True
25.
26.     print("Lockout expired. System returned to normal mode.")
27.
28.     self.failed_attempts = 0
29.     self.failed_face_recognition_count = 0
30.     self.lockout_until = 0
31.
32.     return False
33.
34.
35. def get_lockout_remaining_time(self):
36.     if self.lockout_until == 0:
```

```
        return 0

    remaining = int(
        self.lockout_until - time.monotonic()
    )

    return max(
        0,
        remaining
    )
```

בחלק זה הוגדרו הפונקציות שאחראיות על ניהול מצב האבטחה של המערכת ועל מנגנון ה־ Lockout

הפונקציה reset_security_state() שימשה לאיפוס כל מוני האבטחה לאחר גישה מוצלחת. מספר ניסיונות הגישה הכושלים ומספר אירועי זיהוי הפנים הכושלים אופסו לערך אפס, ובמקביל בוטל כל Lockout שהיה פעיל. פעולה זו אפשרה להתחיל מחדש את מנגנון האבטחה לאחר שהמערכת זיהתה משתמש מורשה.

הפונקציה is_system_locked_out() בדקה האם המערכת נמצאת כרגע במצב Lockout זמן סיום ה־ Lockout נשמר במשתנה lockout_until ונקבע באמצעות time.monotonic(). כל עוד הזמן הנוכחי קטן מזמן הסיום שהוגדר, הפונקציה מחזירה True והמערכת אינה מאפשרת ניסיון גישה חדש.

כאשר זמן ה־ Lockout מסתיים, מוני ניסיונות הגישה וזיהוי הפנים מאופסים והמערכת חוזרת באופן אוטומטי לפעולה רגילה.

הפונקציה get_lockout_remaining_time() מחשבת את מספר השניות שנותרו עד לסיום ה־ Lockout ערך זה שימש להצגת מידע בזמן שנעשה ניסיון גישה כאשר המערכת עדיין נעולה זמנית.

באמצעות מנגנון זה ניתן היה לחסום זמנית ניסיונות גישה נוספים לאחר מספר ניסיונות כושלים, ובמקביל להחזיר את המערכת לפעולה רגילה באופן אוטומטי לאחר סיום פרק הזמן שהוגדר.

8.26. **טיפול בניסיון גישה כושל והפעלת** Lockout

```
def register_failed_attempt(self, source="UNKNOWN", confidence="", image_path=""):
    if self.is_system_locked_out():
        remaining = self.get_lockout_remaining_time()

        print("Access attempt ignored. System is locked out.")

        print(
            "Remaining lockout time:",
            remaining,
            "seconds."
        )

        self.denied_buzzer()

        self.write_log(
            source=source,
            result="IGNORED_LOCKOUT",
            confidence=confidence,
            system_state="LOCKOUT",
            image_path=image_path
        )

        return

    self.failed_attempts += 1

    print("Access denied.")

    print(
        "Source:",
        source
```

```
        )

        print(
            "Failed attempts:",
            self.failed_attempts,
            "/",
            MAX_FAILED_ATTEMPTS
        )

        self.denied_buzzer()

        self.write_log(
            source=source,
            result="ACCESS_DENIED",
            confidence=confidence,
            system_state="LOCKED",
            image_path=image_path
        )

        self.send_telegram_message(
            "Smart Lock Alert\n"
            "Access denied\n"
            "Source: {}\n"
            "Failed attempts: {}/{}\n"
            "Time: {}".format(
                source,
                self.failed_attempts,
                MAX_FAILED_ATTEMPTS,
                now_string()
            )
        )

        if image_path and source != "FACE":
            self.send_telegram_photo(
                image_path,
                "Access denied - source: {}".format(source)
            )

        if self.failed_attempts >= MAX_FAILED_ATTEMPTS:
            print("Maximum failed attempts reached.")

            if source == "KEYPAD":
                print("Capturing image after 3 wrong keypad attempts...")

                captured_keypad_image = self.capture_image(
                    KEYPAD_INTRUDER_IMAGE_PATH
                )

                if captured_keypad_image is not None:
                    print(
                        "Keypad intruder image captured:",
                        captured_keypad_image
                    )

                    self.write_log(
                        source="KEYPAD",
                        result="KEYPAD_INTRUDER_PHOTO_CAPTURED",
                        confidence="",
                        system_state="LOCKED",
                        image_path=captured_keypad_image
                    )

                    self.send_telegram_photo(
                        captured_keypad_image,
                        "Smart Lock Alert\n"
                        "3 wrong keypad attempts detected\n"
                        "Intruder photo captured\n"
                        "Time: {}".format(now_string())
                    )

                else:
                    print("Failed to capture keypad intruder image.")

                    self.send_telegram_message(
```

```
                    "Smart Lock Alert\n"
                    "3 wrong keypad attempts detected\n"
                    "Photo capture failed\n"
                    "Time: {}".format(now_string())
                )

        self.lockout_until = time.monotonic() + LOCKOUT_TIME_SEC

        print("Security lockout activated.")

        print(
            "System locked for",
            LOCKOUT_TIME_SEC,
            "seconds."
        )

        self.lockout_buzzer()

        self.write_log(
            source="SYSTEM",
            result="LOCKOUT",
            confidence="",
            system_state="LOCKOUT",
            image_path=image_path
        )

        self.send_telegram_message(
            "Smart Lock Security Lockout\n"
            "The system was locked for {} seconds\n"
            "Reason: {}\n"
            "Time: {}".format(
                LOCKOUT_TIME_SEC,
                source,
                now_string()
            )
        )
```

בחלק זה מומש מנגנון הטיפול בניסיון גישה כושל. בכל פעם שמתקבלת גישה שאינה מאושרת, נבדק תחילה האם המערכת כבר נמצאת במצב Lockout. אם כן, ניסיון הגישה אינו נבדק מחדש, נשמע חיווי של דחייה והאירוע נשמר בקובץ הלוג.

כאשר המערכת אינה במצב Lockout מספר ניסיונות הגישה הכושלים גדל באחד. לאחר מכן מופעל חיווי קולי של דחיית גישה, האירוע נשמר בקובץ ה־CSV ונשלחת הודעת Telegram הכוללת את מקור הניסיון, מספר הניסיונות הכושלים והזמן שבו התרחש האירוע.

אם קיים קובץ תמונה הקשור לניסיון הגישה, ניתן לשלוח גם אותו באמצעות Telegram במקרה של זיהוי פנים התמונה אינה נשלחת בשלב זה, מכיוון שמנגנון זיהוי הפנים מטפל בנפרד בשליחת תמונה לאחר צבירת מספר אירועים כושלים.

כאשר מספר ניסיונות הגישה הכושלים מגיע לערך שהוגדר ב־ MAX_FAILED_ATTEMPTS מופעל מצב Lockout. אם מקור הניסיונות הוא לוח המקשים, המערכת מצלמת תמונה של האדם שנמצא מול הדלת לאחר שלוש סיסמאות שגויות, שומרת את האירוע בלוג ושולחת את התמונה באמצעות Telegram. אם הצילום נכשל, נשלחת במקום זאת הודעת טקסט המציינת שהצילום לא הצליח.

לאחר מכן נקבע זמן סיום ה־ Lockout בהתאם ל־ LOCKOUT_TIME_SEC מופעל חיווי קולי ייעודי, האירוע נשמר בלוג ונשלחת הודעה נוספת ל־ Telegram על כך שהמערכת ננעלה זמנית.

כך שולב במערכת מנגנון הגנה מפני ניסיונות גישה חוזרים, הכולל תיעוד, חיווי, צילום והתראה מרחוק

8.27. טיפול בכשלי זיהוי פנים

```
def register_failed_face_recognition(self, confidence="", image_path=""):
    if self.is_system_locked_out():
        remaining = self.get_lockout_remaining_time()

        print("Face recognition attempt ignored. System is locked out.")

        print(
            "Remaining lockout time:",
            remaining,
            "seconds."
        )

        self.denied_buzzer()

        self.write_log(
            source="FACE",
            result="IGNORED_LOCKOUT",
            confidence=confidence,
            system_state="LOCKOUT",
            image_path=image_path
        )

        return

    self.failed_face_recognition_count += 1

    print("Face recognition failed.")

    print(
        "Face failed count:",
        self.failed_face_recognition_count,
        "/",
        MAX_FAILED_FACE_RECOGNITION
    )

    self.denied_buzzer()

    self.write_log(
        source="FACE",
        result="FACE_RECOGNITION_FAILED",
        confidence=confidence,
        system_state="LOCKED",
        image_path=image_path
    )

    if self.failed_face_recognition_count >= MAX_FAILED_FACE_RECOGNITION:
        print("Three failed face recognition events counted as one failed access attempt.")
        print("Sending Telegram alert only now.")

        if image_path:
            self.send_telegram_photo(
                image_path,
                "Smart Lock Alert\n"
                "3 failed face recognition events detected\n"
                "This is counted as 1 failed access attempt\n"
                "Time: {}".format(now_string())
            )

        else:
            self.send_telegram_message(
                "Smart Lock Alert\n"
                "3 failed face recognition events detected\n"
                "No image available\n"
                "This is counted as 1 failed access attempt\n"
                "Time: {}".format(now_string())
            )

        self.failed_face_recognition_count = 0

        self.register_failed_attempt(
```

```
71.             source="FACE",
72.             confidence=confidence,
73.             image_path=image_path
74.         )
```

בחלק זה מומש מנגנון נפרד לניהול כשלי זיהוי פנים. בניגוד להזנת סיסמה שגויה בלוח המקשים, כל כשל בודד בזיהוי פנים אינו נספר מיד כניסיון גישה כושל מלא.

בכל פעם שזיהוי הפנים נכשל, מונה failed_face_recognition_count גדל באחד, מופעל חיווי קולי של דחיית גישה והאירוע נשמר בקובץ הלוג. אם המערכת נמצאת כבר במצב Lockout ניסיון הזיהוי אינו מטופל ונשמר רק כאירוע שנדחה עקב מצב הנעילה הזמנית.

כאשר מתקבלים שלושה אירועי זיהוי פנים כושלים, הם נחשבים יחד לניסיון גישה כושל אחד. בשלב זה נשלחת התראה באמצעות Telegram אם קיימת תמונה מהאירוע, היא נשלחת יחד עם ההתראה ; אם לא קיימת תמונה, נשלחת הודעת טקסט בלבד.

לאחר מכן מונה כשלי זיהוי הפנים מתאפס, ונקראת הפונקציה register_failed_attempt(אשר מוסיפה ניסיון כושל אחד למונה האבטחה הכללי. באופן זה נשמרה הפרדה בין תקלות או זיהויים לא מוצלחים בודדים לבין מספר כשלי זיהוי חוזרים שעשויים להצביע על ניסיון גישה לא מורשה.

8.28. טיפול בגישה מוצלחת

```
 1. def register_successful_access(self, source="UNKNOWN", confidence="", image_path=""):
 2.     print("Access granted.")
 3.
 4.     print(
 5.         "Source:",
 6.         source
 7.     )
 8.
 9.     self.reset_security_state()
10.
11.     self.success_buzzer()
12.
13.     self.write_log(
14.         source=source,
15.         result="ACCESS_GRANTED",
16.         confidence=confidence,
17.         system_state="UNLOCKED",
18.         image_path=image_path
19.     )
20.
21.     self.send_telegram_message(
22.         "Smart Lock\n"
23.         "Access granted\n"
24.         "Source: {}\n"
25.         "Time: {}".format(
26.             source,
27.             now_string()
28.         )
29.     )
```

בחלק זה טופל אירוע שבו התקבלה גישה מאושרת, בין אם באמצעות זיהוי פנים ובין אם באמצעות לוח המקשים.

בעת קבלת הרשאה אופסו מוני ניסיונות הגישה הכושלים ומונה כשלי זיהוי הפנים, כך שכל ניסיון מוצלח התחיל מחזור אבטחה חדש. לאחר מכן הופעל חיווי קולי של הצלחה, האירוע נשמר בקובץ ה־ CSV ונשלחה הודעה באמצעות Telegram הכוללת את מקור הגישה ואת זמן האירוע.

הפונקציה אינה מבצעת בעצמה את פתיחת המנעול, אלא מטפלת ברישום ובפעולות הנלוות לאישור הגישה. פתיחת המנעול מתבצעת בשלב הבא באמצעות פונקציות הטיפול בגישה.

8.29. ניתוב תוצאת הגישה

```
def handle_access_granted(self, source="UNKNOWN", confidence="", image_path=""):
    if self.is_access_cycle_active():
        print("Access cycle is already active. New access grant ignored.")
        return

    self.register_successful_access(
        source=source,
        confidence=confidence,
        image_path=image_path
    )

    self.unlock_then_auto_close()


def handle_access_denied(self, source="UNKNOWN", confidence="", image_path=""):
    self.register_failed_attempt(
        source=source,
        confidence=confidence,
        image_path=image_path
    )
```

בחלק זה רוכזה ההחלטה מה לעשות לאחר קבלת תוצאת אימות המשתמש.

כאשר הגישה מאושרת, נבדק תחילה שאין כבר מחזור פתיחה או נעילה פעיל. לאחר מכן נרשמת הגישה המוצלחת ומופעל מחזור פתיחת המנעול באמצעות unlock_then_auto_close()

כאשר הגישה נדחית, האירוע מועבר ישירות למנגנון register_failed_attempt() שמטפל בעדכון מונה הניסיונות, תיעוד האירוע, ההתראות ומנגנון ה־Lockout.

השלב הבא הוא אחד החלקים המרכזיים בתוכנה: **הטיפול באירוע PIR** שבו משולבים חיישן התנועה, חיישן האור, הממסר, המצלמה וגילוי וזיהוי הפנים.

8.30. טיפול באירוע תנועה ושילוב חיישן האור, המצלמה וזיהוי הפנים

```
def handle_motion_event(self):
    if self.is_access_cycle_active():
        return

    print("Motion detected by PIR.")

    if self.is_system_locked_out():
        remaining = self.get_lockout_remaining_time()

        print("Motion detected, but system is locked out.")

        print(
            "Remaining lockout time:",
            remaining,
            "seconds."
        )

        self.denied_buzzer()

        self.write_log(
            source="PIR",
            result="IGNORED_LOCKOUT",
            confidence="",
            system_state="LOCKOUT",
            image_path=""
        )

        return

```

```
30.     relay_was_enabled = False
31.     image_path = None
32.
33.     self.set_motion_led(True)
34.
35.     try:
36.         if self.is_dark():
37.             print("Dark environment detected. Relay ON.")
38.
39.             self.set_relay(True)
40.             relay_was_enabled = True
41.
42.             time.sleep(
43.                 RELAY_PRE_LIGHT_SEC
44.             )
45.
46.         else:
47.             print("Light environment detected. Relay remains OFF.")
48.
49.         image_path = self.capture_image(
50.             CAPTURED_IMAGE_PATH
51.         )
52.
53.     finally:
54.         if relay_was_enabled:
55.             self.set_relay(False)
56.             print("Relay OFF.")
57.
58.         self.set_motion_led(False)
59.
60.     if image_path is None:
61.         print("No image was captured.")
62.         print("This is counted as one failed face recognition event.")
63.
64.         self.register_failed_face_recognition(
65.             confidence="",
66.             image_path=""
67.         )
68.
69.         return
70.
71.     face_found, predicted_id, confidence, result_image_path = self.detect_and_recognize_face(
72.         image_path
73.     )
74.
75.     if not face_found:
76.         print("No face detected in the captured image.")
77.         print("This is counted as one failed face recognition event.")
78.
79.         self.register_failed_face_recognition(
80.             confidence="",
81.             image_path=image_path
82.         )
83.
84.         return
85.
86.     if self.is_authorized_face(
87.         predicted_id,
88.         confidence
89.     ):
90.         self.handle_access_granted(
91.             source="FACE",
92.             confidence="{:.2f}".format(confidence),
93.             image_path=result_image_path
94.         )
95.
96.     else:
97.         print("Unknown or unauthorized face detected.")
98.         print("This is counted as one failed face recognition event.")
99.
100.        if result_image_path and Path(result_image_path).exists():
101.            try:
102.                image = cv2.imread(
103.                    result_image_path
```

```
                )

                unknown_path = self.runtime_path(
                    UNKNOWN_IMAGE_PATH
                )

                cv2.imwrite(
                    str(unknown_path),
                    image
                )

                alert_image_path = str(unknown_path)

            except Exception:
                alert_image_path = result_image_path

        else:
            alert_image_path = image_path

        self.register_failed_face_recognition(
            confidence="{:.2f}".format(confidence),
            image_path=alert_image_path
        )
```

בחלק זה רוכזו מספר תתי־מערכות שפועלות בעקבות זיהוי תנועה על ידי חיישן ה־ PIR

כאשר מזוהה תנועה, התוכנית בודקת תחילה שאין כבר מחזור פתיחה או נעילה פעיל, וכן שהמערכת אינה נמצאת במצב Lockout אם אחד מהמצבים הללו פעיל, אירוע התנועה אינו מפעיל תהליך זיהוי חדש.

כאשר ניתן להמשיך, נדלקת נורת חיווי התנועה ונבדק מצב חיישן האור. אם מזוהה סביבה חשוכה, הממסר מופעל לצורך הדלקת התאורה, ומתבצעת השהיה קצרה לפני הצילום כדי לאפשר לתאורה להתייצב. לאחר מכן מתבצע צילום באמצעות המצלמה.

לאחר הצילום הממסר ונורת התנועה מוחזרים למצב כבוי, גם במקרה שבו אירעה שגיאה במהלך התהליך. אם הצילום נכשל, האירוע נרשם ככשל בזיהוי פנים.

כאשר התקבלה תמונה תקינה, היא מועברת לפונקציה detect_and_recognize_face() אם לא נמצאו פנים, האירוע נספר ככשל זיהוי. אם נמצאו פנים, נבדק האם המזהה וערך ה־ Confidence תואמים למשתמש המורשה.

במקרה של זיהוי תקין, מופעל מנגנון אישור הגישה ומתחיל מחזור פתיחת המנעול. במקרה של אדם לא מורשה, נשמרת תמונה ייעודית של האירוע והמערכת מעבירה אותו למנגנון ניהול כשלי זיהוי הפנים.

באמצעות פונקציה זו חוברו יחד חיישן ה־ PIR חיישן האור, הממסר, המצלמה, Haar Cascade, מודל ה־ LBPH ומנגנון האבטחה לרצף פעולה אחד.

```
def run(self):
    print("================================================")
    print("Starting Final Smart Lock System")
    print("================================================")

    self.init_log_file()
    self.load_system_overlay()
    self.load_face_recognition_models()

    self.init_outputs()
    self.reset_motor_controller()

    print("System is ready.")
    print("Waiting for PIR motion or keypad input.")
    print("Use keypad password: configured password followed by #")
    print("Keypad rule: wrong keypad code = 1 failed attempt.")
    print("Keypad rule: 3 wrong codes = intruder photo + Telegram + lockout.")
    print("Face rule: 3 failed face recognition events = 1 failed attempt + Telegram.")
    print("Door sensor: MC-38 on AR2; 0 = CLOSED, 1 = OPEN.")
    print("Automatic closing: after the door opens and closes.")

    print(
        "Initial door state:",
        self.get_door_state()
    )

    print(
        "Telegram alerts:",
        "ON" if self.telegram_enabled else "OFF"
    )

    print("Stop the cell manually when finished.")
    print("================================================")

    self.send_telegram_message(
        "Smart Lock System Started\nTime: {}".format(
            now_string()
        )
    )

    last_motion_time = 0

    while True:
        try:
            self.update_access_cycle()

            if not self.is_access_cycle_active():
                self.handle_keypad()

                pir_value = self.read_pir()

                if pir_value == 1:
                    current_time = time.monotonic()

                    if current_time - last_motion_time >= MOTION_COOLDOWN_SEC:
                        last_motion_time = current_time

                        self.handle_motion_event()

                    else:
                        time.sleep(
                            0.1
                        )

            time.sleep(
                0.05
            )

        except KeyboardInterrupt:
            print("System stopped manually by user.")

```

```
72.             self.safe_shutdown()
73.
74.             break
75.
76.         except Exception as e:
77.             print(
78.                 "System error:",
79.                 e
80.             )
81.
82.             try:
83.                 self.motor_write(
84.                     MOTOR_STOP
85.                 )
86.
87.             except Exception as stop_error:
88.                 print(
89.                     "Motor stop warning:",
90.                     stop_error
91.                 )
92.
93.             self.write_log(
94.                 source="SYSTEM",
95.                 result="ERROR",
96.                 confidence="",
97.                 system_state="ERROR",
98.                 image_path=""
99.             )
100.
101.            self.send_telegram_message(
102.                "Smart Lock System Error\n{}\nTime: {}".format(
103.                    str(e),
104.                    now_string()
105.                )
106.            )
107.
108.            time.sleep(
109.                1
110.            )
```

בחלק זה מומשה הלולאה הראשית של מערכת המנעול החכם, אשר מרכזת את פעולת כלל תתי־המערכות ומנהלת את המערכת באופן רציף.

בתחילת הפונקציה מתבצע אתחול של קובץ הלוג, טעינת ה־ Overlay טעינת מודלי גילוי וזיהוי הפנים, אתחול רכיבי הפלט ואיפוס בקר מנוע הצעד. לאחר שכל שלבי האתחול מסתיימים בהצלחה, המערכת עוברת למצב עבודה וממתינה לאירועים מחיישן ה־ PIR או מלוח המקשים.

לאחר האתחול מופעלת לולאה אינסופית, שבה מתבצעות שוב ושוב פעולות הבקרה המרכזיות. בכל מחזור נקראת תחילה הפונקציה update_access_cycle(), אשר מעדכנת את מכונת המצבים ומטפלת בתהליך הפתיחה, ההמתנה לדלת והנעילה.

כאשר לא מתבצע מחזור פתיחה או נעילה פעיל, המערכת בודקת את לוח המקשים ואת חיישן ה־ PIR אם התקבלה לחיצה על מקש, היא מטופלת באמצעות handle_keypad() אם זוהתה תנועה, נבדק שעבר פרק הזמן שהוגדר ב־ MOTION_COOLDOWN_SEC ורק לאחר מכן מופעל תהליך הצילום וזיהוי הפנים באמצעות handle_motion_event()

בסיום כל מחזור קיימת השהיה קצרה של 0.05 שניות. השהיה זו מונעת מהלולאה לרוץ במהירות מיותרת ומאפשרת בדיקה מחזורית רציפה של מצב המערכת.

בנוסף הוגדר טיפול במצבי שגיאה. במקרה של עצירה ידנית באמצעות KeyboardInterrupt מתבצעת קריאה לפונקציה safe_shutdown() לצורך כיבוי בטוח. במקרה של חריגה אחרת, המנוע נעצר, האירוע נרשם בקובץ הלוג, נשלחת הודעת שגיאה באמצעות Telegram והמערכת ממתינה שנייה אחת לפני המשך הפעולה.

באמצעות לולאה זו חוברו למעשה כל תתי־המערכות לתהליך עבודה אחד רציף: חיישנים, לוח מקשים, זיהוי פנים, מנוע, חיישן הדלת, מנגנון האבטחה Telegram, ומערכת הלוגים.

8.32. כיבוי בטוח של המערכת

```
def safe_shutdown(self):
    print("Shutting down system safely...")

    try:
        self.motor_write(
            MOTOR_STOP
        )

        self.output_state = 0

        self.write_outputs()

    except Exception as e:
        print(
            "Shutdown warning:",
            e
        )

    self.write_log(
        source="SYSTEM",
        result="SHUTDOWN",
        confidence="",
        system_state="SHUTDOWN",
        image_path=""
    )

    self.send_telegram_message(
        "Smart Lock System Stopped\nTime: {}".format(
            now_string()
        )
    )

    print("System shutdown completed.")
```

בחלק זה מומש מנגנון הכיבוי הבטוח של מערכת המנעול החכם.

בעת סיום העבודה נשלחת תחילה פקודת MOTOR_STOP לבקר מנוע הצעד, כדי לוודא שהמנוע אינו נשאר פעיל. לאחר מכן מאופס משתנה output_state ונכתב לערוץ הפלט, כך שנורות החיווי, הממסר והזמזם נכבים.

פעולות הכיבוי מבוצעות בתוך בלוק try, ולכן גם במקרה שבו מתרחשת תקלה במהלך עצירת החומרה, התוכנית ממשיכה לבצע את פעולות הסיום ולא נעצרת באופן לא מבוקר.

לאחר כיבוי רכיבי החומרה נשמר בקובץ הלוג אירוע מסוג SHUTDOWN ונשלחת באמצעות Telegram הודעה המציינת שמערכת המנעול הופסקה ואת הזמן שבו התבצעה העצירה.

באמצעות פונקציה זו הובטח כי בעת עצירה ידנית של התוכנית המערכת תעבור למצב בטוח ומוגדר, במקום להשאיר רכיבי פלט או את מנוע הצעד במצב פעיל.

8.33. פונקציית הפעלת המערכת

```
1. def run_smart_lock_system():
2.     system = SmartLockSystem()
3.     system.run()
```

בחלק זה הוגדרה פונקציית ההפעלה הראשית של המערכת.

תחילה נוצר אובייקט חדש מהמחלקה SmartLockSystem. יצירת האובייקט מפעילה את פונקציית __init__ שבה מאותחלים כל משתני המערכת, מכונת המצבים, מוני האבטחה, תור ה־Telegram ושאר המשתנים הדרושים לפעולה.

לאחר יצירת האובייקט נקראת הפונקציה run() אשר מבצעת את אתחול החומרה והתוכנה ומפעילה את הלולאה הראשית של מערכת המנעול החכם.

בגרסה הסופית שהותאמה ל־Jupyter Notebook הגדרת המחלקה והפונקציות אינה מפעילה את המערכת באופן אוטומטי. הפעלת המערכת מתבצעת בתא נפרד באמצעות:

```
1. run_smart_lock_system()
```

הפרדה זו אפשרה לטעון תחילה את כל הגדרות התוכנית ורק לאחר מכן להתחיל את פעולת המערכת באופן מבוקר מתוך סביבת Jupyter

8.34. סיכום מימוש התוכנה

מימוש התוכנה של מערכת המנעול החכם בוצע בשפת Python תוך שילוב בין ממשקי החומרה של כרטיס PYNQ-Z2 לבין רכיבי עיבוד התמונה, מנגנון האבטחה ומערכת ההתראות.

התוכנה נבנתה סביב המחלקה SmartLockSystem אשר ריכזה את כלל תתי־המערכות והמשתנים הדרושים להפעלת המנעול. בתחילת ההפעלה אותחלו נתיבי הקבצים, ממשקי ה־GPIO מודלי זיהוי הפנים, מוני האבטחה, מכונת המצבים ותור ה־Telegram

הגישה לרכיבי החומרה בוצעה באמצעות ממשקי AXI GPIO דרך ממשקים אלו נקראו חיישן התנועה PIR חיישן האור LM393 וחיישן הדלת MC-38 ובמקביל הופעלו נורות החיווי, הזמזם, הממסר ובקר מנוע הצעד.

מערכת זיהוי הפנים פעלה בשני שלבים. תחילה זוהו פנים בתמונה באמצעות Haar Cascade ולאחר מכן הועבר אזור הפנים למודל LBPH שאומן מראש. בהתאם למזהה שהתקבל ולערך ה־Confidence הוחלט האם מדובר במשתמש מורשה.

בנוסף לזיהוי הפנים שולב לוח מקשים 4×4 ששימש כאמצעי גישה חלופי. הספרות שהוזנו נשמרו בזיכרון זמני, ולחיצה על # אישרה את הסיסמה. סיסמה נכונה הפעילה את מנגנון פתיחת המנעול, ואילו סיסמה שגויה הגדילה את מונה ניסיונות הגישה הכושלים.

ניהול פתיחת ונעילת הדלת בוצע באמצעות מכונת מצבים לא־חוסמת. לאחר אישור גישה הופעל מנוע הצעד לפתיחת המנעול, ולאחר מכן המערכת המתינה לפתיחת הדלת ולסגירתה באמצעות חיישן ה־MC-38 רק לאחר שאושרה סגירת הדלת ונשמר מצב סגור במשך פרק הזמן שהוגדר, הופעל מנגנון הנעילה.

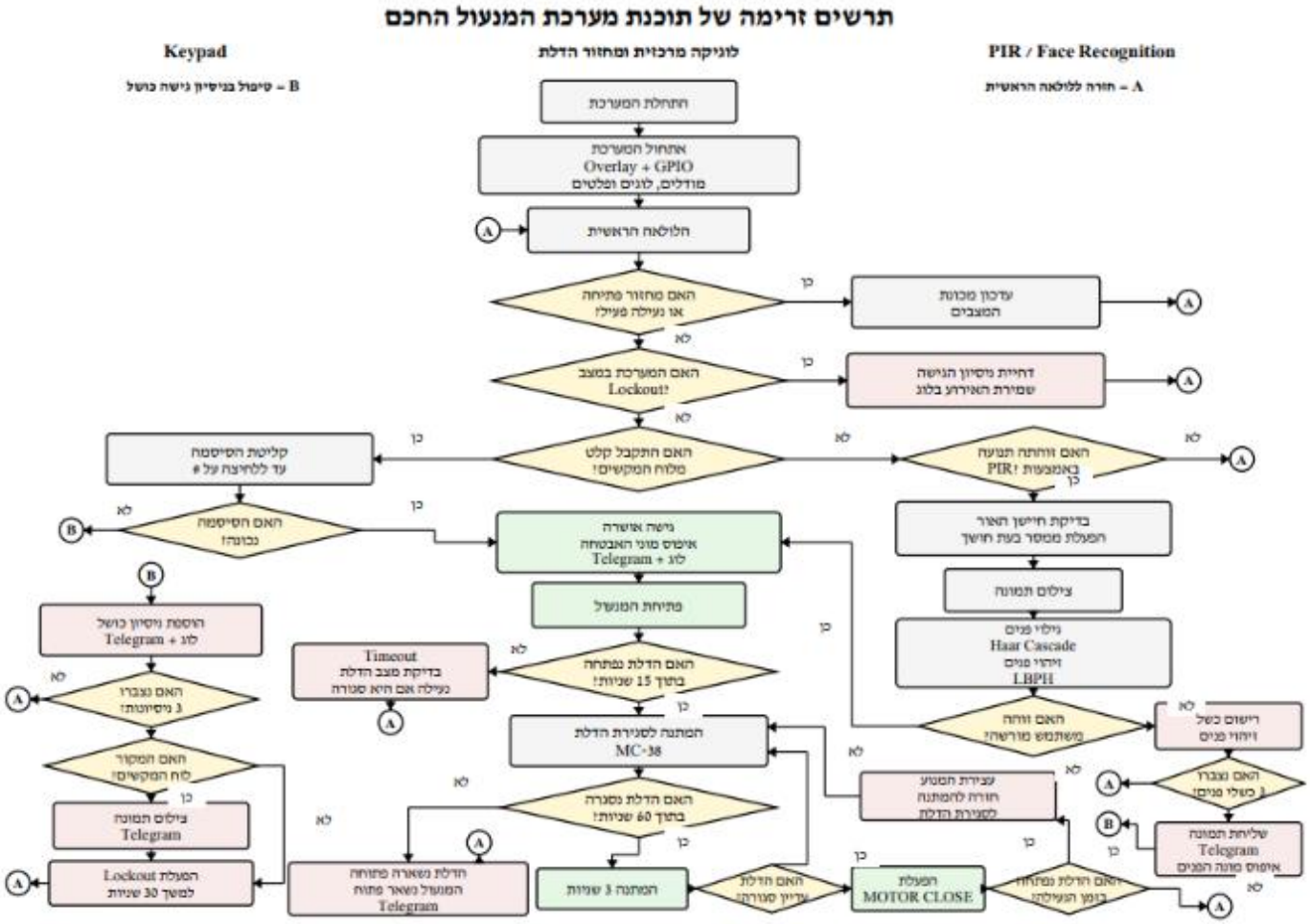
במקרה שבו הדלת נשארה פתוחה מעבר לזמן המרבי, המנעול לא נסגר ונשלחה התראה למשתמש. כך נמנעה הפעלת מנגנון הנעילה כאשר הדלת אינה נמצאת במצב מתאים.

מנגנון האבטחה כלל ניהול ניסיונות גישה כושלים, מונה נפרד לכשלי זיהוי פנים ומצב Lockout לאחר מספר ניסיונות שגויים הוגבלה הגישה למערכת למשך זמן מוגדר, ובמקרה של שלוש סיסמאות שגויות צולמה תמונה ונשלחה באמצעות Telegram

מערכת ההתראות פעלה באמצעות תור ו־ Thread נפרד, כך ששליחת הודעות ותמונות לא חסמה את הלולאה הראשית. בנוסף נשמרו אירועים מרכזיים בקובץ CSV שכלל את זמן האירוע, מקורו, תוצאתו, מצב המערכת ונתיב לתמונה כאשר הייתה זמינה.

הלולאה הראשית חיברה בין כל תתי־המערכות וביצעה באופן רציף קריאת חיישנים, טיפול בלוח המקשים, ניהול מחזור הפתיחה והנעילה, הפעלת זיהוי הפנים וטיפול במצבי שגיאה.

במבנה זה התקבלה תוכנה מודולרית ומסודרת, שבה כל תת־מערכת טופלה באמצעות פונקציות ייעודיות, בעוד שהלוגיקה הראשית ניהלה את רצף הפעולה ואת הקשר בין החומרה, עיבוד התמונה ומנגנוני האבטחה.



איור** 23 **תרשים זרימה של התוכנה

התרשים הציג את זרימת הפעולה הכללית של תוכנת מערכת המנעול החכם. לאחר אתחול המערכת והפעלת הלולאה הראשית נבדקו מצבי המערכת והתקבלו אירועי גישה מלוח המקשים או מחיישן ה־ PIR בהתאם למקור הגישה בוצע אימות באמצעות סיסמה או באמצעות גילוי וזיהוי פנים.

במקרה של גישה מאושרת הופעל מנגנון פתיחת המנעול, ולאחר מכן נוהל מחזור הדלת באמצעות חיישן ה־ MC-38 עד לסגירתה ונעילתה מחדש. במקרים של גישה שגויה הופעל מנגנון האבטחה שכלל ספירת ניסיונות, תיעוד אירועים, שליחת התראות באמצעות Telegram והפעלת מצב Lockout בהתאם לתנאים שהוגדרו.

9. ניסויים ובדיקות

מטרת הבדיקות הייתה לוודא את תקינות פעולתן של תתי־המערכות השונות ואת פעולת המערכת המשולבת בהתאם לדרישות שהוגדרו במהלך התכנון. הבדיקות בוצעו באופן הדרגתי, החל מבדיקת רכיבי החומרה והתוכנה בנפרד, ועד לבדיקת פעולת המערכת המלאה בתרחישי גישה שונים.

9.1. בדיקות וסימולציות בסביבת VIVADO

לצורך אימות החומרה שתוכננה ב־ VHDL בוצעו סימולציות בסביבת Vivado לפני הפעלת המערכת על גבי כרטיס PYNQ-Z2 מטרת הסימולציות הייתה לוודא כי מחלק התדר הפיק את תדר השעון הנדרש, כי רצפי הפאזות של מנוע הצעד התקבלו בסדר הנכון בשני כיווני הסיבוב, וכי הבקר הראשי הפעיל את המנוע בהתאם לפקודות הבקרה.

הבדיקות בוצעו תחילה עבור כל קומפוננטה בנפרד ולאחר מכן עבור המערכת המשולבת. בדרך זו ניתן היה לוודא את תקינות כל חלק באופן עצמאי לפני בדיקת פעולת בקר המנוע המלא.

9.1.1. בדיקת מחלק התדר

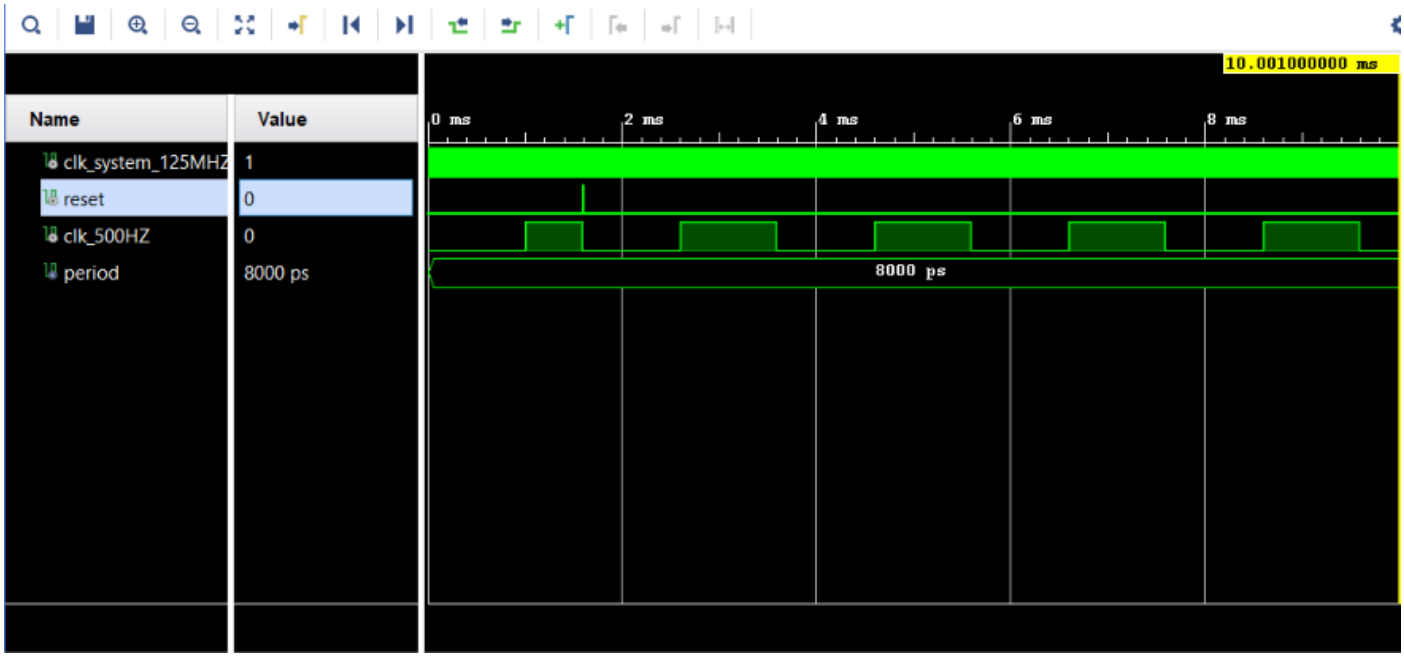
בסימולציה הראשונה נבדקה פעולת מחלק התדר שתוכנן להמיר את שעון המערכת בתדר של 125 MHz לשעון בתדר של 500 Hz אות השעון בכניסה הוגדר עם זמן מחזור של 8 ns המתאים לתדר של :

$$f = \frac{1}{8n} = 125MHz$$

בתוצאות הסימולציה התקבל באות clk_500HZ זמן מחזור של כ־2 ms תוצאה זו מתאימה לתדר:

$$f = \frac{1}{2m} = 500Hz$$

לפיכך נמצא כי מחלק התדר פעל בהתאם לתכנון והפיק את תדר השעון הדרוש להפעלת מכונת המצבים של מנוע הצעד.



איור 24 תוצאת סימולצייה של מחלק התדר

9.1.2. בדיקת סיבוב המנוע עם כיוון השעון

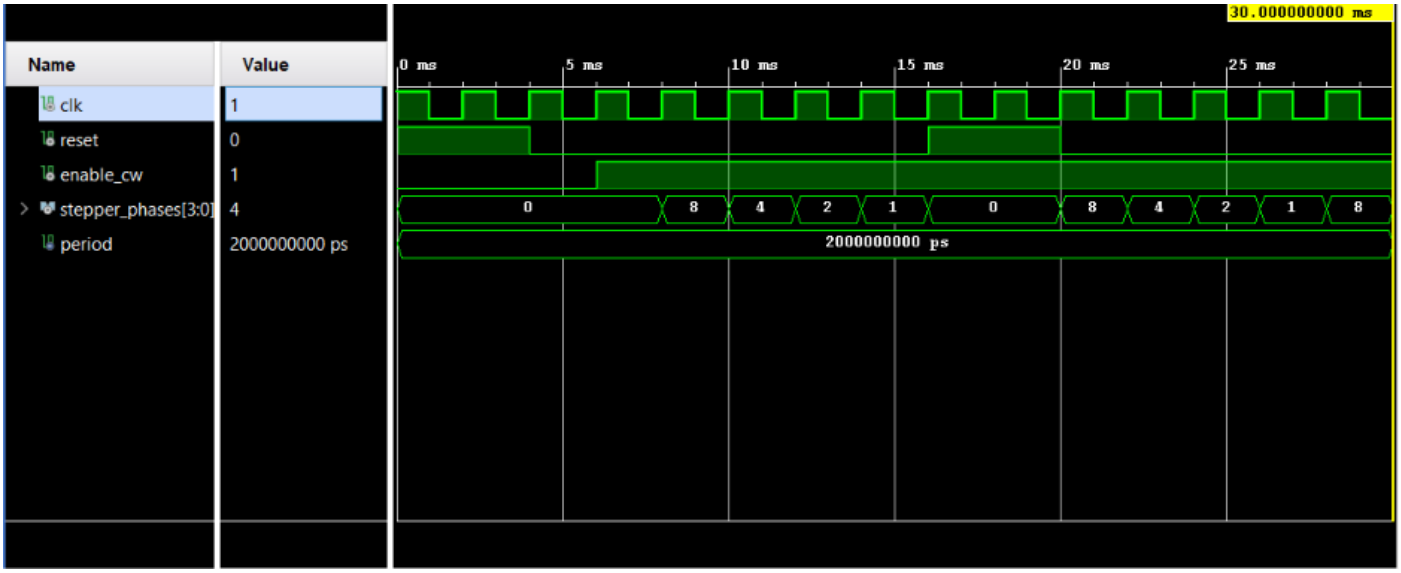
בסימולציה זו נבדקה מכונת המצבים האחראית להפעלת מנוע הצעד עם כיוון השעון. לאחר הפעלת האות enable_cw התקבל במוצא stepper_phases רצף הערכים:

$$1000 \rightarrow 0100 \rightarrow 0010 \rightarrow 0001$$

בחלון הסימולציה הוצגו ערכים אלו בצורה הקסדצימלית כרצף:

$$8 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

הרצף חזר על עצמו כל עוד אות ההפעלה נשאר פעיל. תוצאה זו אישרה כי הפאזות הופעלו בסדר שנקבע עבור סיבוב המנוע עם כיוון השעון.



איור 25 תוצאת סימולציה של המנוע בסיבוב עם כיוון השעון

9.1.3. בדיקת סיבוב המנוע נגד כיוון השעון

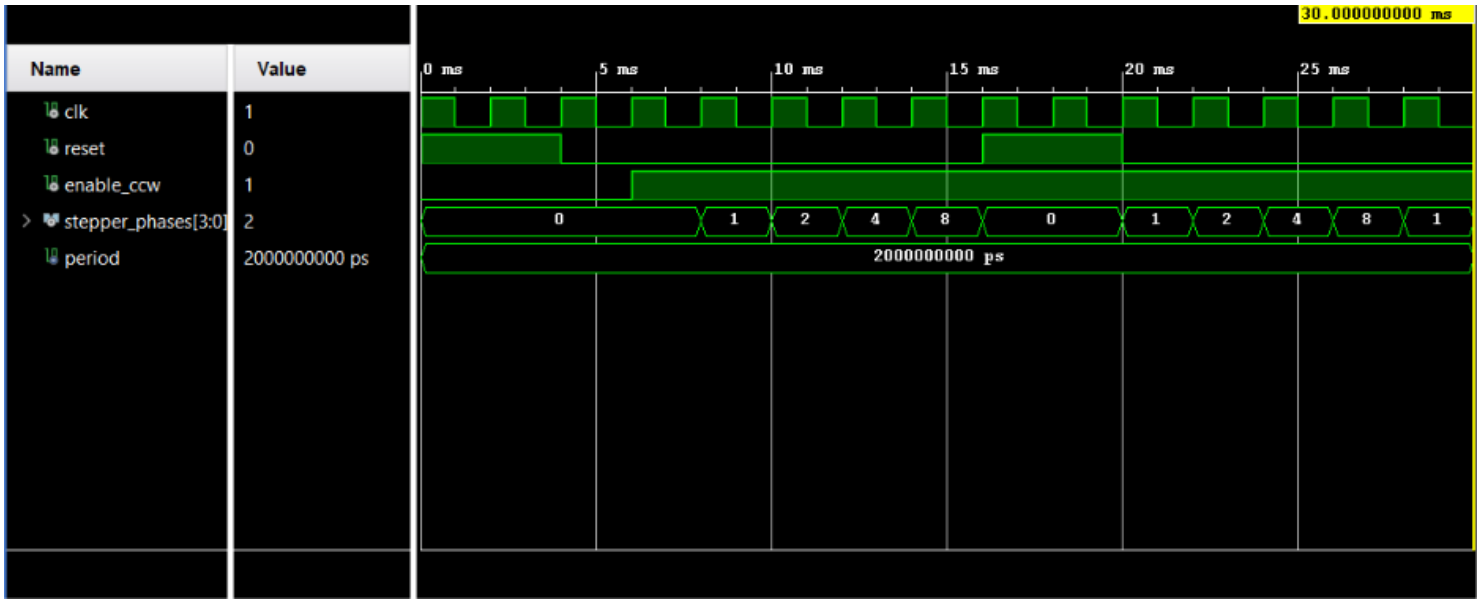
בסימולציה זו נבדקה מכונת המצבים האחראית לסיבוב בכיוון ההפוך. כאשר האות enable_ccw הופעל, התקבל במוצא stepper_phases הרצף:

$$0001 \rightarrow 0010 \rightarrow 0100 \rightarrow 1000$$

בייצוג ההקסדצימלי של Vivado התקבל:

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 8$$

רצף זה הוא הרצף ההפוך לזה שהתקבל בבדיקת הסיבוב עם כיוון השעון, ולכן אושרה פעולת הבקר בשני כיווני הסיבוב.



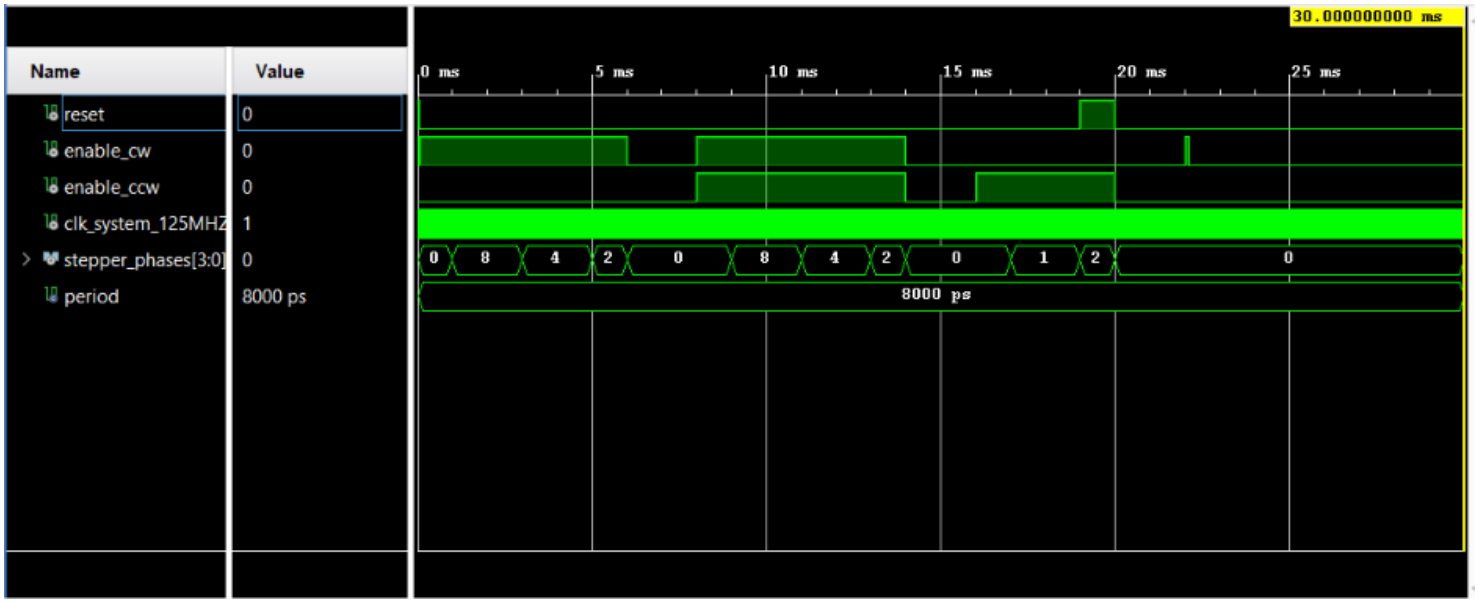
איור 26 תוצאות סימולציה של המנוע עם סיבוב נגד כיוון השעון

9.1.4. בדיקת מנוע הצעד באופן מלא

לאחר בדיקת תתי־המערכות בנפרד נבדק בקר מנוע הצעד המלא. במסגרת סימולציה זו הופעלו אותות enable_cw ו־ enable_ccw בזמנים שונים, ונבדקה תגובת המוצא stepper_phases

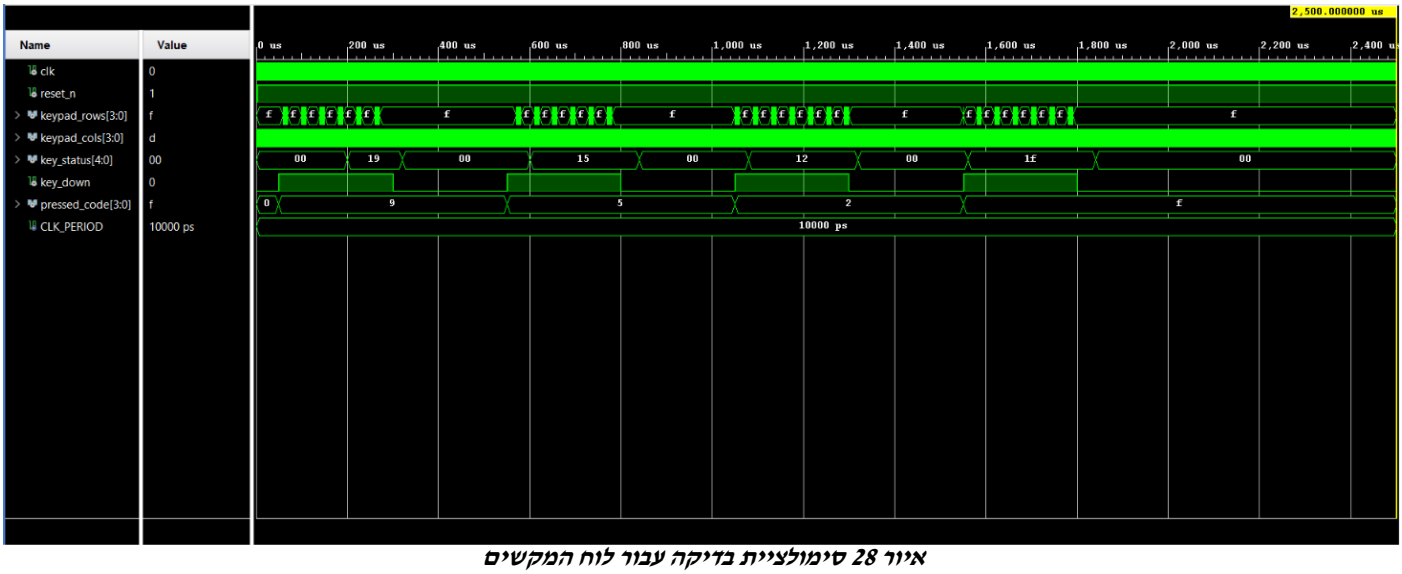
כאשר הופעלה פקודת הסיבוב עם כיוון השעון, התקבל רצף הפאזות המתאים לכיוון זה. לאחר מכן, כאשר הופעלה פקודת הסיבוב נגד כיוון השעון, השתנה סדר הפאזות בהתאם. בנוסף נבדקה תגובת המערכת לאות reset אשר החזיר את בקר המנוע למצב ההתחלתי ועצר את הפעלת הפאזות.

תוצאות הסימולציה הראו כי הבקר הראשי שילב בצורה תקינה את אותות הבקרה ואת מכונות המצבים של שני כיווני הסיבוב, וכי המוצא השתנה בהתאם לפקודה שניתנה



איור 27 סימולציה מנוע הצעד באופן מלא

9.2. בדיקת לוח המקשים



איור 28 סימולציית בדיקה עבור לוח המקשים

בתחילת הסימולציה בוצע איפוס לבקר באמצעות reset_n ולאחר שחרור האיפוס החל הבקר בסריקה מחזורית של ארבע עמודות לוח המקשים. כאשר לא נלחץ אף מקש התקבל ב־ keypad_rows הערך 1111 בהתאם לחיבור Pull-Up של השורות. במהלך כל לחיצה הופעל key_down ובמקביל הוגדר ב־ pressed_code המקש שנבדק. לאחר שהלחיצה נשארה יציבה במשך מספר מחזורי סריקה, מנגנון ה־ Debounce אישר אותה ובמוצא key_status התקבל קוד המקש יחד עם ביט Valid

עבור המקש 9 התקבל key_status = 19 כלומר 11001 כאשר הביט העליון מציין לחיצה תקפה ו־ 1001 מייצג את הספרה 9. עבור המקש 5 התקבל key_status = 15 עבור המקש 2 התקבל key_status = 12 ועבור המקש # התקבל

key_status = 1F לאחר שחרור כל מקש חזר key_status לערך 00 כלומר ביט ה־ Valid חזר ל־0 והבקר זיהה בצורה תקינה את סיום הלחיצה.

מתוצאות הסימולציה ניתן לקבוע כי בקר לוח המקשים פעל בהתאם לתכנון. סריקת המטריצה בוצעה בצורה תקינה, כל מקש זוהה בהתאם לשורה ולעמודה המתאימות, מנגנון ה־ Debounce מנע זיהוי לא יציב, ובמוצא התקבלו הקודים הנכונים עבור הרצף 9, 5, 2 ו־#. בכך אומתה תקינות בקר לוח המקשים והיכולת להעביר את קוד המקש באופן יציב אל צד ה־ PS לצורך המשך טיפול בתוכנת ה־ Python

```
============================================
Starting Final Smart Lock System
============================================
Loading overlay...
Overlay loaded successfully.
Available IP blocks:
- motor_control_gpio
- peripherals_gpio
- keypad_status_gpio
- processing_system7_0
GPIO blocks initialized successfully.
Loading Haar Cascade face detector...
Face detector loaded successfully.
Loading LBPH face recognition model...
LBPH model loaded successfully.
Outputs initialized. System state: LOCKED
Resetting motor controller...
Motor controller reset completed.
System is ready.
Waiting for PIR motion or keypad input.
Use keypad password: 952#
Keypad rule: wrong keypad code = 1 failed attempt.
Keypad rule: 3 wrong codes = intruder photo + Telegram + lockout.
Face rule: 3 failed face recognition events = 1 failed attempt + Telegram.
Door sensor: MC-38 on AR2; 0 = CLOSED, 1 = OPEN.
Automatic closing: after the door opens and closes.
Initial door state: CLOSED
Telegram alerts: ON
Stop the cell manually when finished.
============================================
```

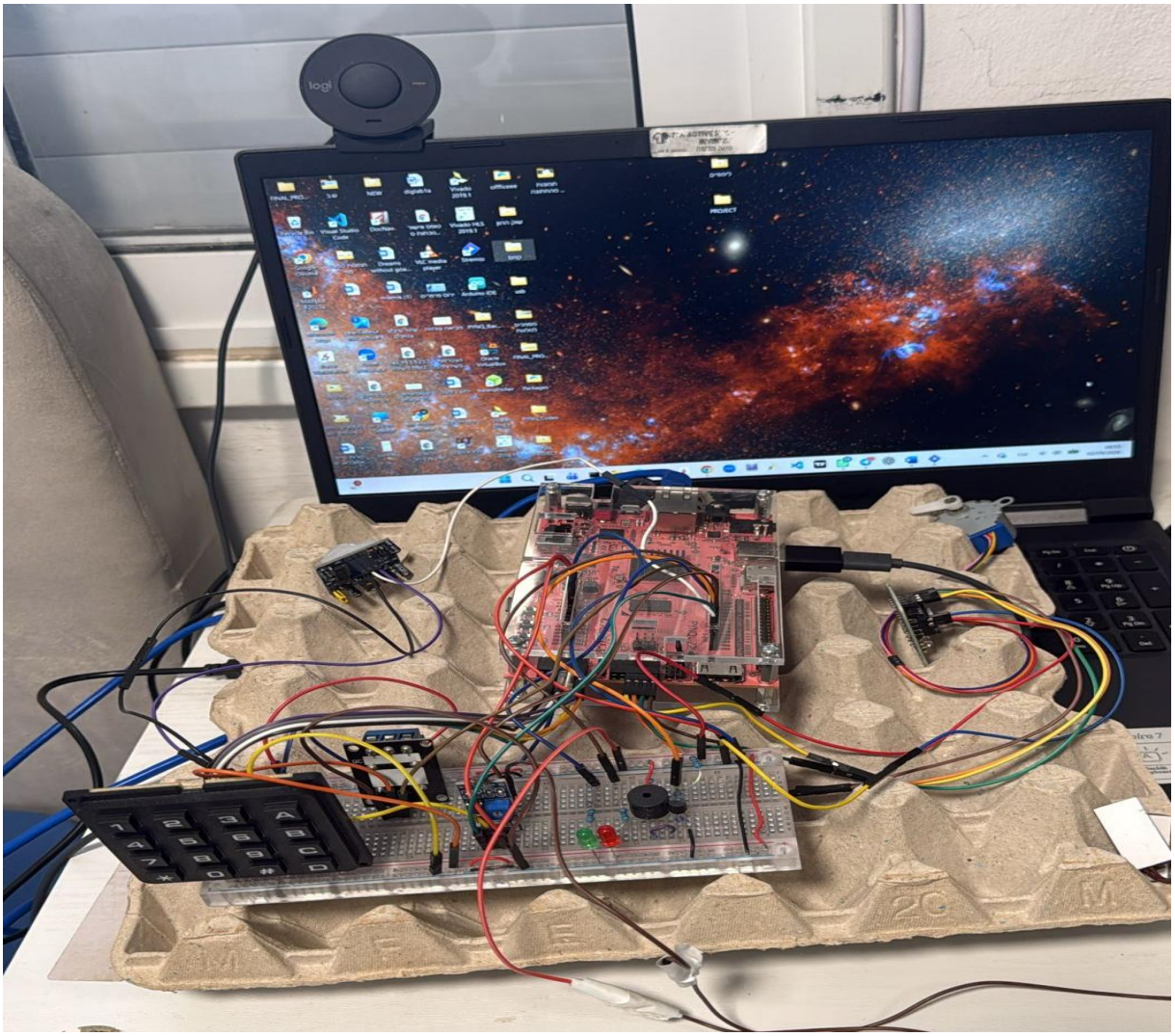
איור 29 תוצאות הבדיקה בסביבת JUPYTER

באיור מוצג תהליך אתחול המערכת לאחר הפעלת התוכנה הראשית. ניתן לראות כי ה־ Overlay ורכיבי ה־ GPIO נטענו בהצלחה, מודלי גילוי וזיהוי הפנים אותחלו, בקר המנוע אופס ומצב המערכת הוגדר כנעול. בנוסף, התקבלה אינדיקציה כי חיישן הדלת זיהה מצב סגור וכי מנגנון ההתראות באמצעות Telegram היה פעיל. לאחר השלמת האתחול עברה המערכת למצב המתנה לאירוע מחיישן התנועה או מלוח המקשים.

10. תוצאות ומסקנות

בפרק זה הוצגו התוצאות שהתקבלו לאחר השלמת שלבי המימוש והבדיקות של מערכת המנעול החכם. הבדיקות כללו סימולציות של החומרה שתוכננה ב־ VHDL בסביבת Vivado וכן בדיקות מעשיות של המערכת המלאה על גבי כרטיס PYNQ-Z2

10.1. תוצאות



איור 30 המערכת לאחר חיבור כול הרכיבים

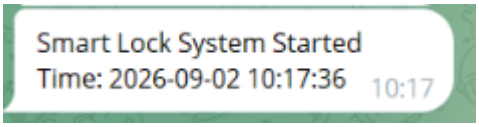
בסימולציות שבוצעו בסביבת Vivado נמצא כי מחלק התדר פעל בהתאם לתכנון והמיר את שעון המערכת בתדר MHz 125 לאות שעון בתדר Hz 500 בנוסף, נבדקו מכונות המצבים של מנוע הצעד ונמצא כי התקבלו רצפי הפאזות המתאימים לסיבוב עם כיוון השעון ולסיבוב נגד כיוון השעון. בבדיקת בקר המנוע המלא נמצא כי אותות הבקרה הפעילו את רצפי הפאזות המתאימים וכי אות האיפוס החזיר את הבקר למצב ההתחלתי.

לאחר מכן נבדקה המערכת המלאה על גבי כרטיס PYNQ-Z2 בעת הפעלת התוכנה נטען ה־ Overlay בהצלחה, זוהו רכיבי ה־ GPIO נטענו מודל Haar Cascade לגילוי פנים ומודל LBPH לזיהוי פנים, ואותחלו רכיבי הפלט ובקר מנוע הצעד. מצב המערכת ההתחלתי הוגדר כנעול, וחיישן הדלת MC-38 זוהה בהתאם למצב הפיזי של הדלת.

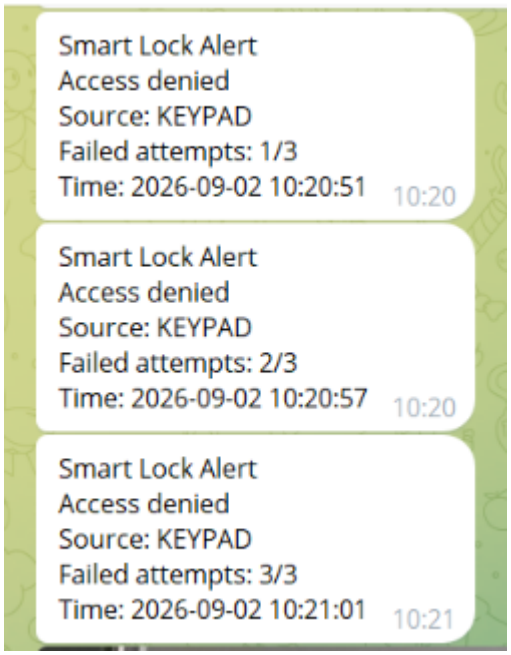
במהלך הבדיקות נמצא כי חיישן ה־ PIR זיהה תנועה באזור הדלת והפעיל את תהליך הצילום. חיישן האור זיהה את מצב התאורה, ובתנאי חשכה הופעל הממסר לצורך הדלקת התאורה לפני הצילום. מצלמת ה־ USB ביצעה צילום, ולאחר מכן בוצעו גילוי וזיהוי פנים באמצעות האלגוריתמים שנבחרו.

כאשר המשתמש המורשה זוהה בהצלחה, התקבלה גישה מאושרת והופעל מנגנון פתיחת המנעול. לאחר פתיחת הדלת עקבה המערכת אחר מצב חיישן MC-38 ורק לאחר זיהוי סגירת הדלת והשהיית הזמן שהוגדרה בוצעה פעולת הנעילה מחדש.

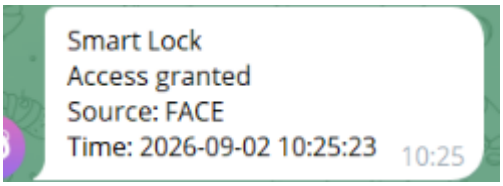
בנוסף, נבדקה פעולת לוח המקשים. הזנת הסיסמה הנכונה אפשרה פתיחת המנעול, בעוד שהזנת קוד שגוי נרשמה כניסיון גישה כושל. לאחר שלושה ניסיונות קוד שגויים הופעל מנגנון האבטחה שכלל צילום תמונה, שליחת התראה באמצעות Telegram והפעלת מצב Lockout למשך הזמן שהוגדר במערכת.



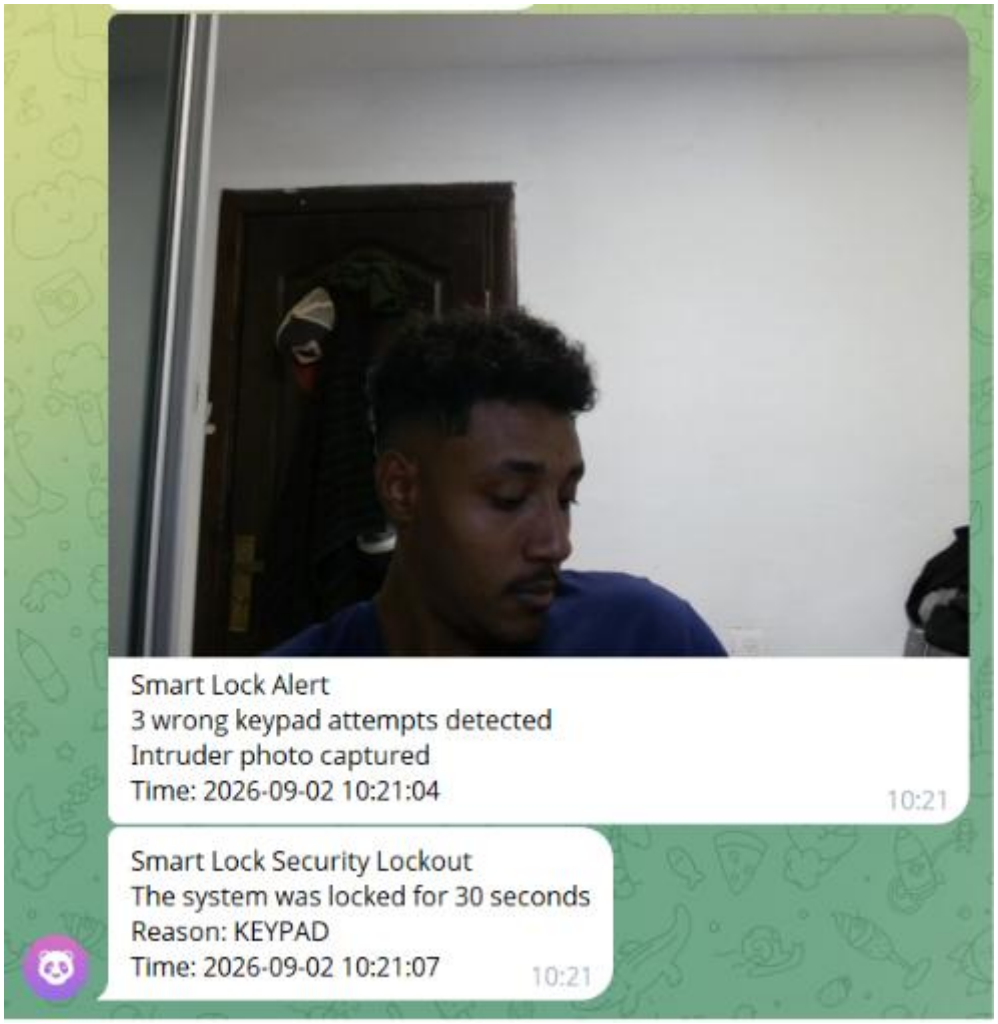
איור 31 בוט הטלגרם כאשר המערכת מופעלת



איור 32 שלוש הקלדות קוד שגויות



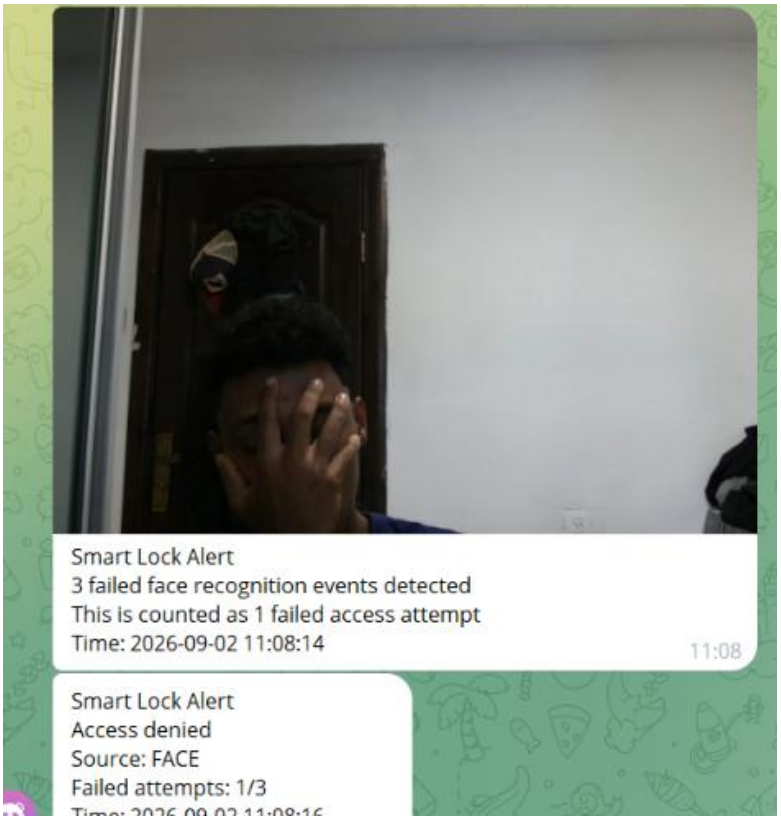
איור 33 זיהוי פנים תקין של המערכת



איור 34 נעילת המערכת לאחר 3 הקשות שגויות

באיור זה ניתן לראות שלאחר 3 הקלדות שגויות של קוד המערכת שולחת תמונה לטלגרם של אותו האדם שניסה ונועלת את המערכת ל30 שניות.

כאשר זוהתה תנועה באמצעות חיישן ה־ PIR אך תהליך זיהוי הפנים נכשל שלוש פעמים רצופות, בוצע צילום של אזור הכניסה ונשלחה התראה הכוללת את התמונה באמצעות Telegram שלושת כשלי הזיהוי נרשמו יחד כניסיון גישה כושל אחד. מנגנון זה נועד לצמצם רישום והתראות מיותרות במקרים שבהם זוהתה תנועה שאינה נגרמת מאדם, כגון תנועת בעלי חיים, ובכך להפחית עומס מיותר על המערכת.נמצא גם כי הזמזם ונורות החיווי סיפקו משוב בהתאם למצבי המערכת, וכי אירועי הגישה והאבטחה נשמרו בקובץ CSV לצורך תיעוד ומעקב.



איור 35 3 פעמים המערכת לא זיהת ושלחה התראה לטלגרם

בסך הכול, תוצאות הבדיקות הראו כי תתי־המערכות השונות פעלו בהתאם לתכנון וכי ניתן היה לשלב אותן למערכת אחת המבצעת תהליך בקרת כניסה מלא.

```
This is counted as one failed face recognition event.
Face recognition failed.
Face failed count: 1 / 3
Motion detected by PIR.
Light environment detected. Relay remains OFF.
Opening camera...
[ WARN:0] global ./modules/videoio/src/cap_gstreamer.cpp (616) isPipelinePlaying OpenCV | GStreamer warning: GStreamer: pipelin
e have not been created
Image captured successfully: smart_lock_captured.jpg
Number of faces detected: 0
No face detected in the captured image.
This is counted as one failed face recognition event.
Face recognition failed.
Face failed count: 2 / 3
Motion detected by PIR.
Light environment detected. Relay remains OFF.
Opening camera...
[ WARN:0] global ./modules/videoio/src/cap_gstreamer.cpp (616) isPipelinePlaying OpenCV | GStreamer warning: GStreamer: pipelin
e have not been created
Image captured successfully: smart_lock_captured.jpg
Number of faces detected: 0
No face detected in the captured image.
This is counted as one failed face recognition event.
Face recognition failed.
Face failed count: 3 / 3
Three failed face recognition events counted as one failed access attempt.
Sending Telegram alert only now.
Access denied.
Source: FACE
Failed attempts: 1 / 3
```

איור 36 המערכת כאשר 3 פעמים לא זוהה פנים

10.2. מסקנות

- השילוב בין ה־ PS לבין ה־ PL בכרטיס PYNQ-Z2 נמצא מתאים למימוש מערכת המנעול החכם, כאשר פעולות התוכנה והעיבוד בוצעו ב־ PS ובקר מנוע הצעד מומש ב־ FPGA
- השימוש בזיהוי פנים אפשר פתיחת מנעול ללא מגע פיזי, אך נמצא כי איכות הזיהוי הושפעה מתנאי התאורה, זווית הצילום ואיכות תמונות האימון .
- שילוב חיישן האור והממסר אפשר לשפר את תנאי הצילום בסביבה חשוכה באמצעות הפעלת תאורה לפני ביצוע הצילום .
- לוח המקשים סיפק אמצעי גיבוי יעיל לפתיחת המנעול כאשר זיהוי הפנים לא בוצע בהצלחה .
- מנגנון ספירת ניסיונות הגישה הכושלים והפעלת מצב Lockout שיפרו את רמת האבטחה והגבילו ניסיונות גישה חוזרים
- שלושה כשלי זיהוי פנים רצופים נרשמו כניסיון גישה כושל אחד. מנגנון זה צמצם התראות ורישומים מיותרים בעקבות תנועות שאינן בהכרח של אדם, כגון תנועת בעלי חיים .
- שילוב חיישן הדלת MC-38 שיפר את אמינות פעולת הנעילה, מאחר שהמנעול ננעל מחדש רק לאחר זיהוי סגירה פיזית של הדלת .
- מערכת ההתראות באמצעות Telegram אפשרה קבלת מידע על אירועים חריגים בזמן אמת, לרבות שליחת תמונה בהתאם לתנאים שהוגדרו במערכת.
- שמירת האירועים בקובץ CSV אפשרה תיעוד ומעקב אחר ניסיונות גישה, תוצאות אימות ומצבי מערכת .
- תוצאות הפרויקט הראו כי ניתן לשלב בהצלחה חומרה ב־ FPGA תוכנה בשפת ,Python חיישנים, ראייה ממוחשבת ותקשורת רשת למערכת בקרת כניסה אחת .
- מטרות הפרויקט הושגו, והמערכת אפשרה זיהוי משתמש מורשה, פתיחת מנעול, בקרה על מצב הדלת, גיבוי באמצעות סיסמה, ניהול ניסיונות גישה ושליחת התראות .
- בעתיד ניתן להרחיב את המערכת באמצעות אלגוריתמי זיהוי פנים מתקדמים יותר, תמיכה במספר משתמשים מורשים, ממשק ניהול מרחוק ושיפור מנגנוני האבטחה והתיעוד, הגדלת אימון האלגוריתם לזיהוי פנים

11. סיכום

בפרויקט זה תוכננה ומומשה מערכת מנעול חכם המבוססת על כרטיס הפיתוח PYNQ-Z2 אשר שילבה בין חומרה מתוכנתת, תוכנה משובצת, עיבוד תמונה, חיישנים ותקשורת. מטרת המערכת הייתה לאפשר בקרת כניסה חכמה ומאובטחת באמצעות זיהוי פנים, תוך מתן אפשרות גיבוי לפתיחת המנעול באמצעות סיסמה בלוח מקשים.

מימוש המערכת התבסס על שילוב בין חלק ה־ PS לבין חלק ה־ PL של רכיב ה־ Zynq-7000 בצד ה־ FPGA תוכנן בקר מנוע צעד בשפת VHDL אשר שלט בכיוון הסיבוב של מנוע צעד באמצעות דרייבר ULN2003 לצורך יצירת קצב העבודה המתאים למנוע תוכנן מחלק תדר, והחומרה שנבנתה שולבה עם תוכנת הבקרה באמצעות ממשקי AXI GPIO

בצד התוכנה פותחה מערכת בקרה בשפת Python אשר ניהלה את כלל תהליך הגישה. לאחר זיהוי תנועה באמצעות חיישן PIR נבדקו תנאי התאורה, ובמידת הצורך הופעלה תאורה באמצעות ממסר. לאחר מכן בוצע צילום באמצעות מצלמת Logitech Brio 300 גילוי פנים באמצעות Haar Cascade וזיהוי פנים באמצעות אלגוריתם LBPH כאשר זוהה משתמש מורשה, הופעל מנגנון פתיחת המנעול .

לאחר פתיחת המנעול בוצע מעקב אחר מצב הדלת באמצעות חיישן MC-38 הנעילה מחדש בוצעה רק לאחר שהדלת זוהתה כסגורה, ובכך נמנע מצב שבו מנגנון הנעילה הופעל כאשר הדלת עדיין פתוחה. בנוסף, שולב לוח מקשים כאמצעי גיבוי, וכן מומשו נורות חיווי וזמזם לצורך מתן משוב למשתמש.

לצורך שיפור רמת האבטחה מומש מנגנון לניהול ניסיונות גישה כושלים. ניסיונות סיסמה שגויים נספרו, ולאחר שלושה ניסיונות הופעל מצב Lockout בוצע צילום ונשלחה התראה באמצעות Telegram גם כשלי זיהוי פנים נוהלו באופן מבוקר, כך ששלושה כשלי זיהוי רצופים טופלו כאירוע כושל אחד, במטרה לצמצם התראות מיותרות כתוצאה מזיהוי תנועה שאינה בהכרח של אדם. בנוסף, אירועי המערכת נשמרו בקובץ CSV לצורך תיעוד ומעקב .

המערכת נבדקה הן באמצעות סימולציות בסביבת Vivado והן בבדיקות מעשיות על גבי כרטיס PYNQ-Z2 תוצאות הבדיקות הראו כי מחלק התדר ובקר מנוע הצעד פעלו בהתאם לתכנון, וכי תתי־המערכות השונות שולבו בהצלחה למערכת אחת. בסיום הפרויקט התקבלה מערכת בקרת כניסה חכמה אשר שילבה זיהוי פנים, סיסמת גיבוי, בקרה על מצב הדלת, התאמה לתנאי תאורה, ניהול אירועי אבטחה ושליחת התראות בזמן אמת. בכך הושגו מטרות הפרויקט והודגם מימוש משולב של FPGA תוכנה משובצת, ראייה ממוחשבת וחיישנים במערכת אבטחה אחת.

12. רשימות

12.1. רשימת טבלאות

טבלה **1** רצף לוגי להפעל מנוע צעד נגד כיוון השעון 15
טבלה **2** רצף לוגי להפעל מנוע צעד עם כיוון השעון 15
טבלה **3** רשימת ציוד חומרה 30
טבלה **4** רשימת ציוד תוכנה 31
טבלה **5** חיבורים של המערכת 34

12.2. רשימת איורים

איור **1** פירוט החיבורים על כרטיס פיתוח PYNQ-Z2 9
איור **2** דיאגרמה של ZYNQ 7000 10
איור **3** פרוטוקול AXI 11
איור **4** AXI GPIO 12
איור **5** במנוע צעד מסוג 28BYJ-48 14
איור **6** דרייבר ULN2003 15
איור **7** חיישן תנועה PIR 16
איור **8** חיישן אור דיגיטלי LM393 17
איור **9** גרף התנגדות בהתאם לעוצמת האור 17
איור **10** חיישן דלת מגנטי MC-38 18

איור **11** מצלמת LOGITECH BRIO 300 USB 19
איור **12** לוח מקשים 4 על 4 20
איור **13** דיאגרמה של לוח מקשים 4 על 4 20
איור **14** SINGLE CHANNEL RELAY MODULE 21
איור **15** זמזם 22
איור **16** נורות LED 22
איור **17** ייצוג תמונת פנים דיגיטלית כמטריצת ערכי פיקסלים 23
איור **18** תהליך גילוי פנים באלגוריתם HAAR CASCADE 26
איור **19** דיאגרמה עבור אלגורתים LBPH 27
איור **20** סכמת בלוקים של כלל המערכת 32
איור **21** תרשים חיבורים כללי של המערכת 33
איור **22** מבנה מערכת החומרה ב VIVADO 50
איור **23** תרשים זרימה של התוכנה 96
איור **24** תוצאת סימולצייה של מחלק התדר 97
איור **25** תוצאת סימולציה של המנוע בסיבוב עם כיוון השעון 98
איור **26** תוצאות סימולציה של המנוע עם סיבוב נגד כיוון השעון 99
איור **27** סימולציה מנוע הצעד באופן מלא 99
איור **28** סימולציית בדיקה עבור לוח המקשים 100
איור **29** תוצאות הבדיקה בסביבת JUPYTER 101
איור **30** המערכת לאחר חיבור כול הרכיבים 102
איור **31** בוט הטלגרם באשר המערכת מופעלת 103
איור **32** שלוש הקלדות קוד שגויות 103
איור **33** זיהוי פנים תקין של המערכת 103
איור **34** נעילת המערכת לאחר 3 הקשות שגויות 104
איור **35** 3 פעמים המערכת לא זיהת ושלחה התראה לטלגרם 105
איור **36** המערכת באשר 3 פעמים לא זוהה פנים 105

13. ביבליוגרפיה

[1] PYNQ Project, “PYNQ-Z2 Board and Overlays,” *Python Productivity for Zynq Documentation*. [Online]. Available: https://pynq.readthedocs.io/en/latest/pynq_overlays/pynqz2.html.

[2] Advanced Micro Devices, Inc., *Zynq-7000 SoC Technical Reference Manual*, UG585, Version 1.15, Feb. 2026. [Online]. Available: https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM.

[3] Advanced Micro Devices, Inc., *AXI GPIO LogiCORE IP Product Guide*, PG144. [Online]. Available: https://docs.amd.com/r/en-US/pg144-axi-gpio.

[4] OpenCV, “OpenCV Documentation,” *Open Source Computer Vision Library*. [Online]. Available: https://docs.opencv.org/4.x/.

[5] P. Viola and M. Jones, “Rapid Object Detection Using a Boosted Cascade of Simple Features,” in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 1, pp. I-511–I-518, 2001, doi: 10.1109/CVPR.2001.990517.

[6] Telegram, “Telegram Bot API.” [Online]. Available: https://core.telegram.org/bots/api.

[7] Texas Instruments, *ULN200x, ULQ200x High-Voltage, High-Current Darlington Transistor Arrays*, Rev. T, Mar. 2025. [Online]. Available: https://www.ti.com/product/ULN2003A.

[8] MikroElektronika, *28BYJ-48 5V Stepper Motor Datasheet*. [Online]. Available: https://www.digikey.com.br/htmldatasheets/production/1839399/0/0/1/28byj-48.html.